

Football Manager

Sprint 3

Generated by Doxygen 1.13.2

1 Football Manager	1
1.1 Main Features	1
1.2 Technologies Used	2
1.3 Project Structure	2
1.4 Requirements	3
1.5 Database Configuration	3
1.6 How to Run the Application	3
1.6.1 Windows	3
1.6.2 Linux / macOS	3
1.7 How to Run Tests	3
1.7.1 Windows	3
1.7.2 Linux / macOS	3
1.8 Code Coverage with JaCoCo	3
1.8.1 Windows	4
1.8.2 Linux / macOS	4
1.9 Performance Testing with JUnitPerf	4
1.9.1 Windows	4
1.9.2 Linux / macOS	4
1.10 Jenkins CI Pipeline	4
1.11 Doxygen Documentation	5
1.12 Swagger/OpenAPI Documentation	5
1.13 Maven Site	5
1.13.1 Windows	5
1.13.2 Linux / macOS	6
1.14 Main API Endpoints	6
1.15 Web Views	6
1.16 Testing Strategy	6
1.17 Sprint 2 Evidence	7
1.18 Sprint 3 Evidence	7
1.19 How to Package the Application	7
1.19.1 Windows	7
1.19.2 Linux / macOS	8
1.20 Authors	8
1.21 Repository	8
1.22 Project Status	8
2 doxygen_mainpage	9
3 Namespace Index	11
3.1 Package List	11
4 Hierarchical Index	13
4.1 Class Hierarchy	13

5 Class Index	15
5.1 Class List	15
6 File Index	17
6.1 File List	17
7 Namespace Documentation	19
7.1 Package com.example.football_manager	19
7.2 Package com.example.football_manager.config	19
7.3 Package com.example.football_manager.controller	19
7.4 Package com.example.football_manager.dto	20
7.4.1 Function Documentation	20
7.4.1.1 MatchResultDTO()	20
7.5 Package com.example.football_manager.model	20
7.6 Package com.example.football_manager.repository	21
7.7 Package com.example.football_manager.service	21
8 Class Documentation	23
8.1 com.example.football_manager.controller.AdminCountryViewController Class Reference	23
8.1.1 Detailed Description	23
8.1.2 Constructor & Destructor Documentation	23
8.1.2.1 AdminCountryViewController()	23
8.1.3 Member Function Documentation	23
8.1.3.1 countriesPage()	23
8.1.3.2 createCountry()	23
8.1.3.3 deleteCountry()	24
8.2 com.example.football_manager.controller.AdminPlayerViewController Class Reference	24
8.2.1 Detailed Description	24
8.2.2 Constructor & Destructor Documentation	24
8.2.2.1 AdminPlayerViewController()	24
8.2.3 Member Function Documentation	24
8.2.3.1 createPlayer()	24
8.2.3.2 deletePlayer()	25
8.2.3.3 managePlayersLanding()	25
8.2.3.4 manageTeamPlayers()	25
8.2.3.5 showEditPlayerForm()	25
8.2.3.6 updatePlayer()	25
8.3 com.example.football_manager.config.AdminSecurityFilter Class Reference	26
8.3.1 Detailed Description	26
8.3.2 Member Function Documentation	26
8.3.2.1 doFilter()	26
8.4 com.example.football_manager.controller.AdminUserViewController Class Reference	26
8.4.1 Detailed Description	26

8.4.2 Constructor & Destructor Documentation	26
8.4.2.1 AdminUserController()	26
8.4.3 Member Function Documentation	27
8.4.3.1 createUser()	27
8.4.3.2 usersPage()	27
8.5 com.example.football_manager.controller.AdminViewController Class Reference	27
8.5.1 Detailed Description	27
8.5.2 Member Function Documentation	27
8.5.2.1 adminPanel()	27
8.5.2.2 manageTeams()	27
8.6 com.example.football_manager.controller.AuthController Class Reference	28
8.6.1 Detailed Description	28
8.6.2 Member Function Documentation	28
8.6.2.1 login()	28
8.6.2.2 register()	28
8.7 com.example.football_manager.dto.AuthResponseDTO Class Reference	28
8.7.1 Detailed Description	28
8.7.2 Constructor & Destructor Documentation	28
8.7.2.1 AuthResponseDTO()	28
8.7.3 Member Function Documentation	29
8.7.3.1 getEmail()	29
8.7.3.2 getId()	29
8.7.3.3 getIsAdmin()	29
8.7.3.4 getMessage()	29
8.7.3.5 getUsername()	29
8.8 com.example.football_manager.controller.AuthViewController Class Reference	29
8.8.1 Detailed Description	29
8.8.2 Member Function Documentation	29
8.8.2.1 index()	29
8.8.2.2 loginPage()	29
8.8.2.3 registerPage()	30
8.9 com.example.football_manager.config.CacheConfig Class Reference	30
8.9.1 Detailed Description	30
8.9.2 Member Function Documentation	30
8.9.2.1 cacheManager()	30
8.10 com.example.football_manager.model.Competition Class Reference	30
8.10.1 Detailed Description	30
8.11 com.example.football_manager.controller.CompetitionController Class Reference	30
8.11.1 Detailed Description	30
8.11.2 Constructor & Destructor Documentation	31
8.11.2.1 CompetitionController()	31
8.11.3 Member Function Documentation	31

8.11.3.1 createCompetition()	31
8.11.3.2 deleteCompetition()	31
8.11.3.3 getAllCompetitions()	31
8.11.3.4 getCompetitionById()	31
8.11.3.5 updateCompetition()	31
8.12 com.example.football_manager.repository.CompetitionRepository Interface Reference	31
8.12.1 Detailed Description	32
8.13 com.example.football_manager.dto.CompetitionRequestDTO Class Reference	32
8.13.1 Detailed Description	32
8.13.2 Constructor & Destructor Documentation	32
8.13.2.1 CompetitionRequestDTO() [1/2]	32
8.13.2.2 CompetitionRequestDTO() [2/2]	32
8.13.3 Member Function Documentation	32
8.13.3.1 getName()	32
8.13.3.2 setName()	32
8.14 com.example.football_manager.service.CompetitionService Class Reference	32
8.14.1 Detailed Description	33
8.14.2 Constructor & Destructor Documentation	33
8.14.2.1 CompetitionService()	33
8.14.3 Member Function Documentation	33
8.14.3.1 createCompetition()	33
8.14.3.2 deleteCompetition()	33
8.14.3.3 getAllCompetitions()	33
8.14.3.4 getCompetitionById()	33
8.14.3.5 updateCompetition()	33
8.15 com.example.football_manager.model.Country Class Reference	33
8.15.1 Detailed Description	33
8.16 com.example.football_manager.controller.CountryController Class Reference	34
8.16.1 Detailed Description	34
8.16.2 Constructor & Destructor Documentation	34
8.16.2.1 CountryController()	34
8.16.3 Member Function Documentation	34
8.16.3.1 createCountry()	34
8.16.3.2 deleteCountry()	34
8.16.3.3 getAllCountries()	34
8.17 com.example.football_manager.repository.CountryRepository Interface Reference	34
8.17.1 Detailed Description	35
8.18 com.example.football_manager.dto.CountryRequestDTO Class Reference	35
8.18.1 Detailed Description	35
8.18.2 Constructor & Destructor Documentation	35
8.18.2.1 CountryRequestDTO() [1/2]	35
8.18.2.2 CountryRequestDTO() [2/2]	35

8.18.3 Member Function Documentation	35
8.18.3.1 getName()	35
8.18.3.2 setName()	35
8.19 com.example.football_manager.service.CountryService Class Reference	35
8.19.1 Detailed Description	36
8.19.2 Constructor & Destructor Documentation	36
8.19.2.1 CountryService()	36
8.19.3 Member Function Documentation	36
8.19.3.1 createCountry()	36
8.19.3.2 deleteCountry()	36
8.19.3.3 getAllCountries()	36
8.20 com.example.football_manager.controller.FavouriteTeamController Class Reference	36
8.20.1 Detailed Description	36
8.20.2 Constructor & Destructor Documentation	36
8.20.2.1 FavouriteTeamController()	36
8.20.3 Member Function Documentation	37
8.20.3.1 addFavouriteTeam()	37
8.20.3.2 removeFavouriteTeam()	37
8.21 com.example.football_manager.FootballManagerApplication Class Reference	37
8.21.1 Detailed Description	37
8.21.2 Member Function Documentation	37
8.21.2.1 main()	37
8.22 com.example.football_manager.controller.GlobalModelAttributes Class Reference	37
8.22.1 Detailed Description	37
8.22.2 Member Function Documentation	37
8.22.2.1 addIsAdmin()	37
8.23 com.example.football_manager.dto.MatchResultRequestDTO.GoalDTO Class Reference	38
8.23.1 Detailed Description	38
8.23.2 Member Function Documentation	38
8.23.2.1 getMinute()	38
8.23.2.2 getStoppageMinute()	38
8.23.2.3 getTeamId()	38
8.23.2.4 setMinute()	38
8.23.2.5 setStoppageMinute()	38
8.23.2.6 setTeamId()	38
8.24 com.example.football_manager.config.LocaleConfig Class Reference	39
8.24.1 Detailed Description	39
8.24.2 Member Function Documentation	39
8.24.2.1 addInterceptors()	39
8.24.2.2 localeChangeInterceptor()	39
8.24.2.3 localeResolver()	39
8.25 com.example.football_manager.dto.LoginDTO Class Reference	39

8.25.1 Detailed Description	39
8.25.2 Member Function Documentation	40
8.25.2.1 getPassword()	40
8.25.2.2 getUsername()	40
8.25.2.3 setPassword()	40
8.25.2.4 setUsername()	40
8.26 com.example.football_manager.model.Match Class Reference	40
8.26.1 Detailed Description	40
8.27 com.example.football_manager.controller.MatchController Class Reference	40
8.27.1 Detailed Description	40
8.27.2 Member Function Documentation	41
8.27.2.1 createMatchFromForm()	41
8.27.2.2 createMatchFromJson()	41
8.27.2.3 deleteMatch()	41
8.27.2.4 getMatches()	41
8.27.2.5 getMatchResults()	41
8.27.2.6 getUpcomingMatches()	41
8.27.2.7 registerResult()	41
8.27.2.8 updateMatch()	41
8.28 com.example.football_manager.model.MatchGoal Class Reference	42
8.28.1 Detailed Description	42
8.29 com.example.football_manager.repository.MatchGoalRepository Interface Reference	42
8.29.1 Detailed Description	42
8.29.2 Member Function Documentation	42
8.29.2.1 deleteByMatchId()	42
8.29.2.2 findByMatchIdOrderByMinuteAscStoppageMinuteAscIdAsc()	42
8.30 com.example.football_manager.repository.MatchRepository Interface Reference	42
8.30.1 Detailed Description	43
8.30.2 Member Function Documentation	43
8.30.2.1 findByFilters()	43
8.30.2.2 findByFinishedFalseAndDatetimeAfterAndLeftTeamIdInOrFinishedFalseAndDatetimeAfterAndRightTeamIdO	
8.30.2.3 findByFinishedFalseAndDatetimeAfterAndLeftTeamIdOrFinishedFalseAndDatetimeAfterAndRightTeamIdO	
8.30.2.4 findByFinishedTrueAndLeftTeamIdOrFinishedTrueAndRightTeamIdOrderByDatetimeDesc()	44
8.30.2.5 findByFinishedTrueOrderByDatetimeDesc()	44
8.31 com.example.football_manager.dto.MatchRequestDTO Class Reference	44
8.31.1 Detailed Description	44
8.31.2 Member Function Documentation	44
8.31.2.1 getAwayScore()	44
8.31.2.2 getAwayTeamId()	45
8.31.2.3 getCompetitionId()	45
8.31.2.4 getHomeScore()	45
8.31.2.5 getHomeTeamId()	45

8.31.2.6 getKickoffTime()	45
8.31.2.7 getStatus()	45
8.31.2.8 getVenue()	45
8.31.2.9 setAwayScore()	45
8.31.2.10 setAwayTeamId()	45
8.31.2.11 setCompetitionId()	45
8.31.2.12 setHomeScore()	46
8.31.2.13 setHomeTeamId()	46
8.31.2.14 setKickoffTime()	46
8.31.2.15 setStatus()	46
8.31.2.16 setVenue()	46
8.32 com.example.football_manager.dto.MatchResultRequestDTO Class Reference	46
8.32.1 Detailed Description	46
8.32.2 Member Function Documentation	46
8.32.2.1 getGoals()	46
8.32.2.2 setGoals()	47
8.33 com.example.football_manager.service.MatchService Class Reference	47
8.33.1 Detailed Description	47
8.33.2 Constructor & Destructor Documentation	47
8.33.2.1 MatchService()	47
8.33.3 Member Function Documentation	47
8.33.3.1 createMatch()	47
8.33.3.2 deleteMatch()	48
8.33.3.3 getAllMatches()	48
8.33.3.4 getFinishedMatchResults()	48
8.33.3.5 getFutureMatchesByFavouriteTeamIds()	48
8.33.3.6 getFutureMatchesByTeamId()	48
8.33.3.7 getGoalsByMatchId()	48
8.33.3.8 getMatchById()	48
8.33.3.9 getMatches()	48
8.33.3.10 getMatchForEdit()	48
8.33.3.11 getRecentMatchesByTeamId()	49
8.33.3.12 getUpcomingMatches()	49
8.33.3.13 registerResult()	49
8.33.3.14 updateMatch()	49
8.34 com.example.football_manager.dto.MatchRequestDTO.MatchStatus Enum Reference	49
8.34.1 Detailed Description	49
8.34.2 Member Data Documentation	49
8.34.2.1 CANCELLED	49
8.34.2.2 FINISHED	50
8.34.2.3 IN_PROGRESS	50
8.34.2.4 SCHEDULED	50

8.35 com.example.football_manager.controller.MatchViewController Class Reference	50
8.35.1 Detailed Description	50
8.35.2 Constructor & Destructor Documentation	50
8.35.2.1 MatchViewController()	50
8.35.3 Member Function Documentation	51
8.35.3.1 saveMatchResult()	51
8.35.3.2 showEditForm()	51
8.35.3.3 showMatchesPage()	51
8.35.3.4 showResultsPage()	51
8.35.3.5 showScheduleForm()	51
8.35.3.6 showScoreForm()	51
8.35.3.7 showUpcomingMatchesPage()	51
8.35.3.8 submitScoreForm()	52
8.35.3.9 updateMatch()	52
8.36 com.example.football_manager.config.OpenApiConfig Class Reference	52
8.36.1 Detailed Description	52
8.37 com.example.football_manager.model.Player Class Reference	52
8.37.1 Detailed Description	52
8.38 com.example.football_manager.dto.PlayerBulkRequestDTO Class Reference	52
8.38.1 Detailed Description	53
8.38.2 Constructor & Destructor Documentation	53
8.38.2.1 PlayerBulkRequestDTO() [1/2]	53
8.38.2.2 PlayerBulkRequestDTO() [2/2]	53
8.38.3 Member Function Documentation	53
8.38.3.1 getPlayers()	53
8.38.3.2 setPlayers()	53
8.39 com.example.football_manager.controller.PlayerController Class Reference	53
8.39.1 Detailed Description	53
8.39.2 Constructor & Destructor Documentation	53
8.39.2.1 PlayerController()	53
8.39.3 Member Function Documentation	54
8.39.3.1 createPlayer()	54
8.39.3.2 createPlayers()	54
8.39.3.3 deletePlayer()	54
8.39.3.4 getPlayersByTeamId()	54
8.39.3.5 updatePlayer()	54
8.40 com.example.football_manager.model.PlayerPosition Enum Reference	54
8.40.1 Detailed Description	54
8.40.2 Member Data Documentation	55
8.40.2.1 DEFENDER	55
8.40.2.2 FORWARD	55
8.40.2.3 GOALKEEPER	55

8.40.2.4 MIDFIELDER	55
8.41 com.example.football_manager.repository.PlayerRepository Interface Reference	55
8.41.1 Detailed Description	55
8.41.2 Member Function Documentation	55
8.41.2.1 existsByTeamIdAndNumber()	55
8.41.2.2 existsByTeamIdAndNumberAndIdNot()	56
8.41.2.3 findByIdAndTeamId()	56
8.41.2.4 findByIdTeamIdOrderByNumberAsc()	56
8.42 com.example.football_manager.dto.PlayerRequestDTO Class Reference	56
8.42.1 Detailed Description	56
8.42.2 Constructor & Destructor Documentation	56
8.42.2.1 PlayerRequestDTO() [1/2]	56
8.42.2.2 PlayerRequestDTO() [2/2]	56
8.42.3 Member Function Documentation	57
8.42.3.1 getName()	57
8.42.3.2 getNumber()	57
8.42.3.3 getPosition()	57
8.42.3.4 setName()	57
8.42.3.5 setNumber()	57
8.42.3.6 setPosition()	57
8.43 com.example.football_manager.service.PlayerService Class Reference	57
8.43.1 Detailed Description	57
8.43.2 Constructor & Destructor Documentation	58
8.43.2.1 PlayerService()	58
8.43.3 Member Function Documentation	58
8.43.3.1 createPlayer()	58
8.43.3.2 createPlayers()	58
8.43.3.3 deletePlayer()	58
8.43.3.4 getPlayerByTeamId()	58
8.43.3.5 getPlayersByTeamId()	58
8.43.3.6 updatePlayer()	58
8.44 com.example.football_manager.dto.ProfileUpdateDTO Class Reference	59
8.44.1 Detailed Description	59
8.44.2 Member Function Documentation	59
8.44.2.1 isPasswordConfirmationValid()	59
8.45 com.example.football_manager.controller.ProfileViewController Class Reference	59
8.45.1 Detailed Description	59
8.45.2 Constructor & Destructor Documentation	59
8.45.2.1 ProfileViewController()	59
8.45.3 Member Function Documentation	59
8.45.3.1 profilePage()	59
8.45.3.2 updateProfile()	60

8.46 com.example.football_manager.dto.RegisterDTO Class Reference	60
8.46.1 Detailed Description	60
8.46.2 Member Function Documentation	60
8.46.2.1 getEmail()	60
8.46.2.2 getIsAdmin()	60
8.46.2.3 getPassword()	60
8.46.2.4 getUsername()	60
8.46.2.5 setEmail()	60
8.46.2.6 setIsAdmin()	61
8.46.2.7 setPassword()	61
8.46.2.8 setUsername()	61
8.47 com.example.football_manager.config.SecurityConfig Class Reference	61
8.47.1 Detailed Description	61
8.47.2 Member Function Documentation	61
8.47.2.1 filterChain()	61
8.47.2.2 passwordEncoder()	61
8.48 com.example.football_manager.dto.StandingDTO Class Reference	61
8.48.1 Detailed Description	62
8.48.2 Constructor & Destructor Documentation	62
8.48.2.1 StandingDTO() [1/2]	62
8.48.2.2 StandingDTO() [2/2]	62
8.48.3 Member Function Documentation	62
8.48.3.1 getDrawn()	62
8.48.3.2 getGoalDifference()	63
8.48.3.3 getGoalsAgainst()	63
8.48.3.4 getGoalsFor()	63
8.48.3.5 getLogoUrl()	63
8.48.3.6 getLost()	63
8.48.3.7 getMatchesPlayed()	63
8.48.3.8 getPoints()	63
8.48.3.9 getTeamName()	63
8.48.3.10 getWon()	63
8.48.3.11 setDrawn()	63
8.48.3.12 setGoalDifference()	63
8.48.3.13 setGoalsAgainst()	64
8.48.3.14 setGoalsFor()	64
8.48.3.15 setLogoUrl()	64
8.48.3.16 setLost()	64
8.48.3.17 setMatchesPlayed()	64
8.48.3.18 setPoints()	64
8.48.3.19 setTeamName()	64
8.48.3.20 setWon()	64

8.49 com.example.football_manager.controller.StandingsController Class Reference	64
8.49.1 Detailed Description	65
8.49.2 Constructor & Destructor Documentation	65
8.49.2.1 StandingsController()	65
8.49.3 Member Function Documentation	65
8.49.3.1 getStandings()	65
8.50 com.example.football_manager.service.StandingsService Class Reference	65
8.50.1 Detailed Description	65
8.50.2 Constructor & Destructor Documentation	65
8.50.2.1 StandingsService()	65
8.50.3 Member Function Documentation	65
8.50.3.1 getCompetitions()	65
8.50.3.2 getStandingsForCompetition()	66
8.51 com.example.football_manager.controller.StandingsViewController Class Reference	66
8.51.1 Detailed Description	66
8.51.2 Constructor & Destructor Documentation	66
8.51.2.1 StandingsViewController()	66
8.51.3 Member Function Documentation	66
8.51.3.1 standingsPage()	66
8.52 com.example.football_manager.model.Team Class Reference	66
8.52.1 Detailed Description	66
8.52.2 Constructor & Destructor Documentation	67
8.52.2.1 Team()	67
8.53 com.example.football_manager.controller.TeamController Class Reference	67
8.53.1 Detailed Description	67
8.53.2 Constructor & Destructor Documentation	67
8.53.2.1 TeamController()	67
8.53.3 Member Function Documentation	67
8.53.3.1 createTeam()	67
8.53.3.2 deleteTeam()	67
8.53.3.3 getAllTeams()	67
8.53.3.4 getTeamById()	68
8.53.3.5 getTeamsContainingName()	68
8.53.3.6 updateTeam()	68
8.54 com.example.football_manager.repository.TeamRepository Interface Reference	68
8.54.1 Detailed Description	68
8.54.2 Member Function Documentation	68
8.54.2.1 findByNameContainingIgnoreCase()	68
8.55 com.example.football_manager.dto.TeamRequestDTO Class Reference	69
8.55.1 Detailed Description	69
8.55.2 Constructor & Destructor Documentation	69
8.55.2.1 TeamRequestDTO() [1/2]	69

8.55.2.2 TeamRequestDTO() [2/2]	69
8.55.3 Member Function Documentation	69
8.55.3.1 getCountryId()	69
8.55.3.2 getLogoUrl()	69
8.55.3.3 getName()	69
8.55.3.4 setCountryId()	69
8.55.3.5 setLogoUrl()	70
8.55.3.6 setName()	70
8.56 com.example.football_manager.service.TeamService Class Reference	70
8.56.1 Detailed Description	70
8.56.2 Constructor & Destructor Documentation	70
8.56.2.1 TeamService()	70
8.56.3 Member Function Documentation	70
8.56.3.1 createTeam()	70
8.56.3.2 deleteTeam()	70
8.56.3.3 getAllTeams()	71
8.56.3.4 getTeamById()	71
8.56.3.5 getTeamsContainingName()	71
8.56.3.6 updateTeam()	71
8.57 com.example.football_manager.controller.TeamViewController Class Reference	71
8.57.1 Detailed Description	71
8.57.2 Member Function Documentation	71
8.57.2.1 createTeam()	71
8.57.2.2 deleteTeam()	72
8.57.2.3 showAddTeamForm()	72
8.57.2.4 showEditTeamForm()	72
8.57.2.5 teamDetailsPage()	72
8.57.2.6 teamsPage()	72
8.57.2.7 updateTeam()	72
8.58 com.example.football_manager.model.User Class Reference	73
8.58.1 Detailed Description	73
8.59 com.example.football_manager.repository.UserRepository Interface Reference	73
8.59.1 Detailed Description	73
8.59.2 Member Function Documentation	73
8.59.2.1 existsByEmail()	73
8.59.2.2 existsByEmailAndIdNot()	73
8.59.2.3 existsByUsername()	73
8.59.2.4 existsByUsernameAndIdNot()	74
8.59.2.5 findByEmail()	74
8.59.2.6 findByUsername()	74
8.59.2.7 findWithFavouriteTeamsById()	74
8.60 com.example.football_manager.service.UserService Class Reference	74

8.60.1 Detailed Description	74
8.60.2 Member Function Documentation	74
8.60.2.1 addFavouriteTeam()	74
8.60.2.2 getFavouriteTeamIdsByUserId()	75
8.60.2.3 getUserById()	75
8.60.2.4 getUserByUsername()	75
8.60.2.5 getUserProfile()	75
8.60.2.6 login()	75
8.60.2.7 registerUser()	75
8.60.2.8 removeFavouriteTeam()	75
8.60.2.9 updateProfile()	75
9 File Documentation	77
9.1 docs/doxygen_mainpage.md File Reference	77
9.2 Football_Manager/src/main/java/com/example/football_manager/config/AdminSecurityFilter.java File Reference	77
9.3 AdminSecurityFilter.java	77
9.4 Football_Manager/src/main/java/com/example/football_manager/config/CacheConfig.java File Reference	78
9.5 CacheConfig.java	78
9.6 Football_Manager/src/main/java/com/example/football_manager/config/LocaleConfig.java File Reference	79
9.7 LocaleConfig.java	79
9.8 Football_Manager/src/main/java/com/example/football_manager/config/OpenApiConfig.java File Reference	80
9.9 OpenApiConfig.java	80
9.10 Football_Manager/src/main/java/com/example/football_manager/config/SecurityConfig.java File Reference	81
9.11 SecurityConfig.java	81
9.12 Football_Manager/src/main/java/com/example/football_manager/controller/AdminCountryView↔Controller.java File Reference	81
9.13 AdminCountryViewController.java	82
9.14 Football_Manager/src/main/java/com/example/football_manager/controller/AdminPlayerView↔Controller.java File Reference	82
9.15 AdminPlayerViewController.java	83
9.16 Football_Manager/src/main/java/com/example/football_manager/controller/AdminUserView↔Controller.java File Reference	85
9.17 AdminUserViewController.java	85
9.18 Football_Manager/src/main/java/com/example/football_manager/controller/AdminViewController.java File Reference	85
9.19 AdminViewController.java	86
9.20 Football_Manager/src/main/java/com/example/football_manager/controller/AuthController.java File Reference	86
9.21 AuthController.java	87

9.22 Football_Manager/src/main/java/com/example/football_manager/controller/AuthViewController.java File Reference	88
9.23 AuthViewController.java	88
9.24 Football_Manager/src/main/java/com/example/football_manager/controller/CompetitionController.java File Reference	89
9.25 CompetitionController.java	89
9.26 Football_Manager/src/main/java/com/example/football_manager/controller/CountryController.java File Reference	90
9.27 CountryController.java	91
9.28 Football_Manager/src/main/java/com/example/football_manager/controller/FavouriteTeamController.java File Reference	92
9.29 FavouriteTeamController.java	92
9.30 Football_Manager/src/main/java/com/example/football_manager/controller/GlobalModelAttributes.java File Reference	93
9.31 GlobalModelAttributes.java	94
9.32 Football_Manager/src/main/java/com/example/football_manager/controller/MatchController.java File Reference	94
9.33 MatchController.java	95
9.34 Football_Manager/src/main/java/com/example/football_manager/controller/MatchViewController.java File Reference	97
9.35 MatchViewController.java	97
9.36 Football_Manager/src/main/java/com/example/football_manager/controller/PlayerController.java File Reference	99
9.37 PlayerController.java	100
9.38 Football_Manager/src/main/java/com/example/football_manager/controller/ProfileViewController.java File Reference	101
9.39 ProfileViewController.java	102
9.40 Football_Manager/src/main/java/com/example/football_manager/controller/StandingsController.java File Reference	103
9.41 StandingsController.java	104
9.42 Football_Manager/src/main/java/com/example/football_manager/controller/StandingsView↔ Controller.java File Reference	104
9.43 StandingsViewController.java	105
9.44 Football_Manager/src/main/java/com/example/football_manager/controller/TeamController.java File Reference	105
9.45 TeamController.java	106
9.46 Football_Manager/src/main/java/com/example/football_manager/controller/TeamViewController.java File Reference	107
9.47 TeamViewController.java	108
9.48 Football_Manager/src/main/java/com/example/football_manager/dto/AuthResponseDTO.java File Reference	109
9.49 AuthResponseDTO.java	110
9.50 Football_Manager/src/main/java/com/example/football_manager/dto/CompetitionRequestDTO.java File Reference	110
9.51 CompetitionRequestDTO.java	111

9.52 Football_Manager/src/main/java/com/example/football_manager/dto/CountryRequestDTO.java File Reference	111
9.53 CountryRequestDTO.java	111
9.54 Football_Manager/src/main/java/com/example/football_manager/dto/LoginDTO.java File Reference	112
9.55 LoginDTO.java	112
9.56 Football_Manager/src/main/java/com/example/football_manager/dto/MatchRequestDTO.java File Reference	112
9.57 MatchRequestDTO.java	113
9.58 Football_Manager/src/main/java/com/example/football_manager/dto/MatchResultDTO.java File Reference	114
9.59 MatchResultDTO.java	114
9.60 Football_Manager/src/main/java/com/example/football_manager/dto/MatchResultRequestDTO.java File Reference	114
9.61 MatchResultRequestDTO.java	115
9.62 Football_Manager/src/main/java/com/example/football_manager/dto/PlayerBulkRequestDTO.java File Reference	115
9.63 PlayerBulkRequestDTO.java	116
9.64 Football_Manager/src/main/java/com/example/football_manager/dto/PlayerRequestDTO.java File Reference	116
9.65 PlayerRequestDTO.java	116
9.66 Football_Manager/src/main/java/com/example/football_manager/dto/ProfileUpdateDTO.java File Reference	117
9.67 ProfileUpdateDTO.java	118
9.68 Football_Manager/src/main/java/com/example/football_manager/dto/RegisterDTO.java File Reference	118
9.69 RegisterDTO.java	119
9.70 Football_Manager/src/main/java/com/example/football_manager/dto/StandingDTO.java File Reference	119
9.71 StandingDTO.java	119
9.72 Football_Manager/src/main/java/com/example/football_manager/dto/TeamRequestDTO.java File Reference	121
9.73 TeamRequestDTO.java	121
9.74 Football_Manager/src/main/java/com/example/football_manager/FootballManagerApplication.java File Reference	122
9.75 FootballManagerApplication.java	122
9.76 Football_Manager/src/main/java/com/example/football_manager/model/Competition.java File Reference	122
9.77 Competition.java	123
9.78 Football_Manager/src/main/java/com/example/football_manager/model/Country.java File Reference	123
9.79 Country.java	123
9.80 Football_Manager/src/main/java/com/example/football_manager/model/Match.java File Reference	124
9.81 Match.java	124
9.82 Football_Manager/src/main/java/com/example/football_manager/model/MatchGoal.java File Reference	124
9.83 MatchGoal.java	125
9.84 Football_Manager/src/main/java/com/example/football_manager/model/Player.java File Reference	125
9.85 Player.java	126

9.86 Football_Manager/src/main/java/com/example/football_manager/model/PlayerPosition.java File Reference	126
9.87 PlayerPosition.java	126
9.88 Football_Manager/src/main/java/com/example/football_manager/model/Team.java File Reference	127
9.89 Team.java	127
9.90 Football_Manager/src/main/java/com/example/football_manager/model/User.java File Reference	128
9.91 User.java	128
9.92 Football_Manager/src/main/java/com/example/football_manager/repository/CompetitionRepository.java File Reference	128
9.93 CompetitionRepository.java	129
9.94 Football_Manager/src/main/java/com/example/football_manager/repository/CountryRepository.java File Reference	129
9.95 CountryRepository.java	129
9.96 Football_Manager/src/main/java/com/example/football_manager/repository/MatchGoalRepository.java File Reference	129
9.97 MatchGoalRepository.java	130
9.98 Football_Manager/src/main/java/com/example/football_manager/repository/MatchRepository.java File Reference	130
9.99 MatchRepository.java	130
9.100 Football_Manager/src/main/java/com/example/football_manager/repository/PlayerRepository.java File Reference	131
9.101 PlayerRepository.java	131
9.102 Football_Manager/src/main/java/com/example/football_manager/repository/TeamRepository.java File Reference	132
9.103 TeamRepository.java	132
9.104 Football_Manager/src/main/java/com/example/football_manager/repository/UserRepository.java File Reference	132
9.105 UserRepository.java	132
9.106 Football_Manager/src/main/java/com/example/football_manager/service/CompetitionService.java File Reference	133
9.107 CompetitionService.java	133
9.108 Football_Manager/src/main/java/com/example/football_manager/service/CountryService.java File Reference	134
9.109 CountryService.java	134
9.110 Football_Manager/src/main/java/com/example/football_manager/service/MatchService.java File Reference	135
9.111 MatchService.java	135
9.112 Football_Manager/src/main/java/com/example/football_manager/service/PlayerService.java File Reference	140
9.113 PlayerService.java	140
9.114 Football_Manager/src/main/java/com/example/football_manager/service/StandingsService.java File Reference	143
9.115 StandingsService.java	143
9.116 Football_Manager/src/main/java/com/example/football_manager/service/TeamService.java File Reference	145
9.117 TeamService.java	145

9.118 Football_Manager/src/main/java/com/example/football_manager/service/UserService.java File Reference	146
9.119 UserService.java	147
9.120 README.md File Reference	149
Index	151

Chapter 1

Football Manager

Football Manager is a Spring Boot web application developed for managing football teams, matches, results, users and favourite teams.

The project was developed as part of the Software Process and Quality course, including automated testing, performance testing, continuous integration and generated documentation.

1.1 Main Features

- User registration and login.
 - Admin and regular user roles.
 - Team management.
 - Country management.
 - Competition management.
 - Match management.
 - Match results.
 - Favourite teams.
 - Web views using Thymeleaf.
 - REST API endpoints.
 - Automated unit and integration tests.
 - Code coverage with JaCoCo.
 - Performance testing with JUnitPerf.
 - Continuous Integration with Jenkins.
 - Documentation generated with Doxygen.
 - Maven Site generation.
-

1.2 Technologies Used

- Java 17
 - Spring Boot
 - Spring Data JPA
 - Spring Security
 - Thymeleaf
 - PostgreSQL
 - Maven
 - JUnit 5
 - JUnit 4
 - Mockito
 - JUnitPerf
 - JaCoCo
 - Jenkins
 - Doxygen
 - Maven Site
 - Docker
-

1.3 Project Structure

```
BSPQ26-E6/  
Football_Manager/  
  src/main/java/com/example/football_manager/  
    config/  
    controller/  
    dto/  
    model/  
    repository/  
    service/  
  
  src/main/resources/  
    templates/  
    static/  
    application.properties  
  
  src/test/java/com/example/football_manager/  
    controller/  
    performance/  
    service/  
  
  pom.xml  
  mvnw  
  mvnw.cmd  
docs/  
  doxygen/  
  sprint3/  
Doxyfile  
Jenkinsfile  
README.md
```

1.4 Requirements

Before running the project, make sure you have:

- Java 17 installed.
- PostgreSQL installed and running.
- Git installed.
- Maven Wrapper included in the project.
- Docker installed if you want to run Jenkins or documentation tools through Docker.

Check Java version:

```
java -version
```

Expected version:

```
Java 17
```

1.5 Database Configuration

The application uses PostgreSQL.

Check the database configuration in:

```
Football_Manager/src/main/resources/application.properties
```

Typical configuration example:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/football_manager
spring.datasource.username=postgres
spring.datasource.password=your_password
spring.jpa.hibernate.ddl-auto=update
```

Make sure the database exists before running the application.

1.6 How to Run the Application

From the project folder:

1.6.1 Windows

```
cd Football_Manager
.\mvnw.cmd spring-boot:run
```

1.6.2 Linux / macOS

```
cd Football_Manager
chmod +x mvnw
./mvnw spring-boot:run
```

The application will be available at:

```
http://localhost:8080
```

1.7 How to Run Tests

1.7.1 Windows

```
cd Football_Manager
.\mvnw.cmd test
```

1.7.2 Linux / macOS

```
cd Football_Manager
./mvnw test
```

1.8 Code Coverage with JaCoCo

To run tests and generate the JaCoCo coverage report:

1.8.1 Windows

```
cd Football_Manager
.\mvnw.cmd clean test jacoco:report
```

1.8.2 Linux / macOS

```
cd Football_Manager
./mvnw clean test jacoco:report
```

The report is generated in:

Football_Manager/target/site/jacoco/index.html

1.9 Performance Testing with JUnitPerf

The project uses JUnitPerf for performance testing.

Performance tests are located in:

Football_Manager/src/test/java/com/example/football_manager/performance/

To run only the performance tests:

1.9.1 Windows

```
cd Football_Manager
.\mvnw.cmd test -Dtest=TeamServicePerformanceTest
```

1.9.2 Linux / macOS

```
cd Football_Manager
./mvnw test -Dtest=TeamServicePerformanceTest
```

JUnitPerf generates an HTML report in:

Football_Manager/build/reports/junitperf_report.html

The performance tests include:

- Single-thread execution.
 - Multi-thread execution.
 - Throughput validation.
 - Duration-based execution.
 - Mean latency validation.
 - Maximum latency validation.
 - An intentionally failing test for Sprint 2 evidence.
-

1.10 Jenkins CI Pipeline

The repository includes a `Jenkinsfile` for Sprint 3 Continuous Integration.

The Jenkins pipeline performs:

1. Checkout from GitHub.
2. Java version verification.
3. Maven compile.
4. Test execution.
5. JaCoCo report generation.
6. Package generation.
7. Artifact archiving.

The Jenkinsfile is located at:

Jenkinsfile

To test the Jenkins pipeline:

1. Open Jenkins.
2. Create a new Pipeline job.
3. Select Pipeline script from SCM.
4. Select Git.
5. Use the repository URL:

`https://github.com/BSPQ25-26/BSPQ26-E6.git`

1. Use branch:

`*/main`

1. Use script path:

Jenkinsfile

1. Run Build Now.

A successful execution should finish with:

`Finished: SUCCESS`

1.11 Doxygen Documentation

The project includes a Doxygen configuration file:

Doxyfile

Doxygen generates documentation from the Java source code.

To generate Doxygen documentation using Docker:

```
docker run --rm -v "${PWD}:/project" -w /project alpine:latest sh -c "apk add --no-cache doxygen && doxygen
Doxyfile"
```

Generated HTML documentation:

`docs/doxygen/html/index.html`

Generated LaTeX documentation:

`docs/doxygen/latex/`

Generated PDF documentation, if compiled:

`docs/doxygen/latex/refman.pdf`

1.12 Swagger/OpenAPI Documentation

The project includes Swagger/OpenAPI documentation for the REST API.

The Swagger documentation is available at:

`/swagger-ui/index.html`

The OpenAPI specification is available at:

`/v3/api-docs`

1.13 Maven Site

To generate the Maven Site:

1.13.1 Windows

```
cd Football_Manager
.\mvnw.cmd site
```

1.13.2 Linux / macOS

```
cd Football_Manager
./mvnw site
```

The generated site is located at:

Football_Manager/target/site/index.html

The Maven Site includes project information and generated reports.

1.14 Main API Endpoints

Some of the main REST endpoints are:

```
GET    /api/teams
POST   /api/teams
PUT    /api/teams/{id}
DELETE /api/teams/{id}

GET    /api/countries
POST   /api/countries
DELETE /api/countries/{id}

GET    /api/competitions
POST   /api/competitions
PUT    /api/competitions/{id}
DELETE /api/competitions/{id}

GET    /api/matches
POST   /api/matches
PUT    /api/matches/{id}
DELETE /api/matches/{id}

GET    /api/matches/results
PATCH /api/matches/{id}/result

POST   /api/auth/register
POST   /api/auth/login
POST   /api/auth/logout
```

1.15 Web Views

Main web pages include:

```
/login
/register
/teams
/teams/{id}
/matches/schedule
/matches/results
/profile
/admin
/admin/manage-teams
```

1.16 Testing Strategy

The project includes different types of tests:

- Unit tests for service classes.
- Controller tests.
- Authentication tests.
- Performance tests.
- Coverage reports with JaCoCo.

Main tested areas:

- User authentication.
- Country management.
- Competition management.
- Match management.

- Team management.
 - Performance of team retrieval operations.
-

1.17 Sprint 2 Evidence

Sprint 2 included:

- Unit tests.
- Controller tests.
- JaCoCo coverage report.
- Performance tests using JUnitPerf.
- Successful performance execution.
- Intentionally failing performance execution.
- Performance HTML report.
- VisualVM evidence, if available.
- Release 2 evidence.

Relevant generated files:

```
Football_Manager/target/site/jacoco/  
Football_Manager/build/reports/junitperf_report.html  
docs/sprint2/
```

1.18 Sprint 3 Evidence

Sprint 3 includes:

- Jenkins CI pipeline.
- Successful Jenkins build evidence.
- Doxygen HTML documentation.
- Doxygen LaTeX/PDF documentation.
- Maven Site.
- Improved README documentation.
- Release 3 evidence.

Relevant files:

```
Jenkinsfile  
Doxyfile  
docs/doxygen/  
docs/sprint3/  
Football_Manager/target/site/
```

1.19 How to Package the Application

To generate the `.jar` file:

1.19.1 Windows

```
cd Football_Manager  
.\mvnw.cmd package -DskipTests
```

1.19.2 Linux / macOS

```
cd Football_Manager
./mvnw package -DskipTests
```

The generated artifact is located in:

`Football_Manager/target/`

1.20 Authors

Group E6

Team members:

- Alba Delgado Ortega
 - Alejandro Vicente Milla Tinajero
 - Alex Ramarathinam Aguirre
 - Elena Torralbo Jiménez
 - Iker Castillo Peiteado
 - Jaime Etxebarria Ugarte
 - Javier Gómez Martínez
-

1.21 Repository

<https://github.com/BSPQ25-26/BSPQ26-E6>

1.22 Project Status

The project includes the main functionality required for the course project, together with testing, performance evaluation, continuous integration and generated documentation evidence for Sprint 2 and Sprint 3.

Chapter 2

doxygen_mainpage

Football Manager Documentation

Overview

Football Manager is a Spring Boot web application developed for managing football teams, matches, results and standings.

The project includes:

- Team management
- Match management
- Match results
- Favourite teams
- User authentication
- Admin views
- REST API controllers
- Automated tests
- CI pipeline with Jenkins
- Code coverage with JaCoCo
- Performance testing with JUnitPerf

Main Packages

- `controller`: REST and web controllers.
- `service`: Business logic.
- `repository`: Data access layer.
- `model`: Entity classes.
- `dto`: Data transfer objects.
- `config`: Security and application configuration.

Sprint 3 Documentation

This documentation was generated automatically using Doxygen as part of Sprint 3.

Chapter 3

Namespace Index

3.1 Package List

Here are the packages with brief descriptions (if available):

com.example.football_manager	19
com.example.football_manager.config	19
com.example.football_manager.controller	19
com.example.football_manager.dto	20
com.example.football_manager.model	20
com.example.football_manager.repository	21
com.example.football_manager.service	21

Chapter 4

Hierarchical Index

4.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

com.example.football_manager.controller.AdminCountryViewController	23
com.example.football_manager.controller.AdminPlayerViewController	24
com.example.football_manager.controller.AdminUserViewController	26
com.example.football_manager.controller.AdminViewController	27
com.example.football_manager.controller.AuthController	28
com.example.football_manager.dto.AuthResponseDTO	28
com.example.football_manager.controller.AuthViewController	29
com.example.football_manager.config.CacheConfig	30
com.example.football_manager.model.Competition	30
com.example.football_manager.controller.CompetitionController	30
com.example.football_manager.dto.CompetitionRequestDTO	32
com.example.football_manager.service.CompetitionService	32
com.example.football_manager.model.Country	33
com.example.football_manager.controller.CountryController	34
com.example.football_manager.dto.CountryRequestDTO	35
com.example.football_manager.service.CountryService	35
com.example.football_manager.controller.FavouriteTeamController	36
Filter	
com.example.football_manager.config.AdminSecurityFilter	26
com.example.football_manager.FootballManagerApplication	37
com.example.football_manager.controller.GlobalModelAttributes	37
com.example.football_manager.dto.MatchResultRequestDTO.GoalDTO	38
JpaRepository	
com.example.football_manager.repository.CompetitionRepository	31
com.example.football_manager.repository.CountryRepository	34
com.example.football_manager.repository.MatchGoalRepository	42
com.example.football_manager.repository.MatchRepository	42
com.example.football_manager.repository.PlayerRepository	55
com.example.football_manager.repository.TeamRepository	68
com.example.football_manager.repository.UserRepository	73
com.example.football_manager.dto.LoginDTO	39
com.example.football_manager.model.Match	40
com.example.football_manager.controller.MatchController	40
com.example.football_manager.model.MatchGoal	42
com.example.football_manager.dto.MatchRequestDTO	44
com.example.football_manager.dto.MatchResultRequestDTO	46
com.example.football_manager.service.MatchService	47
com.example.football_manager.dto.MatchRequestDTO.MatchStatus	49
com.example.football_manager.controller.MatchViewController	50
com.example.football_manager.config.OpenApiConfig	52

com.example.football_manager.model.Player	52
com.example.football_manager.dto.PlayerBulkRequestDTO	52
com.example.football_manager.controller.PlayerController	53
com.example.football_manager.model.PlayerPosition	54
com.example.football_manager.dto.PlayerRequestDTO	56
com.example.football_manager.service.PlayerService	57
com.example.football_manager.dto.ProfileUpdateDTO	59
com.example.football_manager.controller.ProfileViewController	59
com.example.football_manager.dto.RegisterDTO	60
com.example.football_manager.config.SecurityConfig	61
com.example.football_manager.dto.StandingDTO	61
com.example.football_manager.controller.StandingsController	64
com.example.football_manager.service.StandingsService	65
com.example.football_manager.controller.StandingsViewController	66
com.example.football_manager.model.Team	66
com.example.football_manager.controller.TeamController	67
com.example.football_manager.dto.TeamRequestDTO	69
com.example.football_manager.service.TeamService	70
com.example.football_manager.controller.TeamViewController	71
com.example.football_manager.model.User	73
com.example.football_manager.service.UserService	74
WebMvcConfigurer	
com.example.football_manager.config.LocaleConfig	39

Chapter 5

Class Index

5.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

com.example.football_manager.controller.AdminCountryViewController	23
com.example.football_manager.controller.AdminPlayerViewController	24
com.example.football_manager.config.AdminSecurityFilter	26
com.example.football_manager.controller.AdminUserController	26
com.example.football_manager.controller.AdminViewController	27
com.example.football_manager.controller.AuthController	28
com.example.football_manager.dto.AuthResponseDTO	28
com.example.football_manager.controller.AuthViewController	29
com.example.football_manager.config.CacheConfig	30
com.example.football_manager.model.Competition	30
com.example.football_manager.controller.CompetitionController	30
com.example.football_manager.repository.CompetitionRepository	31
com.example.football_manager.dto.CompetitionRequestDTO	32
com.example.football_manager.service.CompetitionService	32
com.example.football_manager.model.Country	33
com.example.football_manager.controller.CountryController	34
com.example.football_manager.repository.CountryRepository	34
com.example.football_manager.dto.CountryRequestDTO	35
com.example.football_manager.service.CountryService	35
com.example.football_manager.controller.FavouriteTeamController	36
com.example.football_manager.FootballManagerApplication	37
com.example.football_manager.controller.GlobalModelAttributes	37
com.example.football_manager.dto.MatchResultRequestDTO.GoalDTO	38
com.example.football_manager.config.LocaleConfig	39
com.example.football_manager.dto.LoginDTO	39
com.example.football_manager.model.Match	40
com.example.football_manager.controller.MatchController	40
com.example.football_manager.model.MatchGoal	42
com.example.football_manager.repository.MatchGoalRepository	42
com.example.football_manager.repository.MatchRepository	42
com.example.football_manager.dto.MatchRequestDTO	44
com.example.football_manager.dto.MatchResultRequestDTO	46
com.example.football_manager.service.MatchService	47
com.example.football_manager.dto.MatchRequestDTO.MatchStatus	49
com.example.football_manager.controller.MatchViewController	50
com.example.football_manager.config.OpenApiConfig	52
com.example.football_manager.model.Player	52
com.example.football_manager.dto.PlayerBulkRequestDTO	52
com.example.football_manager.controller.PlayerController	53
com.example.football_manager.model.PlayerPosition	54

com.example.football_manager.repository.PlayerRepository	55
com.example.football_manager.dto.PlayerRequestDTO	56
com.example.football_manager.service.PlayerService	57
com.example.football_manager.dto.ProfileUpdateDTO	59
com.example.football_manager.controller.ProfileViewController	59
com.example.football_manager.dto.RegisterDTO	60
com.example.football_manager.config.SecurityConfig	61
com.example.football_manager.dto.StandingDTO	
View-only DTO used to render a single row of the standings table	61
com.example.football_manager.controller.StandingsController	64
com.example.football_manager.service.StandingsService	65
com.example.football_manager.controller.StandingsViewController	66
com.example.football_manager.model.Team	66
com.example.football_manager.controller.TeamController	67
com.example.football_manager.repository.TeamRepository	68
com.example.football_manager.dto.TeamRequestDTO	69
com.example.football_manager.service.TeamService	70
com.example.football_manager.controller.TeamViewController	71
com.example.football_manager.model.User	73
com.example.football_manager.repository.UserRepository	73
com.example.football_manager.service.UserService	74

Chapter 6

File Index

6.1 File List

Here is a list of all files with brief descriptions:

Football_Manager/src/main/java/com/example/football_manager/FootballManagerApplication.java	122
Football_Manager/src/main/java/com/example/football_manager/config/AdminSecurityFilter.java	77
Football_Manager/src/main/java/com/example/football_manager/config/CacheConfig.java	78
Football_Manager/src/main/java/com/example/football_manager/config/LocaleConfig.java	79
Football_Manager/src/main/java/com/example/football_manager/config/OpenApiConfig.java	80
Football_Manager/src/main/java/com/example/football_manager/config/SecurityConfig.java	81
Football_Manager/src/main/java/com/example/football_manager/controller/AdminCountryViewController.java	81
Football_Manager/src/main/java/com/example/football_manager/controller/AdminPlayerViewController.java	82
Football_Manager/src/main/java/com/example/football_manager/controller/AdminUserViewController.java	85
Football_Manager/src/main/java/com/example/football_manager/controller/AdminViewController.java	85
Football_Manager/src/main/java/com/example/football_manager/controller/AuthController.java	86
Football_Manager/src/main/java/com/example/football_manager/controller/AuthViewController.java	88
Football_Manager/src/main/java/com/example/football_manager/controller/CompetitionController.java	89
Football_Manager/src/main/java/com/example/football_manager/controller/CountryController.java	90
Football_Manager/src/main/java/com/example/football_manager/controller/FavouriteTeamController.java	92
Football_Manager/src/main/java/com/example/football_manager/controller/GlobalModelAttributes.java	93
Football_Manager/src/main/java/com/example/football_manager/controller/MatchController.java	94
Football_Manager/src/main/java/com/example/football_manager/controller/MatchViewController.java	97
Football_Manager/src/main/java/com/example/football_manager/controller/PlayerController.java	99
Football_Manager/src/main/java/com/example/football_manager/controller/ProfileViewController.java	101
Football_Manager/src/main/java/com/example/football_manager/controller/StandingsController.java	103
Football_Manager/src/main/java/com/example/football_manager/controller/StandingsViewController.java	104
Football_Manager/src/main/java/com/example/football_manager/controller/TeamController.java	105
Football_Manager/src/main/java/com/example/football_manager/controller/TeamViewController.java	107
Football_Manager/src/main/java/com/example/football_manager/dto/AuthResponseDTO.java	109
Football_Manager/src/main/java/com/example/football_manager/dto/CompetitionRequestDTO.java	110
Football_Manager/src/main/java/com/example/football_manager/dto/CountryRequestDTO.java	111
Football_Manager/src/main/java/com/example/football_manager/dto/LoginDTO.java	112
Football_Manager/src/main/java/com/example/football_manager/dto/MatchRequestDTO.java	112
Football_Manager/src/main/java/com/example/football_manager/dto/MatchResultDTO.java	114
Football_Manager/src/main/java/com/example/football_manager/dto/MatchResultRequestDTO.java	114
Football_Manager/src/main/java/com/example/football_manager/dto/PlayerBulkRequestDTO.java	115
Football_Manager/src/main/java/com/example/football_manager/dto/PlayerRequestDTO.java	116
Football_Manager/src/main/java/com/example/football_manager/dto/ProfileUpdateDTO.java	117
Football_Manager/src/main/java/com/example/football_manager/dto/RegisterDTO.java	118
Football_Manager/src/main/java/com/example/football_manager/dto/StandingDTO.java	119
Football_Manager/src/main/java/com/example/football_manager/dto/TeamRequestDTO.java	121
Football_Manager/src/main/java/com/example/football_manager/model/Competition.java	122

Football_Manager/src/main/java/com/example/football_manager/model/Country.java	123
Football_Manager/src/main/java/com/example/football_manager/model/Match.java	124
Football_Manager/src/main/java/com/example/football_manager/model/MatchGoal.java	124
Football_Manager/src/main/java/com/example/football_manager/model/Player.java	125
Football_Manager/src/main/java/com/example/football_manager/model/PlayerPosition.java	126
Football_Manager/src/main/java/com/example/football_manager/model/Team.java	127
Football_Manager/src/main/java/com/example/football_manager/model/User.java	128
Football_Manager/src/main/java/com/example/football_manager/repository/CompetitionRepository.java .	128
Football_Manager/src/main/java/com/example/football_manager/repository/CountryRepository.java . . .	129
Football_Manager/src/main/java/com/example/football_manager/repository/MatchGoalRepository.java .	129
Football_Manager/src/main/java/com/example/football_manager/repository/MatchRepository.java	130
Football_Manager/src/main/java/com/example/football_manager/repository/PlayerRepository.java	131
Football_Manager/src/main/java/com/example/football_manager/repository/TeamRepository.java	132
Football_Manager/src/main/java/com/example/football_manager/repository/UserRepository.java	132
Football_Manager/src/main/java/com/example/football_manager/service/CompetitionService.java	133
Football_Manager/src/main/java/com/example/football_manager/service/CountryService.java	134
Football_Manager/src/main/java/com/example/football_manager/service/MatchService.java	135
Football_Manager/src/main/java/com/example/football_manager/service/PlayerService.java	140
Football_Manager/src/main/java/com/example/football_manager/service/StandingsService.java	143
Football_Manager/src/main/java/com/example/football_manager/service/TeamService.java	145
Football_Manager/src/main/java/com/example/football_manager/service/UserService.java	146

Chapter 7

Namespace Documentation

7.1 Package com.example.football_manager

Packages

- package [config](#)
- package [controller](#)
- package [dto](#)
- package [model](#)
- package [repository](#)
- package [service](#)

Classes

- class [FootballManagerApplication](#)

7.2 Package com.example.football_manager.config

Classes

- class [AdminSecurityFilter](#)
- class [CacheConfig](#)
- class [LocaleConfig](#)
- class [OpenApiConfig](#)
- class [SecurityConfig](#)

7.3 Package com.example.football_manager.controller

Classes

- class [AdminCountryViewController](#)
- class [AdminPlayerViewController](#)
- class [AdminUserViewController](#)
- class [AdminViewController](#)
- class [AuthController](#)
- class [AuthViewController](#)
- class [CompetitionController](#)
- class [CountryController](#)
- class [FavouriteTeamController](#)
- class [GlobalModelAttributes](#)
- class [MatchController](#)
- class [MatchViewController](#)

- class [PlayerController](#)
- class [ProfileViewController](#)
- class [StandingsController](#)
- class [StandingsViewController](#)
- class [TeamController](#)
- class [TeamViewController](#)

7.4 Package com.example.football_manager.dto

Classes

- class [AuthResponseDTO](#)
- class [CompetitionRequestDTO](#)
- class [CountryRequestDTO](#)
- class [LoginDTO](#)
- class [MatchRequestDTO](#)
- class [MatchResultRequestDTO](#)
- class [PlayerBulkRequestDTO](#)
- class [PlayerRequestDTO](#)
- class [ProfileUpdateDTO](#)
- class [RegisterDTO](#)
- class [StandingDTO](#)

View-only DTO used to render a single row of the standings table.

- class [TeamRequestDTO](#)

Functions

- record [MatchResultDTO](#) (@Schema(description="Match identifier", example="220") Long matchId, @Schema(description="Home team name", example="Real Sociedad") String homeTeamName, @Schema(description="Away team name", example="Athletic Club") String awayTeamName, @Schema(description="Home team score", example="2") short homeScore, @Schema(description="Away team score", example="1") short awayScore, @Schema(description="Kickoff time (UTC)", example="2026-04-28T18:30:00Z") OffsetDateTime kickoffTime)

7.4.1 Function Documentation

7.4.1.1 MatchResultDTO()

```
record com.example.football_manager.dto.MatchResultDTO (
    @Schema(description="Match identifier", example="220") Long matchId,
    @Schema(description="Home team name", example="Real Sociedad") String homeTeamName,
    @Schema(description="Away team name", example="Athletic Club") String awayTeamName,
    @Schema(description="Home team score", example="2") short homeScore,
    @Schema(description="Away team score", example="1") short awayScore,
    @Schema(description="Kickoff time (UTC)", example="2026-04-28T18:30:00Z") OffsetDateTime kickoffTime)
```

Definition at line 6 of file [MatchResultDTO.java](#).

7.5 Package com.example.football_manager.model

Classes

- class [Competition](#)
- class [Country](#)
- class [Match](#)

- class [MatchGoal](#)
- class [Player](#)
- enum [PlayerPosition](#)
- class [Team](#)
- class [User](#)

7.6 Package com.example.football_manager.repository

Classes

- interface [CompetitionRepository](#)
- interface [CountryRepository](#)
- interface [MatchGoalRepository](#)
- interface [MatchRepository](#)
- interface [PlayerRepository](#)
- interface [TeamRepository](#)
- interface [UserRepository](#)

7.7 Package com.example.football_manager.service

Classes

- class [CompetitionService](#)
- class [CountryService](#)
- class [MatchService](#)
- class [PlayerService](#)
- class [StandingsService](#)
- class [TeamService](#)
- class [UserService](#)

Chapter 8

Class Documentation

8.1 `com.example.football_manager.controller.AdminCountryView`↔ Controller Class Reference

Public Member Functions

- `AdminCountryViewController` (`CountryService` `countryService`)
- `String` `countriesPage` (`Model` `model`)
- `String` `createCountry` (`@ModelAttribute` `CountryRequestDTO` `countryRequest`)
- `String` `deleteCountry` (`@PathVariable` `Long` `id`)

8.1.1 Detailed Description

Definition at line 10 of file `AdminCountryViewController.java`.

8.1.2 Constructor & Destructor Documentation

8.1.2.1 `AdminCountryViewController()`

```
com.example.football_manager.controller.AdminCountryViewController.AdminCountryViewController (
    CountryService countryService)
```

Definition at line 14 of file `AdminCountryViewController.java`.

8.1.3 Member Function Documentation

8.1.3.1 `countriesPage()`

```
String com.example.football_manager.controller.AdminCountryViewController.countriesPage (
    Model model)
```

Definition at line 19 of file `AdminCountryViewController.java`.

References `countriesPage()`.

Referenced by `countriesPage()`.

8.1.3.2 `createCountry()`

```
String com.example.football_manager.controller.AdminCountryViewController.createCountry (
    @ModelAttribute CountryRequestDTO countryRequest)
```

Definition at line 26 of file `AdminCountryViewController.java`.

References `createCountry()`.

Referenced by `createCountry()`.

8.1.3.3 deleteCountry()

```
String com.example.football_manager.controller.AdminCountryViewController.deleteCountry (
    @PathVariable Long id)
```

Definition at line 32 of file [AdminCountryViewController.java](#).

References [deleteCountry\(\)](#).

Referenced by [deleteCountry\(\)](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/AdminCountryViewController.java](#)

8.2 com.example.football_manager.controller.AdminPlayerView↵ Controller Class Reference

Public Member Functions

- [AdminPlayerViewController](#) ([TeamService](#) teamService, [PlayerService](#) playerService)
- String [managePlayersLanding](#) (Model model)
- String [manageTeamPlayers](#) (@PathVariable Long teamId, Model model, RedirectAttributes redirectAttributes)
- String [createPlayer](#) (@PathVariable Long teamId, @Valid @ModelAttribute("playerRequest") [PlayerRequestDTO](#) playerRequest, BindingResult bindingResult, Model model, RedirectAttributes redirectAttributes)
- String [showEditPlayerForm](#) (@PathVariable Long teamId, @PathVariable Long playerId, Model model, RedirectAttributes redirectAttributes)
- String [updatePlayer](#) (@PathVariable Long teamId, @PathVariable Long playerId, @Valid @ModelAttribute("playerRequest") [PlayerRequestDTO](#) playerRequest, BindingResult bindingResult, Model model, RedirectAttributes redirectAttributes)
- String [deletePlayer](#) (@PathVariable Long teamId, @PathVariable Long playerId, RedirectAttributes redirectAttributes)

8.2.1 Detailed Description

Definition at line 20 of file [AdminPlayerViewController.java](#).

8.2.2 Constructor & Destructor Documentation

8.2.2.1 AdminPlayerViewController()

```
com.example.football_manager.controller.AdminPlayerViewController.AdminPlayerViewController (
    TeamService teamService,
    PlayerService playerService)
```

Definition at line 25 of file [AdminPlayerViewController.java](#).

8.2.3 Member Function Documentation

8.2.3.1 createPlayer()

```
String com.example.football_manager.controller.AdminPlayerViewController.createPlayer (
    @PathVariable Long teamId,
    @Valid @ModelAttribute("playerRequest") PlayerRequestDTO playerRequest,
    BindingResult bindingResult,
    Model model,
    RedirectAttributes redirectAttributes)
```

Definition at line 52 of file [AdminPlayerViewController.java](#).

References [createPlayer\(\)](#).

Referenced by [createPlayer\(\)](#).

8.2.3.2 deletePlayer()

```
String com.example.football_manager.controller.AdminPlayerViewController.deletePlayer (
    @PathVariable Long teamId,
    @PathVariable Long playerId,
    RedirectAttributes redirectAttributes)
```

Definition at line 151 of file [AdminPlayerViewController.java](#).

References [deletePlayer\(\)](#).

Referenced by [deletePlayer\(\)](#).

8.2.3.3 managePlayersLanding()

```
String com.example.football_manager.controller.AdminPlayerViewController.managePlayersLanding
(
    Model model)
```

Definition at line 31 of file [AdminPlayerViewController.java](#).

References [managePlayersLanding\(\)](#).

Referenced by [managePlayersLanding\(\)](#).

8.2.3.4 manageTeamPlayers()

```
String com.example.football_manager.controller.AdminPlayerViewController.manageTeamPlayers (
    @PathVariable Long teamId,
    Model model,
    RedirectAttributes redirectAttributes)
```

Definition at line 36 of file [AdminPlayerViewController.java](#).

References [manageTeamPlayers\(\)](#).

Referenced by [manageTeamPlayers\(\)](#).

8.2.3.5 showEditPlayerForm()

```
String com.example.football_manager.controller.AdminPlayerViewController.showEditPlayerForm (
    @PathVariable Long teamId,
    @PathVariable Long playerId,
    Model model,
    RedirectAttributes redirectAttributes)
```

Definition at line 82 of file [AdminPlayerViewController.java](#).

References [showEditPlayerForm\(\)](#).

Referenced by [showEditPlayerForm\(\)](#).

8.2.3.6 updatePlayer()

```
String com.example.football_manager.controller.AdminPlayerViewController.updatePlayer (
    @PathVariable Long teamId,
    @PathVariable Long playerId,
    @Valid @ModelAttribute("playerRequest") PlayerRequestDTO playerRequest,
    BindingResult bindingResult,
    Model model,
    RedirectAttributes redirectAttributes)
```

Definition at line 116 of file [AdminPlayerViewController.java](#).

References [updatePlayer\(\)](#).

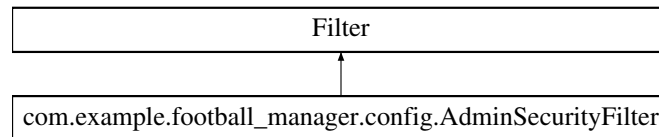
Referenced by [updatePlayer\(\)](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/AdminPlayerViewController.java](#)

8.3 com.example.football_manager.config.AdminSecurityFilter Class Reference

Inheritance diagram for com.example.football_manager.config.AdminSecurityFilter:



Public Member Functions

- void [doFilter](#) (ServletRequest request, ServletResponse response, FilterChain chain) throws IOException, ServletException

8.3.1 Detailed Description

Definition at line 16 of file [AdminSecurityFilter.java](#).

8.3.2 Member Function Documentation

8.3.2.1 doFilter()

```
void com.example.football_manager.config.AdminSecurityFilter.doFilter (
    ServletRequest request,
    ServletResponse response,
    FilterChain chain) throws IOException, ServletException
```

Definition at line 19 of file [AdminSecurityFilter.java](#).

The documentation for this class was generated from the following file:

- Football_Manager/src/main/java/com/example/football_manager/config/[AdminSecurityFilter.java](#)

8.4 com.example.football_manager.controller.AdminUserController Class Reference

Public Member Functions

- [AdminUserController](#) ([UserService](#) userService)
- String [usersPage](#) ()
- String [createUser](#) (@RequestParam String username, @RequestParam String email, @RequestParam String password, @RequestParam(defaultValue="false") Boolean isAdmin)

8.4.1 Detailed Description

Definition at line 8 of file [AdminUserController.java](#).

8.4.2 Constructor & Destructor Documentation

8.4.2.1 AdminUserController()

```
com.example.football_manager.controller.AdminUserController.AdminUserController (
    UserService userService)
```

Definition at line 12 of file [AdminUserController.java](#).

8.4.3 Member Function Documentation

8.4.3.1 createUser()

```
String com.example.football_manager.controller.AdminUserController.createUser (
    @RequestParam String username,
    @RequestParam String email,
    @RequestParam String password,
    @RequestParam(defaultValue="false") Boolean isAdmin)
```

Definition at line 22 of file [AdminUserController.java](#).

References [createUser\(\)](#).

Referenced by [createUser\(\)](#).

8.4.3.2 usersPage()

```
String com.example.football_manager.controller.AdminUserController.usersPage ()
```

Definition at line 17 of file [AdminUserController.java](#).

References [usersPage\(\)](#).

Referenced by [usersPage\(\)](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/AdminUserController.java](#)

8.5 com.example.football_manager.controller.AdminViewController Class Reference

Public Member Functions

- String [adminPanel](#) ()
- String [manageTeams](#) (Model model)

8.5.1 Detailed Description

Definition at line 10 of file [AdminViewController.java](#).

8.5.2 Member Function Documentation

8.5.2.1 adminPanel()

```
String com.example.football_manager.controller.AdminViewController.adminPanel ()
```

Definition at line 16 of file [AdminViewController.java](#).

References [adminPanel\(\)](#).

Referenced by [adminPanel\(\)](#).

8.5.2.2 manageTeams()

```
String com.example.football_manager.controller.AdminViewController.manageTeams (
    Model model)
```

Definition at line 21 of file [AdminViewController.java](#).

References [manageTeams\(\)](#).

Referenced by [manageTeams\(\)](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/AdminViewController.java](#)

8.6 com.example.football_manager.controller.AuthController Class Reference

Public Member Functions

- `ResponseEntity< AuthResponseDTO > register` (`@Valid @RequestBody RegisterDTO registerDTO`)
- `ResponseEntity< AuthResponseDTO > login` (`@Valid @RequestBody LoginDTO loginDTO, HttpSession session`)

8.6.1 Detailed Description

Definition at line 24 of file [AuthController.java](#).

8.6.2 Member Function Documentation

8.6.2.1 login()

```
ResponseEntity< AuthResponseDTO > com.example.football_manager.controller.AuthController.login  
(  
    @Valid @RequestBody LoginDTO loginDTO,  
    HttpSession session)
```

Definition at line 76 of file [AuthController.java](#).

8.6.2.2 register()

```
ResponseEntity< AuthResponseDTO > com.example.football_manager.controller.AuthController.  
register (  
    @Valid @RequestBody RegisterDTO registerDTO)
```

Definition at line 44 of file [AuthController.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/AuthController.java](#)

8.7 com.example.football_manager.dto.AuthResponseDTO Class Reference

Public Member Functions

- [AuthResponseDTO](#) (Long id, String username, String email, Boolean isAdmin, String message)
- Long [getId](#) ()
- String [getUsername](#) ()
- String [getEmail](#) ()
- Boolean [getIsAdmin](#) ()
- String [getMessage](#) ()

8.7.1 Detailed Description

Definition at line 5 of file [AuthResponseDTO.java](#).

8.7.2 Constructor & Destructor Documentation

8.7.2.1 AuthResponseDTO()

```
com.example.football_manager.dto.AuthResponseDTO.AuthResponseDTO (  
    Long id,  
    String username,  
    String email,  
    Boolean isAdmin,  
    String message)
```


Definition at line 18 of file [AuthResponseDTO.java](#).

8.7.3 Member Function Documentation

8.7.3.1 getEmail()

`String com.example.football_manager.dto.AuthResponseDTO.getEmail ()`

Definition at line 34 of file [AuthResponseDTO.java](#).

8.7.3.2 getId()

`Long com.example.football_manager.dto.AuthResponseDTO.getId ()`

Definition at line 26 of file [AuthResponseDTO.java](#).

8.7.3.3 getIsAdmin()

`Boolean com.example.football_manager.dto.AuthResponseDTO.getIsAdmin ()`

Definition at line 38 of file [AuthResponseDTO.java](#).

8.7.3.4 getMessage()

`String com.example.football_manager.dto.AuthResponseDTO.getMessage ()`

Definition at line 42 of file [AuthResponseDTO.java](#).

8.7.3.5 getUsername()

`String com.example.football_manager.dto.AuthResponseDTO.getUsername ()`

Definition at line 30 of file [AuthResponseDTO.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/dto/AuthResponseDTO.java](#)

8.8 com.example.football_manager.controller.AuthViewController Class Reference

Public Member Functions

- `String index ()`
- `String loginPage ()`
- `String registerPage ()`

8.8.1 Detailed Description

Definition at line 7 of file [AuthViewController.java](#).

8.8.2 Member Function Documentation

8.8.2.1 index()

`String com.example.football_manager.controller.AuthViewController.index ()`

Definition at line 11 of file [AuthViewController.java](#).

References [index\(\)](#).

Referenced by [index\(\)](#).

8.8.2.2 loginPage()

`String com.example.football_manager.controller.AuthViewController.loginPage ()`

Definition at line 16 of file [AuthViewController.java](#).

References [loginPage\(\)](#).

Referenced by [loginPage\(\)](#).

8.8.2.3 registerPage()

`String com.example.football_manager.controller.AuthViewController.registerPage ()`

Definition at line 21 of file [AuthViewController.java](#).

References [registerPage\(\)](#).

Referenced by [registerPage\(\)](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/AuthViewController.java](#)

8.9 com.example.football_manager.config.CacheConfig Class Reference

Public Member Functions

- `CacheManager` [cacheManager](#) ()

8.9.1 Detailed Description

Definition at line 16 of file [CacheConfig.java](#).

8.9.2 Member Function Documentation

8.9.2.1 cacheManager()

`CacheManager com.example.football_manager.config.CacheConfig.cacheManager ()`

Definition at line 19 of file [CacheConfig.java](#).

References [cacheManager\(\)](#).

Referenced by [cacheManager\(\)](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/config/CacheConfig.java](#)

8.10 com.example.football_manager.model.Competition Class Reference

8.10.1 Detailed Description

Definition at line 13 of file [Competition.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/model/Competition.java](#)

8.11 com.example.football_manager.controller.CompetitionController Class Reference

Public Member Functions

- [CompetitionController](#) ([CompetitionService](#) competitionService)
- `ResponseEntity< Competition >` [createCompetition](#) (@Valid @RequestBody [CompetitionRequestDTO](#) dto)
- `ResponseEntity< Competition >` [updateCompetition](#) (@PathVariable Long id, @Valid @RequestBody [CompetitionRequestDTO](#) dto)
- `ResponseEntity< Competition >` [getCompetitionById](#) (@PathVariable Long id)
- `ResponseEntity< List< Competition > >` [getAllCompetitions](#) ()
- `ResponseEntity< Void >` [deleteCompetition](#) (@PathVariable Long id)

8.11.1 Detailed Description

Definition at line 22 of file [CompetitionController.java](#).

8.11.2 Constructor & Destructor Documentation

8.11.2.1 CompetitionController()

```
com.example.football_manager.controller.CompetitionController.CompetitionController (
    CompetitionService competitionService)
```

Definition at line 26 of file [CompetitionController.java](#).

8.11.3 Member Function Documentation

8.11.3.1 createCompetition()

```
ResponseEntity< Competition > com.example.football_manager.controller.CompetitionController.↵
createCompetition (
    @Valid @RequestBody CompetitionRequestDTO dto)
```

Definition at line 44 of file [CompetitionController.java](#).

8.11.3.2 deleteCompetition()

```
ResponseEntity< Void > com.example.football_manager.controller.CompetitionController.↵
deleteCompetition (
    @PathVariable Long id)
```

Definition at line 104 of file [CompetitionController.java](#).

8.11.3.3 getAllCompetitions()

```
ResponseEntity< List< Competition > > com.example.football_manager.controller.Competition↵
Controller.getAllCompetitions ()
```

Definition at line 91 of file [CompetitionController.java](#).

8.11.3.4 getCompetitionById()

```
ResponseEntity< Competition > com.example.football_manager.controller.CompetitionController.↵
getCompetitionById (
    @PathVariable Long id)
```

Definition at line 79 of file [CompetitionController.java](#).

8.11.3.5 updateCompetition()

```
ResponseEntity< Competition > com.example.football_manager.controller.CompetitionController.↵
updateCompetition (
    @PathVariable Long id,
    @Valid @RequestBody CompetitionRequestDTO dto)
```

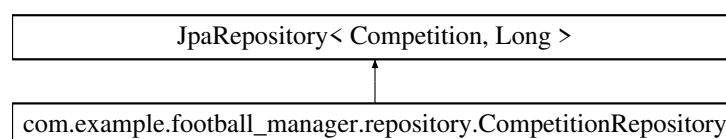
Definition at line 64 of file [CompetitionController.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/CompetitionController.java](#)

8.12 com.example.football_manager.repository.CompetitionRepository Interface Reference

Inheritance diagram for com.example.football_manager.repository.CompetitionRepository:



8.12.1 Detailed Description

Definition at line 8 of file [CompetitionRepository.java](#).

The documentation for this interface was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/repository/CompetitionRepository.java](#)

8.13 com.example.football_manager.dto.CompetitionRequestDTO Class Reference

Public Member Functions

- [CompetitionRequestDTO](#) ()
- [CompetitionRequestDTO](#) (String name)
- String [getName](#) ()
- void [setName](#) (String name)

8.13.1 Detailed Description

Definition at line 7 of file [CompetitionRequestDTO.java](#).

8.13.2 Constructor & Destructor Documentation

8.13.2.1 CompetitionRequestDTO() [1/2]

`com.example.football_manager.dto.CompetitionRequestDTO.CompetitionRequestDTO ()`

Definition at line 14 of file [CompetitionRequestDTO.java](#).

8.13.2.2 CompetitionRequestDTO() [2/2]

`com.example.football_manager.dto.CompetitionRequestDTO.CompetitionRequestDTO (
String name)`

Definition at line 17 of file [CompetitionRequestDTO.java](#).

8.13.3 Member Function Documentation

8.13.3.1 getName()

`String com.example.football_manager.dto.CompetitionRequestDTO.getName ()`

Definition at line 21 of file [CompetitionRequestDTO.java](#).

8.13.3.2 setName()

`void com.example.football_manager.dto.CompetitionRequestDTO.setName (
String name)`

Definition at line 25 of file [CompetitionRequestDTO.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/dto/CompetitionRequestDTO.java](#)

8.14 com.example.football_manager.service.CompetitionService Class Reference

Public Member Functions

- [CompetitionService](#) ([CompetitionRepository](#) competitionRepository)
- [Competition](#) [createCompetition](#) ([CompetitionRequestDTO](#) dto)
- [Competition](#) [updateCompetition](#) (Long id, [CompetitionRequestDTO](#) dto)

- Optional< [Competition](#) > [getCompetitionById](#) (Long id)
- List< [Competition](#) > [getAllCompetitions](#) ()
- void [deleteCompetition](#) (Long id)

8.14.1 Detailed Description

Definition at line 12 of file [CompetitionService.java](#).

8.14.2 Constructor & Destructor Documentation

8.14.2.1 CompetitionService()

```
com.example.football_manager.service.CompetitionService.CompetitionService (
    CompetitionRepository competitionRepository)
```

Definition at line 16 of file [CompetitionService.java](#).

8.14.3 Member Function Documentation

8.14.3.1 createCompetition()

```
Competition com.example.football_manager.service.CompetitionService.createCompetition (
    CompetitionRequestDTO dto)
```

Definition at line 20 of file [CompetitionService.java](#).

8.14.3.2 deleteCompetition()

```
void com.example.football_manager.service.CompetitionService.deleteCompetition (
    Long id)
```

Definition at line 42 of file [CompetitionService.java](#).

8.14.3.3 getAllCompetitions()

```
List< Competition > com.example.football_manager.service.CompetitionService.getAllCompetitions
()
```

Definition at line 38 of file [CompetitionService.java](#).

8.14.3.4 getCompetitionById()

```
Optional< Competition > com.example.football_manager.service.CompetitionService.getCompetition↔
ById (
    Long id)
```

Definition at line 34 of file [CompetitionService.java](#).

8.14.3.5 updateCompetition()

```
Competition com.example.football_manager.service.CompetitionService.updateCompetition (
    Long id,
    CompetitionRequestDTO dto)
```

Definition at line 26 of file [CompetitionService.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/service/CompetitionService.java](#)

8.15 com.example.football_manager.model.Country Class Reference

8.15.1 Detailed Description

Definition at line 13 of file [Country.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/model/Country.java](#)

8.16 com.example.football_manager.controller.CountryController Class Reference

Public Member Functions

- [CountryController](#) ([CountryService](#) countryService)
- [ResponseEntity](#)< [Country](#) > [createCountry](#) (@Valid @RequestBody [CountryRequestDTO](#) dto)
- [ResponseEntity](#)< List< [Country](#) > > [getAllCountries](#) ()
- [ResponseEntity](#)< Void > [deleteCountry](#) (@PathVariable Long id)

8.16.1 Detailed Description

Definition at line 22 of file [CountryController.java](#).

8.16.2 Constructor & Destructor Documentation

8.16.2.1 CountryController()

```
com.example.football_manager.controller.CountryController.CountryController (
    CountryService countryService)
```

Definition at line 26 of file [CountryController.java](#).

8.16.3 Member Function Documentation

8.16.3.1 createCountry()

```
ResponseEntity< Country > com.example.football_manager.controller.CountryController.create↵
Country (
    @Valid @RequestBody CountryRequestDTO dto)
```

Definition at line 44 of file [CountryController.java](#).

8.16.3.2 deleteCountry()

```
ResponseEntity< Void > com.example.football_manager.controller.CountryController.deleteCountry
(
    @PathVariable Long id)
```

Definition at line 71 of file [CountryController.java](#).

8.16.3.3 getAllCountries()

```
ResponseEntity< List< Country > > com.example.football_manager.controller.CountryController.↵
getAllCountries ()
```

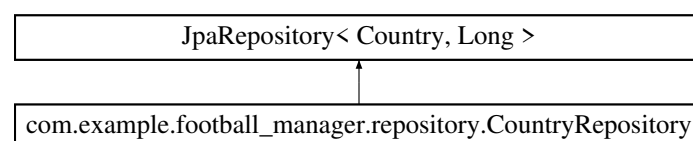
Definition at line 57 of file [CountryController.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/CountryController.java](#)

8.17 com.example.football_manager.repository.CountryRepository Interface Reference

Inheritance diagram for com.example.football_manager.repository.CountryRepository:



8.17.1 Detailed Description

Definition at line 8 of file [CountryRepository.java](#).

The documentation for this interface was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/repository/CountryRepository.java](#)

8.18 com.example.football_manager.dto.CountryRequestDTO Class Reference

Public Member Functions

- [CountryRequestDTO](#) ()
- [CountryRequestDTO](#) (String name)
- String [getName](#) ()
- void [setName](#) (String name)

8.18.1 Detailed Description

Definition at line 7 of file [CountryRequestDTO.java](#).

8.18.2 Constructor & Destructor Documentation

8.18.2.1 CountryRequestDTO() [1/2]

```
com.example.football_manager.dto.CountryRequestDTO.CountryRequestDTO ()
```

Definition at line 14 of file [CountryRequestDTO.java](#).

8.18.2.2 CountryRequestDTO() [2/2]

```
com.example.football_manager.dto.CountryRequestDTO.CountryRequestDTO (  
    String name)
```

Definition at line 17 of file [CountryRequestDTO.java](#).

8.18.3 Member Function Documentation

8.18.3.1 getName()

```
String com.example.football_manager.dto.CountryRequestDTO.getName ()
```

Definition at line 21 of file [CountryRequestDTO.java](#).

8.18.3.2 setName()

```
void com.example.football_manager.dto.CountryRequestDTO.setName (  
    String name)
```

Definition at line 25 of file [CountryRequestDTO.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/dto/CountryRequestDTO.java](#)

8.19 com.example.football_manager.service.CountryService Class Reference

Public Member Functions

- [CountryService](#) ([CountryRepository](#) countryRepository)
- [Country](#) [createCountry](#) ([CountryRequestDTO](#) dto)
- List< [Country](#) > [getAllCountries](#) ()
- void [deleteCountry](#) (Long id)

8.19.1 Detailed Description

Definition at line 11 of file [CountryService.java](#).

8.19.2 Constructor & Destructor Documentation

8.19.2.1 CountryService()

```
com.example.football_manager.service.CountryService.CountryService (
    CountryRepository countryRepository)
```

Definition at line 15 of file [CountryService.java](#).

8.19.3 Member Function Documentation

8.19.3.1 createCountry()

```
Country com.example.football_manager.service.CountryService.createCountry (
    CountryRequestDTO dto)
```

Definition at line 19 of file [CountryService.java](#).

8.19.3.2 deleteCountry()

```
void com.example.football_manager.service.CountryService.deleteCountry (
    Long id)
```

Definition at line 29 of file [CountryService.java](#).

8.19.3.3 getAllCountries()

```
List< Country > com.example.football_manager.service.CountryService.getAllCountries ()
```

Definition at line 25 of file [CountryService.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/service/CountryService.java](#)

8.20 com.example.football_manager.controller.FavouriteTeamController Class Reference

Public Member Functions

- [FavouriteTeamController](#) ([UserService](#) userService)
- [ResponseEntity< String > addFavouriteTeam](#) (@PathVariable Long teamId, HttpSession session)
- [ResponseEntity< String > removeFavouriteTeam](#) (@PathVariable Long teamId, HttpSession session)

8.20.1 Detailed Description

Definition at line 20 of file [FavouriteTeamController.java](#).

8.20.2 Constructor & Destructor Documentation

8.20.2.1 FavouriteTeamController()

```
com.example.football_manager.controller.FavouriteTeamController.FavouriteTeamController (
    UserService userService)
```

Definition at line 24 of file [FavouriteTeamController.java](#).

8.20.3 Member Function Documentation

8.20.3.1 addFavouriteTeam()

```
ResponseEntity< String > com.example.football_manager.controller.FavouriteTeamController.add↵
FavouriteTeam (
    @PathVariable Long teamId,
    HttpSession session)
```

Definition at line 39 of file [FavouriteTeamController.java](#).

8.20.3.2 removeFavouriteTeam()

```
ResponseEntity< String > com.example.football_manager.controller.FavouriteTeamController.↵
removeFavouriteTeam (
    @PathVariable Long teamId,
    HttpSession session)
```

Definition at line 64 of file [FavouriteTeamController.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/FavouriteTeamController.java](#)

8.21 com.example.football_manager.FootballManagerApplication Class Reference

Static Public Member Functions

- static void [main](#) (String[] args)

8.21.1 Detailed Description

Definition at line 7 of file [FootballManagerApplication.java](#).

8.21.2 Member Function Documentation

8.21.2.1 main()

```
static void com.example.football_manager.FootballManagerApplication.main (
    String[] args) [static]
```

Definition at line 9 of file [FootballManagerApplication.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/FootballManagerApplication.java](#)

8.22 com.example.football_manager.controller.GlobalModelAttributes Class Reference

Public Member Functions

- void [addIsAdmin](#) (Model model, HttpServletRequest request)

8.22.1 Detailed Description

Definition at line 10 of file [GlobalModelAttributes.java](#).

8.22.2 Member Function Documentation

8.22.2.1 addIsAdmin()

```
void com.example.football_manager.controller.GlobalModelAttributes.addIsAdmin (
    Model model,
```

```
HttpServletRequest request)
```

Definition at line 13 of file [GlobalModelAttributes.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/GlobalModelAttributes.java](#)

8.23 com.example.football_manager.dto.MatchResultRequestDTO.GoalDTO Class Reference

Public Member Functions

- Long [getTeamId](#) ()
- void [setTeamId](#) (Long teamId)
- Integer [getMinute](#) ()
- void [setMinute](#) (Integer minute)
- Integer [getStoppageMinute](#) ()
- void [setStoppageMinute](#) (Integer stoppageMinute)

8.23.1 Detailed Description

Definition at line 25 of file [MatchResultRequestDTO.java](#).

8.23.2 Member Function Documentation

8.23.2.1 getMinute()

```
Integer com.example.football_manager.dto.MatchResultRequestDTO.GoalDTO.getMinute ()
```

Definition at line 50 of file [MatchResultRequestDTO.java](#).

8.23.2.2 getStoppageMinute()

```
Integer com.example.football_manager.dto.MatchResultRequestDTO.GoalDTO.getStoppageMinute ()
```

Definition at line 58 of file [MatchResultRequestDTO.java](#).

8.23.2.3 getTeamId()

```
Long com.example.football_manager.dto.MatchResultRequestDTO.GoalDTO.getTeamId ()
```

Definition at line 42 of file [MatchResultRequestDTO.java](#).

8.23.2.4 setMinute()

```
void com.example.football_manager.dto.MatchResultRequestDTO.GoalDTO.setMinute (
    Integer minute)
```

Definition at line 54 of file [MatchResultRequestDTO.java](#).

8.23.2.5 setStoppageMinute()

```
void com.example.football_manager.dto.MatchResultRequestDTO.GoalDTO.setStoppageMinute (
    Integer stoppageMinute)
```

Definition at line 62 of file [MatchResultRequestDTO.java](#).

8.23.2.6 setTeamId()

```
void com.example.football_manager.dto.MatchResultRequestDTO.GoalDTO.setTeamId (
    Long teamId)
```

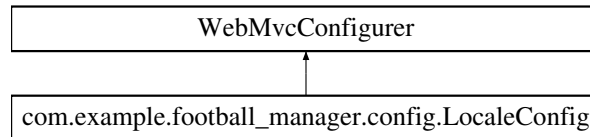
Definition at line 46 of file [MatchResultRequestDTO.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/dto/MatchResultRequestDTO.java](#)

8.24 com.example.football_manager.config.LocaleConfig Class Reference

Inheritance diagram for com.example.football_manager.config.LocaleConfig:



Public Member Functions

- LocaleResolver [localeResolver](#) ()
- LocaleChangeInterceptor [localeChangeInterceptor](#) ()
- void [addInterceptors](#) (InterceptorRegistry registry)

8.24.1 Detailed Description

Definition at line 13 of file [LocaleConfig.java](#).

8.24.2 Member Function Documentation

8.24.2.1 addInterceptors()

```
void com.example.football_manager.config.LocaleConfig.addInterceptors (
    InterceptorRegistry registry)
```

Definition at line 30 of file [LocaleConfig.java](#).

References [localeChangeInterceptor\(\)](#).

8.24.2.2 localeChangeInterceptor()

```
LocaleChangeInterceptor com.example.football_manager.config.LocaleConfig.localeChangeInterceptor
()
```

Definition at line 23 of file [LocaleConfig.java](#).

Referenced by [addInterceptors\(\)](#).

8.24.2.3 localeResolver()

```
LocaleResolver com.example.football_manager.config.LocaleConfig.localeResolver ()
```

Definition at line 16 of file [LocaleConfig.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/config/LocaleConfig.java](#)

8.25 com.example.football_manager.dto.LoginDTO Class Reference

Public Member Functions

- String [getUsername](#) ()
- void [setUsername](#) (String username)
- String [getPassword](#) ()
- void [setPassword](#) (String password)

8.25.1 Detailed Description

Definition at line 6 of file [LoginDTO.java](#).

8.25.2 Member Function Documentation

8.25.2.1 getPassword()

`String com.example.football_manager.dto.LoginDTO.getPassword ()`

Definition at line 24 of file [LoginDTO.java](#).

8.25.2.2 getUsername()

`String com.example.football_manager.dto.LoginDTO.getUsername ()`

Definition at line 16 of file [LoginDTO.java](#).

8.25.2.3 setPassword()

`void com.example.football_manager.dto.LoginDTO.setPassword (
String password)`

Definition at line 28 of file [LoginDTO.java](#).

8.25.2.4 setUsername()

`void com.example.football_manager.dto.LoginDTO.setUsername (
String username)`

Definition at line 20 of file [LoginDTO.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/dto/LoginDTO.java](#)

8.26 com.example.football_manager.model.Match Class Reference

8.26.1 Detailed Description

Definition at line 12 of file [Match.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/model/Match.java](#)

8.27 com.example.football_manager.controller.MatchController Class Reference

Public Member Functions

- `ResponseEntity< List< Match > > getMatches ( @RequestParam(required=false) Long teamId, @RequestParam(required=false) MatchRequestDTO.MatchStatus status, @RequestParam(required=false) Long competitionId)`
- `ResponseEntity< List< Match > > getUpcomingMatches ()`
- `ResponseEntity< String > createMatchFromJson (@Valid @RequestBody MatchRequestDTO matchDTO)`
- `ResponseEntity< String > createMatchFromForm (@Valid @ModelAttribute MatchRequestDTO matchDTO)`
- `ResponseEntity< List< MatchResultDTO > > getMatchResults ()`
- `ResponseEntity< String > updateMatch (@PathVariable Long id, @RequestBody MatchRequestDTO matchDTO)`
- `ResponseEntity< String > deleteMatch (@PathVariable Long id)`
- `ResponseEntity< String > registerResult ( @PathVariable Long id, @Valid @RequestBody MatchResultRequestDTO resultDTO)`

8.27.1 Detailed Description

Definition at line 27 of file [MatchController.java](#).

8.27.2 Member Function Documentation

8.27.2.1 createMatchFromForm()

```
ResponseEntity< String > com.example.football_manager.controller.MatchController.createMatch↵
FromForm (
    @Valid @ModelAttribute MatchRequestDTO matchDTO)
```

Definition at line 93 of file [MatchController.java](#).

8.27.2.2 createMatchFromJson()

```
ResponseEntity< String > com.example.football_manager.controller.MatchController.createMatch↵
FromJson (
    @Valid @RequestBody MatchRequestDTO matchDTO)
```

Definition at line 75 of file [MatchController.java](#).

8.27.2.3 deleteMatch()

```
ResponseEntity< String > com.example.football_manager.controller.MatchController.deleteMatch (
    @PathVariable Long id)
```

Definition at line 150 of file [MatchController.java](#).

8.27.2.4 getMatches()

```
ResponseEntity< List< Match > > com.example.football_manager.controller.MatchController.get↵
Matches (
    @RequestParam(required=false) Long teamId,
    @RequestParam(required=false) MatchRequestDTO.MatchStatus status,
    @RequestParam(required=false) Long competitionId)
```

Definition at line 41 of file [MatchController.java](#).

8.27.2.5 getMatchResults()

```
ResponseEntity< List< MatchResultDTO > > com.example.football_manager.controller.Match↵
Controller.getMatchResults ()
```

Definition at line 114 of file [MatchController.java](#).

8.27.2.6 getUpcomingMatches()

```
ResponseEntity< List< Match > > com.example.football_manager.controller.MatchController.get↵
UpcomingMatches ()
```

Definition at line 57 of file [MatchController.java](#).

References [getUpcomingMatches\(\)](#).

Referenced by [getUpcomingMatches\(\)](#).

8.27.2.7 registerResult()

```
ResponseEntity< String > com.example.football_manager.controller.MatchController.register↵
Result (
    @PathVariable Long id,
    @Valid @RequestBody MatchResultRequestDTO resultDTO)
```

Definition at line 172 of file [MatchController.java](#).

8.27.2.8 updateMatch()

```
ResponseEntity< String > com.example.football_manager.controller.MatchController.updateMatch (
    @PathVariable Long id,
    @RequestBody MatchRequestDTO matchDTO)
```

Definition at line 133 of file [MatchController.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/MatchController.java](#)

8.28 com.example.football_manager.model.MatchGoal Class Reference

8.28.1 Detailed Description

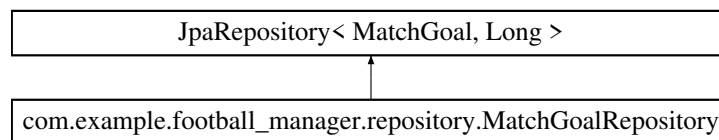
Definition at line 13 of file [MatchGoal.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/model/MatchGoal.java](#)

8.29 com.example.football_manager.repository.MatchGoalRepository Interface Reference

Inheritance diagram for com.example.football_manager.repository.MatchGoalRepository:



Public Member Functions

- void [deleteByMatchId](#) (Long matchId)
- List< [MatchGoal](#) > [findByMatchIdOrderByMinuteAscStoppageMinuteAscIdAsc](#) (Long matchId)

8.29.1 Detailed Description

Definition at line 10 of file [MatchGoalRepository.java](#).

8.29.2 Member Function Documentation

8.29.2.1 deleteByMatchId()

```
void com.example.football_manager.repository.MatchGoalRepository.deleteByMatchId (
    Long matchId)
```

8.29.2.2 findByMatchIdOrderByMinuteAscStoppageMinuteAscIdAsc()

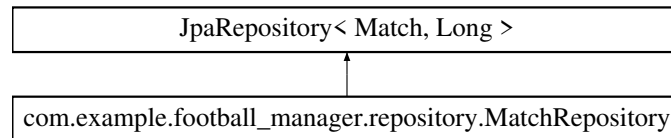
```
List< MatchGoal > com.example.football_manager.repository.MatchGoalRepository.findByMatchId↔
OrderByMinuteAscStoppageMinuteAscIdAsc (
    Long matchId)
```

The documentation for this interface was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/repository/MatchGoalRepository.java](#)

8.30 com.example.football_manager.repository.MatchRepository Interface Reference

Inheritance diagram for com.example.football_manager.repository.MatchRepository:



Public Member Functions

- List< [Match](#) > [findByFilters](#) (@Param("teamId") Long teamId, @Param("finished") Boolean finished, @Param("competitionId") Long competitionId)
- List< [Match](#) > [findByFinishedTrueOrderByDatetimeDesc](#) ()
- List< [Match](#) > [findByFinishedTrueAndLeftTeamIdOrFinishedTrueAndRightTeamIdOrderByDatetimeDesc](#) (Long leftTeamId, Long rightTeamId)
- List< [Match](#) > [findByFinishedFalseAndDatetimeAfterAndLeftTeamIdOrFinishedFalseAndDatetimeAfterAndRightTeamIdOrderByDatetimeAsc](#) (OffsetDateTime now1, Long leftTeamId, OffsetDateTime now2, Long rightTeamId)
- List< [Match](#) > [findByFinishedFalseAndDatetimeAfterAndLeftTeamIdInOrFinishedFalseAndDatetimeAfterAndRightTeamIdInOrderByDatetimeAsc](#) (OffsetDateTime now1, Collection< Long > leftTeamIds, OffsetDateTime now2, Collection< Long > rightTeamIds)

8.30.1 Detailed Description

Definition at line 14 of file [MatchRepository.java](#).

8.30.2 Member Function Documentation

8.30.2.1 findByFilters()

```
List< Match > com.example.football_manager.repository.MatchRepository.findByFilters (
    @Param("teamId") Long teamId,
    @Param("finished") Boolean finished,
    @Param("competitionId") Long competitionId)
```

References [findByFilters\(\)](#).

Referenced by [findByFilters\(\)](#).

8.30.2.2 findByFinishedFalseAndDatetimeAfterAndLeftTeamIdInOrFinishedFalseAndDatetimeAfterAndRightTeamIdInOrderByDatetimeAsc

```
List< Match > com.example.football_manager.repository.MatchRepository.findByFinishedFalseAndDatetimeAfterAndLeftTeamIdInOrFinishedFalseAndDatetimeAfterAndRightTeamIdInOrderByDatetimeAsc (
    OffsetDateTime now1,
    Collection< Long > leftTeamIds,
    OffsetDateTime now2,
    Collection< Long > rightTeamIds)
```

References [findByFinishedFalseAndDatetimeAfterAndLeftTeamIdInOrFinishedFalseAndDatetimeAfterAndRightTeamIdInOrderByDatetimeAsc](#).

Referenced by [findByFinishedFalseAndDatetimeAfterAndLeftTeamIdInOrFinishedFalseAndDatetimeAfterAndRightTeamIdInOrderByDatetimeAsc](#).

8.30.2.3 findByFinishedFalseAndDatetimeAfterAndLeftTeamIdOrFinishedFalseAndDatetimeAfterAndRightTeamIdOrderByDatetimeAsc

```
List< Match > com.example.football_manager.repository.MatchRepository.findByFinishedFalseAndDatetimeAfterAndLeftTeamIdOrFinishedFalseAndDatetimeAfterAndRightTeamIdOrderByDatetimeAsc (
    OffsetDateTime now1,
    Long leftTeamId,
    OffsetDateTime now2,
    Long rightTeamId)
```

References [findByFinishedFalseAndDatetimeAfterAndLeftTeamIdOrFinishedFalseAndDatetimeAfterAndRightTeamIdOrderByDatetimeAsc](#).

Referenced by [findByFinishedFalseAndDatetimeAfterAndLeftTeamIdOrFinishedFalseAndDatetimeAfterAndRightTeamIdOrderByDatetimeAsc](#).

8.30.2.4 findByFinishedTrueAndLeftTeamIdOrFinishedTrueAndRightTeamIdOrderByDatetimeDesc()

```
List< Match > com.example.football_manager.repository.MatchRepository.findByFinishedTrueAnd←
LeftTeamIdOrFinishedTrueAndRightTeamIdOrderByDatetimeDesc (
    Long leftTeamId,
    Long rightTeamId)
```

References [findByFinishedTrueAndLeftTeamIdOrFinishedTrueAndRightTeamIdOrderByDatetimeDesc\(\)](#).

Referenced by [findByFinishedTrueAndLeftTeamIdOrFinishedTrueAndRightTeamIdOrderByDatetimeDesc\(\)](#).

8.30.2.5 findByFinishedTrueOrderByDatetimeDesc()

```
List< Match > com.example.football_manager.repository.MatchRepository.findByFinishedTrue←
OrderByDatetimeDesc ()
```

References [findByFinishedTrueOrderByDatetimeDesc\(\)](#).

Referenced by [findByFinishedTrueOrderByDatetimeDesc\(\)](#).

The documentation for this interface was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/repository/MatchRepository.java](#)

8.31 com.example.football_manager.dto.MatchRequestDTO Class Reference

Classes

- enum [MatchStatus](#)

Public Member Functions

- Long [getHomeTeamId](#) ()
- void [setHomeTeamId](#) (Long homeTeamId)
- Long [getAwayTeamId](#) ()
- void [setAwayTeamId](#) (Long awayTeamId)
- Long [getCompetitionId](#) ()
- void [setCompetitionId](#) (Long competitionId)
- LocalDateTime [getKickoffTime](#) ()
- void [setKickoffTime](#) (LocalDateTime kickoffTime)
- String [getVenue](#) ()
- void [setVenue](#) (String venue)
- [MatchStatus](#) [getStatus](#) ()
- void [setStatus](#) ([MatchStatus](#) status)
- Integer [getHomeScore](#) ()
- void [setHomeScore](#) (Integer homeScore)
- Integer [getAwayScore](#) ()
- void [setAwayScore](#) (Integer awayScore)

8.31.1 Detailed Description

Definition at line 9 of file [MatchRequestDTO.java](#).

8.31.2 Member Function Documentation

8.31.2.1 getAwayScore()

```
Integer com.example.football_manager.dto.MatchRequestDTO.getAwayScore ()
```

Definition at line 59 of file [MatchRequestDTO.java](#).

Referenced by [com.example.football_manager.service.MatchService.createMatch\(\)](#), and [com.example.football_manager.service.Mat](#)

8.31.2.2 getAwayTeamId()

`Long com.example.football_manager.dto.MatchRequestDTO.getAwayTeamId ()`

Definition at line 41 of file [MatchRequestDTO.java](#).

Referenced by [com.example.football_manager.service.MatchService.createMatch\(\)](#), and [com.example.football_manager.service.Mat](#)

8.31.2.3 getCompetitionId()

`Long com.example.football_manager.dto.MatchRequestDTO.getCompetitionId ()`

Definition at line 44 of file [MatchRequestDTO.java](#).

Referenced by [com.example.football_manager.service.MatchService.createMatch\(\)](#).

8.31.2.4 getHomeScore()

`Integer com.example.football_manager.dto.MatchRequestDTO.getHomeScore ()`

Definition at line 56 of file [MatchRequestDTO.java](#).

Referenced by [com.example.football_manager.service.MatchService.createMatch\(\)](#), and [com.example.football_manager.service.Mat](#)

8.31.2.5 getHomeTeamId()

`Long com.example.football_manager.dto.MatchRequestDTO.getHomeTeamId ()`

Definition at line 38 of file [MatchRequestDTO.java](#).

References [getHomeTeamId\(\)](#).

Referenced by [com.example.football_manager.service.MatchService.createMatch\(\)](#), [getHomeTeamId\(\)](#), and [com.example.football_manager.service.MatchService.updateMatch\(\)](#).

8.31.2.6 getKickoffTime()

`LocalDateTime com.example.football_manager.dto.MatchRequestDTO.getKickoffTime ()`

Definition at line 47 of file [MatchRequestDTO.java](#).

Referenced by [com.example.football_manager.service.MatchService.createMatch\(\)](#), and [com.example.football_manager.service.Mat](#)

8.31.2.7 getStatus()

`MatchStatus com.example.football_manager.dto.MatchRequestDTO.getStatus ()`

Definition at line 53 of file [MatchRequestDTO.java](#).

Referenced by [com.example.football_manager.service.MatchService.createMatch\(\)](#), and [com.example.football_manager.service.Mat](#)

8.31.2.8 getVenue()

`String com.example.football_manager.dto.MatchRequestDTO.getVenue ()`

Definition at line 50 of file [MatchRequestDTO.java](#).

Referenced by [com.example.football_manager.service.MatchService.createMatch\(\)](#), and [com.example.football_manager.service.Mat](#)

8.31.2.9 setAwayScore()

`void com.example.football_manager.dto.MatchRequestDTO.setAwayScore (
 Integer awayScore)`

Definition at line 60 of file [MatchRequestDTO.java](#).

8.31.2.10 setAwayTeamId()

`void com.example.football_manager.dto.MatchRequestDTO.setAwayTeamId (
 Long awayTeamId)`

Definition at line 42 of file [MatchRequestDTO.java](#).

8.31.2.11 setCompetitionId()

`void com.example.football_manager.dto.MatchRequestDTO.setCompetitionId (
 Long competitionId)`

Definition at line 45 of file [MatchRequestDTO.java](#).

8.31.2.12 setHomeScore()

```
void com.example.football_manager.dto.MatchRequestDTO.setHomeScore (
    Integer homeScore)
```

Definition at line 57 of file [MatchRequestDTO.java](#).

8.31.2.13 setHomeTeamId()

```
void com.example.football_manager.dto.MatchRequestDTO.setHomeTeamId (
    Long homeTeamId)
```

Definition at line 39 of file [MatchRequestDTO.java](#).

8.31.2.14 setKickoffTime()

```
void com.example.football_manager.dto.MatchRequestDTO.setKickoffTime (
    LocalDateTime kickoffTime)
```

Definition at line 48 of file [MatchRequestDTO.java](#).

8.31.2.15 setStatus()

```
void com.example.football_manager.dto.MatchRequestDTO.setStatus (
    MatchStatus status)
```

Definition at line 54 of file [MatchRequestDTO.java](#).

8.31.2.16 setVenue()

```
void com.example.football_manager.dto.MatchRequestDTO.setVenue (
    String venue)
```

Definition at line 51 of file [MatchRequestDTO.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/dto/MatchRequestDTO.java](#)

8.32 com.example.football_manager.dto.MatchResultRequestDTO Class Reference

Classes

- class [GoalDTO](#)

Public Member Functions

- List< [GoalDTO](#) > [getGoals](#) ()
- void [setGoals](#) (List< [GoalDTO](#) > goals)

8.32.1 Detailed Description

Definition at line 11 of file [MatchResultRequestDTO.java](#).

8.32.2 Member Function Documentation

8.32.2.1 getGoals()

```
List< GoalDTO > com.example.football_manager.dto.MatchResultRequestDTO.getGoals ()
```

Definition at line 17 of file [MatchResultRequestDTO.java](#).

Referenced by [com.example.football_manager.service.MatchService.registerResult\(\)](#).

8.32.2.2 setGoals()

```
void com.example.football_manager.dto.MatchResultRequestDTO.setGoals (
    List< GoalDTO > goals)
```

Definition at line 21 of file [MatchResultRequestDTO.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/dto/MatchResultRequestDTO.java](#)

8.33 com.example.football_manager.service.MatchService Class Reference

Public Member Functions

- [MatchService](#) ([MatchRepository](#) matchRepository, [MatchGoalRepository](#) matchGoalRepository, [TeamRepository](#) teamRepository, [CompetitionRepository](#) competitionRepository)
- String [createMatch](#) ([MatchRequestDTO](#) request)
- String [updateMatch](#) (Long id, [MatchRequestDTO](#) request)
- String [deleteMatch](#) (Long id)
- String [registerResult](#) (Long id, [MatchResultRequestDTO](#) request)
- List< [MatchResultDTO](#) > [getFinishedMatchResults](#) ()
- Optional< [MatchRequestDTO](#) > [getMatchForEdit](#) (Long id)
- List< [Match](#) > [getAllMatches](#) ()
- List< [Match](#) > [getUpcomingMatches](#) ()
- List< [Match](#) > [getMatches](#) (Long teamId, [MatchRequestDTO.MatchStatus](#) status, Long competitionId)
- [Match](#) [getMatchById](#) (Long id)
- List< [MatchGoal](#) > [getGoalsByMatchId](#) (Long matchId)
- List< [Match](#) > [getRecentMatchesByTeamId](#) (Long teamId)
- List< [Match](#) > [getFutureMatchesByTeamId](#) (Long teamId)
- List< [Match](#) > [getFutureMatchesByFavouriteTeamIds](#) (Set< Long > favouriteTeamIds)

8.33.1 Detailed Description

Definition at line 29 of file [MatchService.java](#).

8.33.2 Constructor & Destructor Documentation

8.33.2.1 MatchService()

```
com.example.football_manager.service.MatchService.MatchService (
    MatchRepository matchRepository,
    MatchGoalRepository matchGoalRepository,
    TeamRepository teamRepository,
    CompetitionRepository competitionRepository)
```

Definition at line 37 of file [MatchService.java](#).

8.33.3 Member Function Documentation

8.33.3.1 createMatch()

```
String com.example.football_manager.service.MatchService.createMatch (
    MatchRequestDTO request)
```

Definition at line 51 of file [MatchService.java](#).

References [com.example.football_manager.dto.MatchRequestDTO.MatchStatus.FINISHED](#), [com.example.football_manager.dto.MatchRequestDTO.getAwayTeamId\(\)](#), [com.example.football_manager.dto.MatchRequestDTO.getHomeScore\(\)](#), [com.example.football_manager.dto.MatchRequestDTO.getKickoffTime\(\)](#), [com.example.football_manager.dto.MatchRequestDTO.getVenue\(\)](#), and [com.example.football_manager.dto.MatchRequestDTO.getVenue\(\)](#).

8.33.3.2 deleteMatch()

```
String com.example.football_manager.service.MatchService.deleteMatch (
    Long id)
```

Definition at line 146 of file [MatchService.java](#).

8.33.3.3 getAllMatches()

```
List< Match > com.example.football_manager.service.MatchService.getAllMatches ()
```

Definition at line 270 of file [MatchService.java](#).

Referenced by [getMatches\(\)](#).

8.33.3.4 getFinishedMatchResults()

```
List< MatchResultDTO > com.example.football_manager.service.MatchService.getFinishedMatch←
Results ()
```

Definition at line 199 of file [MatchService.java](#).

References [getFinishedMatchResults\(\)](#).

Referenced by [getFinishedMatchResults\(\)](#).

8.33.3.5 getFutureMatchesByFavouriteTeamIds()

```
List< Match > com.example.football_manager.service.MatchService.getFutureMatchesByFavourite←
TeamIds (
    Set< Long > favouriteTeamIds)
```

Definition at line 322 of file [MatchService.java](#).

8.33.3.6 getFutureMatchesByTeamId()

```
List< Match > com.example.football_manager.service.MatchService.getFutureMatchesByTeamId (
    Long teamId)
```

Definition at line 310 of file [MatchService.java](#).

8.33.3.7 getGoalsByMatchId()

```
List< MatchGoal > com.example.football_manager.service.MatchService.getGoalsByMatchId (
    Long matchId)
```

Definition at line 298 of file [MatchService.java](#).

8.33.3.8 getMatchById()

```
Match com.example.football_manager.service.MatchService.getMatchById (
    Long id)
```

Definition at line 293 of file [MatchService.java](#).

8.33.3.9 getMatches()

```
List< Match > com.example.football_manager.service.MatchService.getMatches (
    Long teamId,
    MatchRequestDTO.MatchStatus status,
    Long competitionId)
```

Definition at line 283 of file [MatchService.java](#).

References [getAllMatches\(\)](#), and [getMatches\(\)](#).

Referenced by [getMatches\(\)](#).

8.33.3.10 getMatchForEdit()

```
Optional< MatchRequestDTO > com.example.football_manager.service.MatchService.getMatchForEdit
(
    Long id)
```

Definition at line 214 of file [MatchService.java](#).

References [com.example.football_manager.dto.MatchRequestDTO.MatchStatus.FINISHED](#), and [com.example.football_manager.dto.](#)

8.33.3.11 getRecentMatchesByTeamId()

```
List< Match > com.example.football_manager.service.MatchService.getRecentMatchesByTeamId (
    Long teamId)
```

Definition at line 302 of file [MatchService.java](#).

8.33.3.12 getUpcomingMatches()

```
List< Match > com.example.football_manager.service.MatchService.getUpcomingMatches ()
```

Definition at line 275 of file [MatchService.java](#).

References [getUpcomingMatches\(\)](#).

Referenced by [getUpcomingMatches\(\)](#).

8.33.3.13 registerResult()

```
String com.example.football_manager.service.MatchService.registerResult (
    Long id,
    MatchResultRequestDTO request)
```

Definition at line 158 of file [MatchService.java](#).

References [com.example.football_manager.dto.MatchResultRequestDTO.getGoals\(\)](#).

8.33.3.14 updateMatch()

```
String com.example.football_manager.service.MatchService.updateMatch (
    Long id,
    MatchRequestDTO request)
```

Definition at line 106 of file [MatchService.java](#).

References [com.example.football_manager.dto.MatchRequestDTO.MatchStatus.FINISHED](#), [com.example.football_manager.dto.MatchRequestDTO.MatchStatus.SCHEDULED](#), [com.example.football_manager.dto.MatchRequestDTO.MatchStatus.IN_PROGRESS](#), [com.example.football_manager.dto.MatchRequestDTO.MatchStatus.FINISHED](#), [com.example.football_manager.dto.MatchRequestDTO.MatchStatus.CANCELLED](#), [com.example.football_manager.dto.MatchRequestDTO.getAwayTeamId\(\)](#), [com.example.football_manager.dto.MatchRequestDTO.getHomeTeamId\(\)](#), [com.example.football_manager.dto.MatchRequestDTO.getStatus\(\)](#), and [com.example.football_manager.dto.MatchRequestDTO.getVenueId\(\)](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/service/MatchService.java](#)

8.34 com.example.football_manager.dto.MatchRequestDTO.MatchStatus Enum Reference

Public Attributes

- [SCHEDULED](#)
- [IN_PROGRESS](#)
- [FINISHED](#)
- [CANCELLED](#)

8.34.1 Detailed Description

Definition at line 11 of file [MatchRequestDTO.java](#).

8.34.2 Member Data Documentation

8.34.2.1 CANCELLED

```
com.example.football_manager.dto.MatchRequestDTO.MatchStatus.CANCELLED
```

Definition at line 15 of file [MatchRequestDTO.java](#).

8.34.2.2 FINISHED

`com.example.football_manager.dto.MatchRequestDTO.MatchStatus.FINISHED`

Definition at line 14 of file [MatchRequestDTO.java](#).

Referenced by [com.example.football_manager.service.MatchService.createMatch\(\)](#), [com.example.football_manager.service.MatchService.updateMatch\(\)](#) and [com.example.football_manager.service.MatchService.updateMatch\(\)](#).

8.34.2.3 IN_PROGRESS

`com.example.football_manager.dto.MatchRequestDTO.MatchStatus.IN_PROGRESS`

Definition at line 13 of file [MatchRequestDTO.java](#).

8.34.2.4 SCHEDULED

`com.example.football_manager.dto.MatchRequestDTO.MatchStatus.SCHEDULED`

Definition at line 12 of file [MatchRequestDTO.java](#).

Referenced by [com.example.football_manager.service.MatchService.getMatchForEdit\(\)](#).

The documentation for this enum was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/dto/MatchRequestDTO.java](#)

8.35 com.example.football_manager.controller.MatchViewController

Class Reference

Public Member Functions

- [MatchViewController](#) ([MatchService](#) matchService, [TeamService](#) teamService)
- String [showMatchesPage](#) ([@RequestParam](#)(required=false) Long teamId, [@RequestParam](#)(required=false) [MatchRequestDTO.MatchStatus](#) status, [@RequestParam](#)(required=false) Long competitionId, [Model](#) model, [HttpSession](#) session)
- String [showScheduleForm](#) ([Model](#) model)
- String [showResultsPage](#) ([Model](#) model)
- String [showEditForm](#) ([@PathVariable](#) Long id, [Model](#) model)
- String [updateMatch](#) ([@PathVariable](#) Long id, [@ModelAttribute](#)("matchRequest") [MatchRequestDTO](#) matchRequest, [Model](#) model)
- String [showScoreForm](#) ([@PathVariable](#) Long id, [Model](#) model)
- String [submitScoreForm](#) ([@PathVariable](#) Long id, [@ModelAttribute](#)("resultRequest") [MatchResultRequestDTO](#) resultRequest, [Model](#) model, [RedirectAttributes](#) redirectAttributes)
- String [saveMatchResult](#) ([@PathVariable](#) Long id, [@ModelAttribute](#) [MatchResultRequestDTO](#) requestDTO)
- String [showUpcomingMatchesPage](#) ([Model](#) model, [HttpSession](#) session)

8.35.1 Detailed Description

Definition at line 26 of file [MatchViewController.java](#).

8.35.2 Constructor & Destructor Documentation

8.35.2.1 MatchViewController()

```
com.example.football_manager.controller.MatchViewController.MatchViewController (
    MatchService matchService,
    TeamService teamService)
```

Definition at line 31 of file [MatchViewController.java](#).

8.35.3 Member Function Documentation

8.35.3.1 saveMatchResult()

```
String com.example.football_manager.controller.MatchViewController.saveMatchResult (
    @PathVariable Long id,
    @ModelAttribute MatchResultRequestDTO requestDTO)
```

Definition at line 144 of file [MatchViewController.java](#).

References [saveMatchResult\(\)](#).

Referenced by [saveMatchResult\(\)](#).

8.35.3.2 showEditForm()

```
String com.example.football_manager.controller.MatchViewController.showEditForm (
    @PathVariable Long id,
    Model model)
```

Definition at line 71 of file [MatchViewController.java](#).

References [showEditForm\(\)](#).

Referenced by [showEditForm\(\)](#).

8.35.3.3 showMatchesPage()

```
String com.example.football_manager.controller.MatchViewController.showMatchesPage (
    @RequestParam(required=false) Long teamId,
    @RequestParam(required=false) MatchRequestDTO.MatchStatus status,
    @RequestParam(required=false) Long competitionId,
    Model model,
    HttpSession session)
```

Definition at line 37 of file [MatchViewController.java](#).

References [showMatchesPage\(\)](#).

Referenced by [showMatchesPage\(\)](#).

8.35.3.4 showResultsPage()

```
String com.example.football_manager.controller.MatchViewController.showResultsPage (
    Model model)
```

Definition at line 65 of file [MatchViewController.java](#).

References [showResultsPage\(\)](#).

Referenced by [showResultsPage\(\)](#).

8.35.3.5 showScheduleForm()

```
String com.example.football_manager.controller.MatchViewController.showScheduleForm (
    Model model)
```

Definition at line 58 of file [MatchViewController.java](#).

References [showScheduleForm\(\)](#).

Referenced by [showScheduleForm\(\)](#).

8.35.3.6 showScoreForm()

```
String com.example.football_manager.controller.MatchViewController.showScoreForm (
    @PathVariable Long id,
    Model model)
```

Definition at line 90 of file [MatchViewController.java](#).

References [showScoreForm\(\)](#).

Referenced by [showScoreForm\(\)](#).

8.35.3.7 showUpcomingMatchesPage()

```
String com.example.football_manager.controller.MatchViewController.showUpcomingMatchesPage (
```

```
Model model,
    HttpSession session)
```

Definition at line 155 of file [MatchViewController.java](#).

References [showUpcomingMatchesPage\(\)](#).

Referenced by [showUpcomingMatchesPage\(\)](#).

8.35.3.8 submitScoreForm()

```
String com.example.football_manager.controller.MatchViewController.submitScoreForm (
    @PathVariable Long id,
    @ModelAttribute("resultRequest") MatchResultRequestDTO resultRequest,
    Model model,
    RedirectAttributes redirectAttributes)
```

Definition at line 102 of file [MatchViewController.java](#).

References [submitScoreForm\(\)](#).

Referenced by [submitScoreForm\(\)](#).

8.35.3.9 updateMatch()

```
String com.example.football_manager.controller.MatchViewController.updateMatch (
    @PathVariable Long id,
    @ModelAttribute("matchRequest") MatchRequestDTO matchDTO)
```

Definition at line 83 of file [MatchViewController.java](#).

References [updateMatch\(\)](#).

Referenced by [updateMatch\(\)](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/MatchViewController.java](#)

8.36 com.example.football_manager.config.OpenApiConfig Class Reference

8.36.1 Detailed Description

Definition at line 24 of file [OpenApiConfig.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/config/OpenApiConfig.java](#)

8.37 com.example.football_manager.model.Player Class Reference

8.37.1 Detailed Description

Definition at line 22 of file [Player.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/model/Player.java](#)

8.38 com.example.football_manager.dto.PlayerBulkRequestDTO Class Reference

Public Member Functions

- [PlayerBulkRequestDTO \(\)](#)
- [PlayerBulkRequestDTO \(List< PlayerRequestDTO > players\)](#)
- [List< PlayerRequestDTO > getPlayers \(\)](#)
- [void setPlayers \(List< PlayerRequestDTO > players\)](#)

8.38.1 Detailed Description

Definition at line 8 of file [PlayerBulkRequestDTO.java](#).

8.38.2 Constructor & Destructor Documentation

8.38.2.1 PlayerBulkRequestDTO() [1/2]

```
com.example.football_manager.dto.PlayerBulkRequestDTO.PlayerBulkRequestDTO ()
```

Definition at line 13 of file [PlayerBulkRequestDTO.java](#).

8.38.2.2 PlayerBulkRequestDTO() [2/2]

```
com.example.football_manager.dto.PlayerBulkRequestDTO.PlayerBulkRequestDTO (
    List< PlayerRequestDTO > players)
```

Definition at line 16 of file [PlayerBulkRequestDTO.java](#).

8.38.3 Member Function Documentation

8.38.3.1 getPlayers()

```
List< PlayerRequestDTO > com.example.football_manager.dto.PlayerBulkRequestDTO.getPlayers ()
```

Definition at line 20 of file [PlayerBulkRequestDTO.java](#).

8.38.3.2 setPlayers()

```
void com.example.football_manager.dto.PlayerBulkRequestDTO.setPlayers (
    List< PlayerRequestDTO > players)
```

Definition at line 24 of file [PlayerBulkRequestDTO.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/dto/PlayerBulkRequestDTO.java](#)

8.39 com.example.football_manager.controller.PlayerController Class Reference

Public Member Functions

- [PlayerController](#) ([PlayerService](#) playerService)
- [ResponseEntity<?>](#) [createPlayer](#) ([@PathVariable](#) Long teamId, [@Valid](#) [@RequestBody](#) [PlayerRequestDTO](#) dto)
- [ResponseEntity<?>](#) [createPlayers](#) ([@PathVariable](#) Long teamId, [@Valid](#) [@RequestBody](#) [PlayerBulkRequestDTO](#) dto)
- [ResponseEntity<?>](#) [getPlayersByTeamId](#) ([@PathVariable](#) Long teamId)
- [ResponseEntity<?>](#) [updatePlayer](#) ([@PathVariable](#) Long teamId, [@PathVariable](#) Long playerId, [@Valid](#) [@RequestBody](#) [PlayerRequestDTO](#) dto)
- [ResponseEntity<?>](#) [deletePlayer](#) ([@PathVariable](#) Long teamId, [@PathVariable](#) Long playerId)

8.39.1 Detailed Description

Definition at line 23 of file [PlayerController.java](#).

8.39.2 Constructor & Destructor Documentation

8.39.2.1 PlayerController()

```
com.example.football_manager.controller.PlayerController.PlayerController (
    PlayerService playerService)
```

Definition at line 27 of file [PlayerController.java](#).

8.39.3 Member Function Documentation

8.39.3.1 createPlayer()

```
ResponseEntity<?> com.example.football_manager.controller.PlayerController.createPlayer (
    @PathVariable Long teamId,
    @Valid @RequestBody PlayerRequestDTO dto)
```

Definition at line 40 of file [PlayerController.java](#).

8.39.3.2 createPlayers()

```
ResponseEntity<?> com.example.football_manager.controller.PlayerController.createPlayers (
    @PathVariable Long teamId,
    @Valid @RequestBody PlayerBulkRequestDTO dto)
```

Definition at line 60 of file [PlayerController.java](#).

8.39.3.3 deletePlayer()

```
ResponseEntity<?> com.example.football_manager.controller.PlayerController.deletePlayer (
    @PathVariable Long teamId,
    @PathVariable Long playerId)
```

Definition at line 119 of file [PlayerController.java](#).

8.39.3.4 getPlayersByTeamId()

```
ResponseEntity<?> com.example.football_manager.controller.PlayerController.getPlayersByTeamId (
    @PathVariable Long teamId)
```

Definition at line 80 of file [PlayerController.java](#).

8.39.3.5 updatePlayer()

```
ResponseEntity<?> com.example.football_manager.controller.PlayerController.updatePlayer (
    @PathVariable Long teamId,
    @PathVariable Long playerId,
    @Valid @RequestBody PlayerRequestDTO dto)
```

Definition at line 98 of file [PlayerController.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/PlayerController.java](#)

8.40 com.example.football_manager.model.PlayerPosition Enum Reference

Public Attributes

- [GOALKEEPER](#)
- [DEFENDER](#)
- [MIDFIELDER](#)
- [FORWARD](#)

8.40.1 Detailed Description

Definition at line 3 of file [PlayerPosition.java](#).

8.40.2 Member Data Documentation

8.40.2.1 DEFENDER

`com.example.football_manager.model.PlayerPosition.DEFENDER`

Definition at line 5 of file [PlayerPosition.java](#).

8.40.2.2 FORWARD

`com.example.football_manager.model.PlayerPosition.FORWARD`

Definition at line 7 of file [PlayerPosition.java](#).

8.40.2.3 GOALKEEPER

`com.example.football_manager.model.PlayerPosition.GOALKEEPER`

Definition at line 4 of file [PlayerPosition.java](#).

8.40.2.4 MIDFIELDER

`com.example.football_manager.model.PlayerPosition.MIDFIELDER`

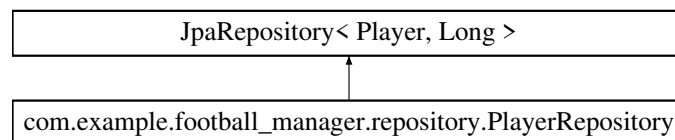
Definition at line 6 of file [PlayerPosition.java](#).

The documentation for this enum was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/model/PlayerPosition.java](#)

8.41 com.example.football_manager.repository.PlayerRepository Interface Reference

Inheritance diagram for `com.example.football_manager.repository.PlayerRepository`:



Public Member Functions

- boolean [existsByTeamIdAndNumber](#) (Long teamId, Integer number)
- boolean [existsByTeamIdAndNumberAndIdNot](#) (Long teamId, Integer number, Long id)
- List< [Player](#) > [findByTeamIdOrderByNumberAsc](#) (Long teamId)
- java.util.Optional< [Player](#) > [findByIdAndTeamId](#) (Long id, Long teamId)

8.41.1 Detailed Description

Definition at line 10 of file [PlayerRepository.java](#).

8.41.2 Member Function Documentation

8.41.2.1 existsByTeamIdAndNumber()

```
boolean com.example.football_manager.repository.PlayerRepository.existsByTeamIdAndNumber (
    Long teamId,
    Integer number)
```

8.41.2.2 existsByTeamIdAndNumberAndIdNot()

```
boolean com.example.football_manager.repository.PlayerRepository.existsByTeamIdAndNumberAndIdNot (
    Long teamId,
    Integer number,
    Long id)
```

8.41.2.3 findByIdAndTeamId()

```
java.util.Optional< Player > com.example.football_manager.repository.PlayerRepository.findByIdAndTeamId (
    Long id,
    Long teamId)
```

8.41.2.4 findByTeamIdOrderByNumberAsc()

```
List< Player > com.example.football_manager.repository.PlayerRepository.findByTeamIdOrderByNumberAsc (
    Long teamId)
```

The documentation for this interface was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/repository/PlayerRepository.java](#)

8.42 com.example.football_manager.dto.PlayerRequestDTO Class Reference

Public Member Functions

- [PlayerRequestDTO](#) ()
- [PlayerRequestDTO](#) (String name, Integer number, [PlayerPosition](#) position)
- String [getName](#) ()
- void [setName](#) (String name)
- Integer [getNumber](#) ()
- void [setNumber](#) (Integer number)
- [PlayerPosition](#) [getPosition](#) ()
- void [setPosition](#) ([PlayerPosition](#) position)

8.42.1 Detailed Description

Definition at line 10 of file [PlayerRequestDTO.java](#).

8.42.2 Constructor & Destructor Documentation

8.42.2.1 PlayerRequestDTO() [1/2]

```
com.example.football_manager.dto.PlayerRequestDTO.PlayerRequestDTO ()
```

Definition at line 26 of file [PlayerRequestDTO.java](#).

8.42.2.2 PlayerRequestDTO() [2/2]

```
com.example.football_manager.dto.PlayerRequestDTO.PlayerRequestDTO (
    String name,
    Integer number,
    PlayerPosition position)
```

Definition at line 29 of file [PlayerRequestDTO.java](#).

8.42.3 Member Function Documentation

8.42.3.1 getName()

`String com.example.football_manager.dto.PlayerRequestDTO.getName ()`

Definition at line 35 of file [PlayerRequestDTO.java](#).

8.42.3.2 getNumber()

`Integer com.example.football_manager.dto.PlayerRequestDTO.getNumber ()`

Definition at line 43 of file [PlayerRequestDTO.java](#).

8.42.3.3 getPosition()

`PlayerPosition com.example.football_manager.dto.PlayerRequestDTO.getPosition ()`

Definition at line 51 of file [PlayerRequestDTO.java](#).

8.42.3.4 setName()

`void com.example.football_manager.dto.PlayerRequestDTO.setName (
String name)`

Definition at line 39 of file [PlayerRequestDTO.java](#).

8.42.3.5 setNumber()

`void com.example.football_manager.dto.PlayerRequestDTO.setNumber (
Integer number)`

Definition at line 47 of file [PlayerRequestDTO.java](#).

8.42.3.6 setPosition()

`void com.example.football_manager.dto.PlayerRequestDTO.setPosition (
PlayerPosition position)`

Definition at line 55 of file [PlayerRequestDTO.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/dto/PlayerRequestDTO.java](#)

8.43 com.example.football_manager.service.PlayerService Class Reference

Public Member Functions

- [PlayerService](#) ([PlayerRepository](#) playerRepository, [TeamRepository](#) teamRepository)
- [Player](#) [createPlayer](#) (Long teamId, [PlayerRequestDTO](#) dto)
- List< [Player](#) > [createPlayers](#) (Long teamId, List< [PlayerRequestDTO](#) > playerRequests)
- List< [Player](#) > [getPlayersByTeamId](#) (Long teamId)
- [Player](#) [getPlayerByTeamId](#) (Long teamId, Long playerId)
- [Player](#) [updatePlayer](#) (Long teamId, Long playerId, [PlayerRequestDTO](#) dto)
- void [deletePlayer](#) (Long teamId, Long playerId)

8.43.1 Detailed Description

Definition at line 16 of file [PlayerService.java](#).

8.43.2 Constructor & Destructor Documentation

8.43.2.1 `PlayerService()`

```
com.example.football_manager.service.PlayerService.PlayerService (  
    PlayerRepository playerRepository,  
    TeamRepository teamRepository)
```

Definition at line 21 of file [PlayerService.java](#).

8.43.3 Member Function Documentation

8.43.3.1 `createPlayer()`

```
Player com.example.football_manager.service.PlayerService.createPlayer (  
    Long teamId,  
    PlayerRequestDTO dto)
```

Definition at line 26 of file [PlayerService.java](#).

8.43.3.2 `createPlayers()`

```
List< Player > com.example.football_manager.service.PlayerService.createPlayers (  
    Long teamId,  
    List< PlayerRequestDTO > playerRequests)
```

Definition at line 51 of file [PlayerService.java](#).

8.43.3.3 `deletePlayer()`

```
void com.example.football_manager.service.PlayerService.deletePlayer (  
    Long teamId,  
    Long playerId)
```

Definition at line 154 of file [PlayerService.java](#).

8.43.3.4 `getPlayerByTeamId()`

```
Player com.example.football_manager.service.PlayerService.getPlayerByTeamId (  
    Long teamId,  
    Long playerId)
```

Definition at line 106 of file [PlayerService.java](#).

8.43.3.5 `getPlayersByTeamId()`

```
List< Player > com.example.football_manager.service.PlayerService.getPlayersByTeamId (  
    Long teamId)
```

Definition at line 94 of file [PlayerService.java](#).

8.43.3.6 `updatePlayer()`

```
Player com.example.football_manager.service.PlayerService.updatePlayer (  
    Long teamId,  
    Long playerId,  
    PlayerRequestDTO dto)
```

Definition at line 125 of file [PlayerService.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/service/PlayerService.java](#)

8.44 com.example.football_manager.dto.ProfileUpdateDTO Class Reference

Public Member Functions

- boolean [isPasswordConfirmationValid](#) ()

8.44.1 Detailed Description

Definition at line 16 of file [ProfileUpdateDTO.java](#).

8.44.2 Member Function Documentation

8.44.2.1 isPasswordConfirmationValid()

```
boolean com.example.football_manager.dto.ProfileUpdateDTO.isPasswordConfirmationValid ()
```

Definition at line 39 of file [ProfileUpdateDTO.java](#).

References [isPasswordConfirmationValid\(\)](#).

Referenced by [isPasswordConfirmationValid\(\)](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/dto/ProfileUpdateDTO.java](#)

8.45 com.example.football_manager.controller.ProfileViewController Class Reference

Public Member Functions

- [ProfileViewController](#) ([UserService](#) userService)
- String [profilePage](#) (Model model, HttpSession session)
- String [updateProfile](#) (@Valid @ModelAttribute("profileForm") [ProfileUpdateDTO](#) profileForm, BindingResult bindingResult, Model model, HttpSession session)

8.45.1 Detailed Description

Definition at line 18 of file [ProfileViewController.java](#).

8.45.2 Constructor & Destructor Documentation

8.45.2.1 ProfileViewController()

```
com.example.football_manager.controller.ProfileViewController.ProfileViewController (  
    UserService userService)
```

Definition at line 22 of file [ProfileViewController.java](#).

8.45.3 Member Function Documentation

8.45.3.1 profilePage()

```
String com.example.football_manager.controller.ProfileViewController.profilePage (  
    Model model,  
    HttpSession session)
```

Definition at line 27 of file [ProfileViewController.java](#).

References [profilePage\(\)](#).

Referenced by [profilePage\(\)](#).

8.45.3.2 updateProfile()

```
String com.example.football_manager.controller.ProfileViewController.updateProfile (  
    @Valid @ModelAttribute("profileForm") ProfileUpdatedDTO profileForm,  
    BindingResult bindingResult,  
    Model model,  
    HttpSession session)
```

Definition at line 46 of file [ProfileViewController.java](#).

References [updateProfile\(\)](#).

Referenced by [updateProfile\(\)](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/ProfileViewController.java](#)

8.46 com.example.football_manager.dto.RegisterDTO Class Reference

Public Member Functions

- String [getUsername](#) ()
- void [setUsername](#) (String username)
- String [getEmail](#) ()
- void [setEmail](#) (String email)
- String [getPassword](#) ()
- void [setPassword](#) (String password)
- Boolean [getIsAdmin](#) ()
- void [setIsAdmin](#) (Boolean isAdmin)

8.46.1 Detailed Description

Definition at line 8 of file [RegisterDTO.java](#).

8.46.2 Member Function Documentation

8.46.2.1 getEmail()

```
String com.example.football_manager.dto.RegisterDTO.getEmail ()
```

Definition at line 31 of file [RegisterDTO.java](#).

8.46.2.2 getIsAdmin()

```
Boolean com.example.football_manager.dto.RegisterDTO.getIsAdmin ()
```

Definition at line 37 of file [RegisterDTO.java](#).

8.46.2.3 getPassword()

```
String com.example.football_manager.dto.RegisterDTO.getPassword ()
```

Definition at line 34 of file [RegisterDTO.java](#).

8.46.2.4 getUsername()

```
String com.example.football_manager.dto.RegisterDTO.getUsername ()
```

Definition at line 28 of file [RegisterDTO.java](#).

8.46.2.5 setEmail()

```
void com.example.football_manager.dto.RegisterDTO.setEmail (  
    String email)
```

Definition at line 32 of file [RegisterDTO.java](#).

8.46.2.6 setIsAdmin()

```
void com.example.football_manager.dto.RegisterDTO.setIsAdmin (
    Boolean isAdmin)
```

Definition at line 38 of file [RegisterDTO.java](#).

8.46.2.7 setPassword()

```
void com.example.football_manager.dto.RegisterDTO.setPassword (
    String password)
```

Definition at line 35 of file [RegisterDTO.java](#).

8.46.2.8 setUsername()

```
void com.example.football_manager.dto.RegisterDTO.setUsername (
    String username)
```

Definition at line 29 of file [RegisterDTO.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/dto/RegisterDTO.java](#)

8.47 com.example.football_manager.config.SecurityConfig Class Reference

Public Member Functions

- SecurityFilterChain [filterChain](#) (HttpSecurity http) throws Exception
- PasswordEncoder [passwordEncoder](#) ()

8.47.1 Detailed Description

Definition at line 11 of file [SecurityConfig.java](#).

8.47.2 Member Function Documentation**8.47.2.1 filterChain()**

```
SecurityFilterChain com.example.football_manager.config.SecurityConfig.filterChain (
    HttpSecurity http) throws Exception
```

Definition at line 14 of file [SecurityConfig.java](#).

8.47.2.2 passwordEncoder()

```
PasswordEncoder com.example.football_manager.config.SecurityConfig.passwordEncoder ()
```

Definition at line 25 of file [SecurityConfig.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/config/SecurityConfig.java](#)

8.48 com.example.football_manager.dto.StandingDTO Class Reference

View-only DTO used to render a single row of the standings table.

Public Member Functions

- [StandingDTO](#) ()
- [StandingDTO](#) (String teamName, String logoUrl, int matchesPlayed, int won, int drawn, int lost, int goalsFor, int goalsAgainst, int goalDifference, int points)
- String [getTeamName](#) ()

- void [setTeamName](#) (String teamName)
- String [getLogoUrl](#) ()
- void [setLogoUrl](#) (String logoUrl)
- int [getMatchesPlayed](#) ()
- void [setMatchesPlayed](#) (int matchesPlayed)
- int [getWon](#) ()
- void [setWon](#) (int won)
- int [getDrawn](#) ()
- void [setDrawn](#) (int drawn)
- int [getLost](#) ()
- void [setLost](#) (int lost)
- int [getGoalsFor](#) ()
- void [setGoalsFor](#) (int goalsFor)
- int [getGoalsAgainst](#) ()
- void [setGoalsAgainst](#) (int goalsAgainst)
- int [getGoalDifference](#) ()
- void [setGoalDifference](#) (int goalDifference)
- int [getPoints](#) ()
- void [setPoints](#) (int points)

8.48.1 Detailed Description

View-only DTO used to render a single row of the standings table.

It is currently populated with mock data in `StandingsViewController` and will later be produced by the real standings service.

Definition at line 8 of file [StandingDTO.java](#).

8.48.2 Constructor & Destructor Documentation

8.48.2.1 `StandingDTO()` [1/2]

```
com.example.football_manager.dto.StandingDTO.StandingDTO ()
```

Definition at line 21 of file [StandingDTO.java](#).

8.48.2.2 `StandingDTO()` [2/2]

```
com.example.football_manager.dto.StandingDTO.StandingDTO (
    String teamName,
    String logoUrl,
    int matchesPlayed,
    int won,
    int drawn,
    int lost,
    int goalsFor,
    int goalsAgainst,
    int goalDifference,
    int points)
```

Definition at line 24 of file [StandingDTO.java](#).

8.48.3 Member Function Documentation

8.48.3.1 `getDrawn()`

```
int com.example.football_manager.dto.StandingDTO.getDrawn ()
```

Definition at line 70 of file [StandingDTO.java](#).

8.48.3.2 getGoalDifference()

```
int com.example.football_manager.dto.StandingDTO.getGoalDifference ()
```

Definition at line 102 of file [StandingDTO.java](#).

Referenced by [com.example.football_manager.service.StandingsService.getStandingsForCompetition\(\)](#).

8.48.3.3 getGoalsAgainst()

```
int com.example.football_manager.dto.StandingDTO.getGoalsAgainst ()
```

Definition at line 94 of file [StandingDTO.java](#).

8.48.3.4 getGoalsFor()

```
int com.example.football_manager.dto.StandingDTO.getGoalsFor ()
```

Definition at line 86 of file [StandingDTO.java](#).

Referenced by [com.example.football_manager.service.StandingsService.getStandingsForCompetition\(\)](#).

8.48.3.5 getLogoUrl()

```
String com.example.football_manager.dto.StandingDTO.getLogoUrl ()
```

Definition at line 46 of file [StandingDTO.java](#).

8.48.3.6 getLost()

```
int com.example.football_manager.dto.StandingDTO.getLost ()
```

Definition at line 78 of file [StandingDTO.java](#).

8.48.3.7 getMatchesPlayed()

```
int com.example.football_manager.dto.StandingDTO.getMatchesPlayed ()
```

Definition at line 54 of file [StandingDTO.java](#).

8.48.3.8 getPoints()

```
int com.example.football_manager.dto.StandingDTO.getPoints ()
```

Definition at line 110 of file [StandingDTO.java](#).

Referenced by [com.example.football_manager.service.StandingsService.getStandingsForCompetition\(\)](#).

8.48.3.9 getTeamName()

```
String com.example.football_manager.dto.StandingDTO.getTeamName ()
```

Definition at line 38 of file [StandingDTO.java](#).

Referenced by [com.example.football_manager.service.StandingsService.getStandingsForCompetition\(\)](#).

8.48.3.10 getWon()

```
int com.example.football_manager.dto.StandingDTO.getWon ()
```

Definition at line 62 of file [StandingDTO.java](#).

8.48.3.11 setDrawn()

```
void com.example.football_manager.dto.StandingDTO.setDrawn (  
    int drawn)
```

Definition at line 74 of file [StandingDTO.java](#).

8.48.3.12 setGoalDifference()

```
void com.example.football_manager.dto.StandingDTO.setGoalDifference (  
    int goalDifference)
```

Definition at line 106 of file [StandingDTO.java](#).

8.48.3.13 setGoalsAgainst()

```
void com.example.football_manager.dto.StandingDTO.setGoalsAgainst (
    int goalsAgainst)
```

Definition at line 98 of file [StandingDTO.java](#).

8.48.3.14 setGoalsFor()

```
void com.example.football_manager.dto.StandingDTO.setGoalsFor (
    int goalsFor)
```

Definition at line 90 of file [StandingDTO.java](#).

8.48.3.15 setLogoUrl()

```
void com.example.football_manager.dto.StandingDTO.setLogoUrl (
    String logoUrl)
```

Definition at line 50 of file [StandingDTO.java](#).

8.48.3.16 setLost()

```
void com.example.football_manager.dto.StandingDTO.setLost (
    int lost)
```

Definition at line 82 of file [StandingDTO.java](#).

8.48.3.17 setMatchesPlayed()

```
void com.example.football_manager.dto.StandingDTO.setMatchesPlayed (
    int matchesPlayed)
```

Definition at line 58 of file [StandingDTO.java](#).

8.48.3.18 setPoints()

```
void com.example.football_manager.dto.StandingDTO.setPoints (
    int points)
```

Definition at line 114 of file [StandingDTO.java](#).

8.48.3.19 setTeamName()

```
void com.example.football_manager.dto.StandingDTO.setTeamName (
    String teamName)
```

Definition at line 42 of file [StandingDTO.java](#).

8.48.3.20 setWon()

```
void com.example.football_manager.dto.StandingDTO.setWon (
    int won)
```

Definition at line 66 of file [StandingDTO.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/dto/StandingDTO.java](#)

8.49 com.example.football_manager.controller.StandingsController Class Reference

Public Member Functions

- [StandingsController](#) ([StandingsService](#) standingsService, [CompetitionRepository](#) competitionRepository)
- `ResponseEntity< List< StandingDTO > >` [getStandings](#) (@RequestParam Long competitionId)

8.49.1 Detailed Description

Definition at line 23 of file [StandingsController.java](#).

8.49.2 Constructor & Destructor Documentation

8.49.2.1 StandingsController()

```
com.example.football_manager.controller.StandingsController.StandingsController (
    StandingsService standingsService,
    CompetitionRepository competitionRepository)
```

Definition at line 28 of file [StandingsController.java](#).

8.49.3 Member Function Documentation

8.49.3.1 getStandings()

```
ResponseEntity< List< StandingDTO > > com.example.football_manager.controller.StandingsController.getStandings (
    @RequestParam Long competitionId)
```

Definition at line 43 of file [StandingsController.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/StandingsController.java](#)

8.50 com.example.football_manager.service.StandingsService Class Reference

Public Member Functions

- [StandingsService](#) ([CompetitionRepository](#) competitionRepository, [MatchRepository](#) matchRepository)
- List< [Competition](#) > [getCompetitions](#) ()
- List< [StandingDTO](#) > [getStandingsForCompetition](#) (Long competitionId)

8.50.1 Detailed Description

Definition at line 19 of file [StandingsService.java](#).

8.50.2 Constructor & Destructor Documentation

8.50.2.1 StandingsService()

```
com.example.football_manager.service.StandingsService.StandingsService (
    CompetitionRepository competitionRepository,
    MatchRepository matchRepository)
```

Definition at line 24 of file [StandingsService.java](#).

8.50.3 Member Function Documentation

8.50.3.1 getCompetitions()

```
List< Competition > com.example.football_manager.service.StandingsService.getCompetitions ()
```

Definition at line 31 of file [StandingsService.java](#).

References [getCompetitions\(\)](#).

Referenced by [getCompetitions\(\)](#).

8.50.3.2 getStandingsForCompetition()

```
List< StandingDTO > com.example.football_manager.service.StandingsService.getStandingsForCompetition (
    Long competitionId)
```

Definition at line 36 of file [StandingsService.java](#).

References [com.example.football_manager.dto.StandingDTO.getGoalDifference\(\)](#), [com.example.football_manager.dto.StandingDTO.getPoints\(\)](#), [getStandingsForCompetition\(\)](#), and [com.example.football_manager.dto](#).

Referenced by [getStandingsForCompetition\(\)](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/service/StandingsService.java](#)

8.51 com.example.football_manager.controller.StandingsViewController Class Reference

Public Member Functions

- [StandingsViewController](#) ([StandingsService](#) standingsService)
- String [standingsPage](#) (@RequestParam(required=false) Long competitionId, Model model)

8.51.1 Detailed Description

Definition at line 14 of file [StandingsViewController.java](#).

8.51.2 Constructor & Destructor Documentation

8.51.2.1 StandingsViewController()

```
com.example.football_manager.controller.StandingsViewController.StandingsViewController (
    StandingsService standingsService)
```

Definition at line 18 of file [StandingsViewController.java](#).

8.51.3 Member Function Documentation

8.51.3.1 standingsPage()

```
String com.example.football_manager.controller.StandingsViewController.standingsPage (
    @RequestParam(required=false) Long competitionId,
    Model model)
```

Definition at line 23 of file [StandingsViewController.java](#).

References [standingsPage\(\)](#).

Referenced by [standingsPage\(\)](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/StandingsViewController.java](#)

8.52 com.example.football_manager.model.Team Class Reference

Public Member Functions

- [Team](#) (Long id, String name, String logoUrl, [Country](#) country)

8.52.1 Detailed Description

Definition at line 19 of file [Team.java](#).

8.52.2 Constructor & Destructor Documentation

8.52.2.1 Team()

```
com.example.football_manager.model.Team.Team (
    Long id,
    String name,
    String logoUrl,
    Country country)
```

Definition at line 42 of file [Team.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/model/Team.java](#)

8.53 com.example.football_manager.controller.TeamController Class Reference

Public Member Functions

- [TeamController](#) ([TeamService](#) teamService)
- [ResponseEntity](#)< [Team](#) > [createTeam](#) (@Valid @RequestBody [TeamRequestDTO](#) dto)
- [ResponseEntity](#)< [Team](#) > [updateTeam](#) (@PathVariable Long id, @Valid @RequestBody [TeamRequestDTO](#) dto)
- [ResponseEntity](#)< Void > [deleteTeam](#) (@PathVariable Long id)
- [ResponseEntity](#)< [Team](#) > [getTeamById](#) (@PathVariable Long id)
- [ResponseEntity](#)< List< [Team](#) > > [getTeamsContainingName](#) (@RequestParam String name)
- [ResponseEntity](#)< List< [Team](#) > > [getAllTeams](#) ()

8.53.1 Detailed Description

Definition at line 23 of file [TeamController.java](#).

8.53.2 Constructor & Destructor Documentation

8.53.2.1 TeamController()

```
com.example.football_manager.controller.TeamController.TeamController (
    TeamService teamService)
```

Definition at line 27 of file [TeamController.java](#).

8.53.3 Member Function Documentation

8.53.3.1 createTeam()

```
ResponseEntity< Team > com.example.football_manager.controller.TeamController.createTeam (
    @Valid @RequestBody TeamRequestDTO dto)
```

Definition at line 45 of file [TeamController.java](#).

8.53.3.2 deleteTeam()

```
ResponseEntity< Void > com.example.football_manager.controller.TeamController.deleteTeam (
    @PathVariable Long id)
```

Definition at line 80 of file [TeamController.java](#).

8.53.3.3 getAllTeams()

```
ResponseEntity< List< Team > > com.example.football_manager.controller.TeamController.get←
AllTeams ()
```

Definition at line 118 of file [TeamController.java](#).

8.53.3.4 getTeamById()

```
ResponseEntity< Team > com.example.football_manager.controller.TeamController.getTeamById (
    @PathVariable Long id)
```

Definition at line 94 of file [TeamController.java](#).

8.53.3.5 getTeamsContainingName()

```
ResponseEntity< List< Team > > com.example.football_manager.controller.TeamController.getTeamsContainingName (
    @RequestParam String name)
```

Definition at line 106 of file [TeamController.java](#).

8.53.3.6 updateTeam()

```
ResponseEntity< Team > com.example.football_manager.controller.TeamController.updateTeam (
    @PathVariable Long id,
    @Valid @RequestBody TeamRequestDTO dto)
```

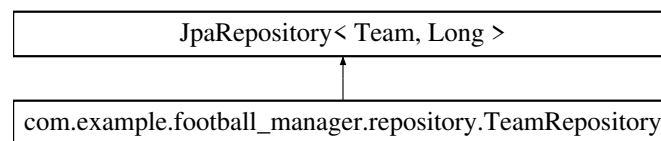
Definition at line 65 of file [TeamController.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/TeamController.java](#)

8.54 com.example.football_manager.repository.TeamRepository Interface Reference

Inheritance diagram for com.example.football_manager.repository.TeamRepository:



Public Member Functions

- `List< Team > findByNameContainingIgnoreCase (String name)`

8.54.1 Detailed Description

Definition at line 10 of file [TeamRepository.java](#).

8.54.2 Member Function Documentation

8.54.2.1 findByNameContainingIgnoreCase()

```
List< Team > com.example.football_manager.repository.TeamRepository.findByNameContainingIgnoreCase (
    String name)
```

The documentation for this interface was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/repository/TeamRepository.java](#)

8.55 com.example.football_manager.dto.TeamRequestDTO Class Reference

Public Member Functions

- [TeamRequestDTO](#) ()
- [TeamRequestDTO](#) (String name, String logoUrl, Long countryId)
- String [getName](#) ()
- void [setName](#) (String name)
- String [getLogoUrl](#) ()
- void [setLogoUrl](#) (String logoUrl)
- Long [getCountryId](#) ()
- void [setCountryId](#) (Long countryId)

8.55.1 Detailed Description

Definition at line 8 of file [TeamRequestDTO.java](#).

8.55.2 Constructor & Destructor Documentation

8.55.2.1 TeamRequestDTO() [1/2]

`com.example.football_manager.dto.TeamRequestDTO.TeamRequestDTO ()`

Definition at line 23 of file [TeamRequestDTO.java](#).

8.55.2.2 TeamRequestDTO() [2/2]

```
com.example.football_manager.dto.TeamRequestDTO.TeamRequestDTO (  
    String name,  
    String logoUrl,  
    Long countryId)
```

Definition at line 26 of file [TeamRequestDTO.java](#).

8.55.3 Member Function Documentation

8.55.3.1 getCountryId()

`Long com.example.football_manager.dto.TeamRequestDTO.getCountryId ()`

Definition at line 48 of file [TeamRequestDTO.java](#).

8.55.3.2 getLogoUrl()

`String com.example.football_manager.dto.TeamRequestDTO.getLogoUrl ()`

Definition at line 40 of file [TeamRequestDTO.java](#).

8.55.3.3 getName()

`String com.example.football_manager.dto.TeamRequestDTO.getName ()`

Definition at line 32 of file [TeamRequestDTO.java](#).

8.55.3.4 setCountryId()

```
void com.example.football_manager.dto.TeamRequestDTO.setCountryId (  
    Long countryId)
```

Definition at line 52 of file [TeamRequestDTO.java](#).

Referenced by [com.example.football_manager.controller.TeamViewController.showEditTeamForm\(\)](#).

8.55.3.5 setLogoUrl()

```
void com.example.football_manager.dto.TeamRequestDTO.setLogoUrl (
    String logoUrl)
```

Definition at line 44 of file [TeamRequestDTO.java](#).

Referenced by [com.example.football_manager.controller.TeamViewController.showEditTeamForm\(\)](#).

8.55.3.6 setName()

```
void com.example.football_manager.dto.TeamRequestDTO.setName (
    String name)
```

Definition at line 36 of file [TeamRequestDTO.java](#).

Referenced by [com.example.football_manager.controller.TeamViewController.showEditTeamForm\(\)](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/dto/TeamRequestDTO.java](#)

8.56 com.example.football_manager.service.TeamService Class Reference

Public Member Functions

- [TeamService](#) ([TeamRepository](#) teamRepository, [CountryRepository](#) countryRepository)
- [Team createTeam](#) ([TeamRequestDTO](#) dto)
- [Team updateTeam](#) (Long id, [TeamRequestDTO](#) dto)
- void [deleteTeam](#) (Long id)
- Optional< [Team](#) > [getTeamById](#) (Long id)
- List< [Team](#) > [getTeamsContainingName](#) (String name)
- List< [Team](#) > [getAllTeams](#) ()

8.56.1 Detailed Description

Definition at line 17 of file [TeamService.java](#).

8.56.2 Constructor & Destructor Documentation

8.56.2.1 TeamService()

```
com.example.football_manager.service.TeamService.TeamService (
    TeamRepository teamRepository,
    CountryRepository countryRepository)
```

Definition at line 22 of file [TeamService.java](#).

8.56.3 Member Function Documentation

8.56.3.1 createTeam()

```
Team com.example.football_manager.service.TeamService.createTeam (
    TeamRequestDTO dto)
```

Definition at line 28 of file [TeamService.java](#).

References [createTeam\(\)](#).

Referenced by [createTeam\(\)](#).

8.56.3.2 deleteTeam()

```
void com.example.football_manager.service.TeamService.deleteTeam (
    Long id)
```

Definition at line 56 of file [TeamService.java](#).

References [deleteTeam\(\)](#).

Referenced by [deleteTeam\(\)](#).

8.56.3.3 getAllTeams()

List< [Team](#) > com.example.football_manager.service.TeamService.getAllTeams ()

Definition at line 75 of file [TeamService.java](#).

References [getAllTeams\(\)](#).

Referenced by [getAllTeams\(\)](#).

8.56.3.4 getTeamById()

Optional< [Team](#) > com.example.football_manager.service.TeamService.getTeamById (
Long id)

Definition at line 63 of file [TeamService.java](#).

8.56.3.5 getTeamsContainingName()

List< [Team](#) > com.example.football_manager.service.TeamService.getTeamsContainingName (
String name)

Definition at line 67 of file [TeamService.java](#).

8.56.3.6 updateTeam()

[Team](#) com.example.football_manager.service.TeamService.updateTeam (
Long id,
[TeamRequestDTO](#) dto)

Definition at line 41 of file [TeamService.java](#).

References [updateTeam\(\)](#).

Referenced by [updateTeam\(\)](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/service/TeamService.java](#)

8.57 com.example.football_manager.controller.TeamViewController Class Reference

Public Member Functions

- String [teamsPage](#) (Model model, HttpSession session)
- String [teamDetailsPage](#) (@PathVariable Long id, Model model, HttpSession session)
- String [showAddTeamForm](#) (Model model)
- String [createTeam](#) (@ModelAttribute [TeamRequestDTO](#) teamDTO)
- String [showEditTeamForm](#) (@PathVariable Long id, Model model)
- String [updateTeam](#) (@PathVariable Long id, @ModelAttribute [TeamRequestDTO](#) teamDTO)
- String [deleteTeam](#) (@PathVariable Long id)

8.57.1 Detailed Description

Definition at line 22 of file [TeamViewController.java](#).

8.57.2 Member Function Documentation

8.57.2.1 createTeam()

String com.example.football_manager.controller.TeamViewController.createTeam (
@ModelAttribute [TeamRequestDTO](#) teamDTO)

Definition at line 84 of file [TeamViewController.java](#).

References [createTeam\(\)](#).

Referenced by [createTeam\(\)](#).

8.57.2.2 deleteTeam()

```
String com.example.football_manager.controller.TeamViewController.deleteTeam (  
    @PathVariable Long id)
```

Definition at line 118 of file [TeamViewController.java](#).

References [deleteTeam\(\)](#).

Referenced by [deleteTeam\(\)](#).

8.57.2.3 showAddTeamForm()

```
String com.example.football_manager.controller.TeamViewController.showAddTeamForm (  
    Model model)
```

Definition at line 76 of file [TeamViewController.java](#).

References [showAddTeamForm\(\)](#).

Referenced by [showAddTeamForm\(\)](#).

8.57.2.4 showEditTeamForm()

```
String com.example.football_manager.controller.TeamViewController.showEditTeamForm (  
    @PathVariable Long id,  
    Model model)
```

Definition at line 91 of file [TeamViewController.java](#).

References [com.example.football_manager.dto.TeamRequestDTO.setCountryId\(\)](#), [com.example.football_manager.dto.TeamRequestDTO.setName\(\)](#), and [showEditTeamForm\(\)](#).

Referenced by [showEditTeamForm\(\)](#).

8.57.2.5 teamDetailsPage()

```
String com.example.football_manager.controller.TeamViewController.teamDetailsPage (  
    @PathVariable Long id,  
    Model model,  
    HttpSession session)
```

Definition at line 60 of file [TeamViewController.java](#).

References [teamDetailsPage\(\)](#).

Referenced by [teamDetailsPage\(\)](#).

8.57.2.6 teamsPage()

```
String com.example.football_manager.controller.TeamViewController.teamsPage (  
    Model model,  
    HttpSession session)
```

Definition at line 40 of file [TeamViewController.java](#).

References [teamsPage\(\)](#).

Referenced by [teamsPage\(\)](#).

8.57.2.7 updateTeam()

```
String com.example.football_manager.controller.TeamViewController.updateTeam (  
    @PathVariable Long id,  
    @ModelAttribute TeamRequestDTO teamDTO)
```

Definition at line 111 of file [TeamViewController.java](#).

References [updateTeam\(\)](#).

Referenced by [updateTeam\(\)](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/controller/TeamViewController.java](#)

8.58 com.example.football_manager.model.User Class Reference

8.58.1 Detailed Description

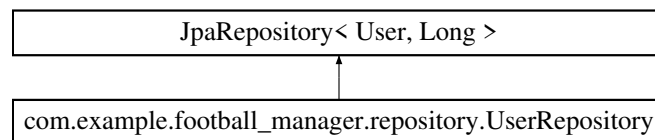
Definition at line 15 of file [User.java](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/model/User.java](#)

8.59 com.example.football_manager.repository.UserRepository Interface Reference

Inheritance diagram for com.example.football_manager.repository.UserRepository:



Public Member Functions

- Optional< [User](#) > [findByUsername](#) (String username)
- Optional< [User](#) > [findByEmail](#) (String email)
- Optional< [User](#) > [findWithFavouriteTeamsByld](#) (Long id)
- boolean [existsByUsername](#) (String username)
- boolean [existsByEmail](#) (String email)
- boolean [existsByUsernameAndIdNot](#) (String username, Long id)
- boolean [existsByEmailAndIdNot](#) (String email, Long id)

8.59.1 Detailed Description

Definition at line 11 of file [UserRepository.java](#).

8.59.2 Member Function Documentation

8.59.2.1 existsByEmail()

```
boolean com.example.football_manager.repository.UserRepository.existsByEmail (
    String email)
```

References [existsByEmail\(\)](#).

Referenced by [existsByEmail\(\)](#).

8.59.2.2 existsByEmailAndIdNot()

```
boolean com.example.football_manager.repository.UserRepository.existsByEmailAndIdNot (
    String email,
    Long id)
```

References [existsByEmailAndIdNot\(\)](#).

Referenced by [existsByEmailAndIdNot\(\)](#).

8.59.2.3 existsByUsername()

```
boolean com.example.football_manager.repository.UserRepository.existsByUsername (
    String username)
```

References [existsByUsername\(\)](#).

Referenced by [existsByUsername\(\)](#).

8.59.2.4 existsByUsernameAndIdNot()

```
boolean com.example.football_manager.repository.UserRepository.existsByUsernameAndIdNot (
    String username,
    Long id)
```

References [existsByUsernameAndIdNot\(\)](#).

Referenced by [existsByUsernameAndIdNot\(\)](#).

8.59.2.5 findByEmail()

```
Optional< User > com.example.football_manager.repository.UserRepository.findByEmail (
    String email)
```

8.59.2.6 findByUsername()

```
Optional< User > com.example.football_manager.repository.UserRepository.findByUsername (
    String username)
```

8.59.2.7 findWithFavouriteTeamsById()

```
Optional< User > com.example.football_manager.repository.UserRepository.findWithFavourite↵
TeamsById (
    Long id)
```

References [findWithFavouriteTeamsById\(\)](#).

Referenced by [findWithFavouriteTeamsById\(\)](#).

The documentation for this interface was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/repository/UserRepository.java](#)

8.60 com.example.football_manager.service.UserService Class Reference

Public Member Functions

- [User registerUser](#) (String username, String email, String rawPassword, Boolean isAdmin)
- [User login](#) (String username, String rawPassword)
- [User getUserById](#) (Long id)
- [User getUserByUsername](#) (String username)
- [User addFavouriteTeam](#) (Long userId, Long teamId)
- [User removeFavouriteTeam](#) (Long userId, Long teamId)
- [Set< Long > getFavouriteTeamIdsByUserId](#) (Long userId)
- [User getUserProfile](#) (Long userId)
- [User updateProfile](#) (Long userId, String username, String email, String currentPassword, String new↵ Password, String confirmNewPassword)

8.60.1 Detailed Description

Definition at line 18 of file [UserService.java](#).

8.60.2 Member Function Documentation

8.60.2.1 addFavouriteTeam()

```
User com.example.football_manager.service.UserService.addFavouriteTeam (
    Long userId,
    Long teamId)
```

Definition at line 68 of file [UserService.java](#).

References [getUserById\(\)](#).

8.60.2.2 getFavouriteTeamIdsByUserId()

```
Set< Long > com.example.football_manager.service.UserService.getFavouriteTeamIdsByUserId (
    Long userId)
```

Definition at line 106 of file [UserService.java](#).

References [getUserById\(\)](#).

8.60.2.3 getUserById()

```
User com.example.football_manager.service.UserService.getUserById (
    Long id)
```

Definition at line 58 of file [UserService.java](#).

Referenced by [addFavouriteTeam\(\)](#), [getFavouriteTeamIdsByUserId\(\)](#), [removeFavouriteTeam\(\)](#), and [updateProfile\(\)](#).

8.60.2.4 getUserByUsername()

```
User com.example.football_manager.service.UserService.getUserByUsername (
    String username)
```

Definition at line 63 of file [UserService.java](#).

8.60.2.5 getUserProfile()

```
User com.example.football_manager.service.UserService.getUserProfile (
    Long userId)
```

Definition at line 122 of file [UserService.java](#).

8.60.2.6 login()

```
User com.example.football_manager.service.UserService.login (
    String username,
    String rawPassword)
```

Definition at line 47 of file [UserService.java](#).

8.60.2.7 registerUser()

```
User com.example.football_manager.service.UserService.registerUser (
    String username,
    String email,
    String rawPassword,
    Boolean isAdmin)
```

Definition at line 29 of file [UserService.java](#).

8.60.2.8 removeFavouriteTeam()

```
User com.example.football_manager.service.UserService.removeFavouriteTeam (
    Long userId,
    Long teamId)
```

Definition at line 86 of file [UserService.java](#).

References [getUserById\(\)](#).

8.60.2.9 updateProfile()

```
User com.example.football_manager.service.UserService.updateProfile (
    Long userId,
    String username,
    String email,
    String currentPassword,
    String newPassword,
    String confirmNewPassword)
```

Definition at line 127 of file [UserService.java](#).

References [getUserById\(\)](#).

The documentation for this class was generated from the following file:

- [Football_Manager/src/main/java/com/example/football_manager/service/UserService.java](#)

Chapter 9

File Documentation

9.1 docs/doxygen_mainpage.md File Reference

9.2 Football_Manager/src/main/java/com/example/football_manager/config/AdminSecurityFilter.java File Reference ↩

```
import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import org.springframework.stereotype.Component;
import java.io.IOException;
```

Classes

- class [com.example.football_manager.config.AdminSecurityFilter](#)

Packages

- package [com.example.football_manager.config](#)

9.3 AdminSecurityFilter.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.config;
00002
00003 import jakarta.servlet.Filter;
00004 import jakarta.servlet.FilterChain;
00005 import jakarta.servlet.ServletException;
00006 import jakarta.servlet.ServletRequest;
00007 import jakarta.servlet.ServletResponse;
00008 import jakarta.servlet.http.HttpServletRequest;
00009 import jakarta.servlet.http.HttpServletResponse;
00010 import jakarta.servlet.http.HttpSession;
00011 import org.springframework.stereotype.Component;
00012
00013 import java.io.IOException;
00014
00015 @Component
00016 public class AdminSecurityFilter implements Filter {
00017
00018     @Override
00019     public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
00020         throws IOException, ServletException {
```

```

00021
00022     HttpServletRequest req = (HttpServletRequest) request;
00023     HttpServletResponse res = (HttpServletResponse) response;
00024
00025     String uri = req.getRequestURI();
00026
00027     boolean isProtectedPath = uri.startsWith("/admin") ||
00028         uri.startsWith("/teams/add") ||
00029         uri.startsWith("/teams/edit") ||
00030         uri.startsWith("/teams/delete") ||
00031         uri.startsWith("/matches/schedule") ||
00032         uri.startsWith("/matches/edit") ||
00033         uri.startsWith("/matches/results") ||
00034         uri.matches("^/matches/\\d+/(score|results).*$");
00035
00036     if (isProtectedPath) {
00037         HttpSession session = req.getSession(false);
00038
00039         Boolean isAdmin = (session != null) ? (Boolean) session.getAttribute("isAdmin") : null;
00040
00041         if (isAdmin == null || !isAdmin) {
00042             res.sendRedirect("/teams");
00043             return;
00044         }
00045     }
00046
00047     chain.doFilter(request, response);
00048 }
00049 }

```

9.4 Football_Manager/src/main/java/com/example/football_↵ manager/config/CacheConfig.java File Reference

```

import com.github.benmanes.caffeine.cache.Caffeine;
import org.springframework.cache.CacheManager;
import org.springframework.cache.annotation.EnableCaching;
import org.springframework.cache.caffeine.CaffeineCache;
import org.springframework.cache.support.SimpleCacheManager;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import java.time.Duration;
import java.util.List;

```

Classes

- class [com.example.football_manager.config.CacheConfig](#)

Packages

- package [com.example.football_manager.config](#)

9.5 CacheConfig.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.config;
00002
00003 import com.github.benmanes.caffeine.cache.Caffeine;
00004 import org.springframework.cache.CacheManager;
00005 import org.springframework.cache.annotation.EnableCaching;
00006 import org.springframework.cache.caffeine.CaffeineCache;
00007 import org.springframework.cache.support.SimpleCacheManager;
00008 import org.springframework.context.annotation.Bean;
00009 import org.springframework.context.annotation.Configuration;
00010
00011 import java.time.Duration;
00012 import java.util.List;
00013
00014 @Configuration
00015 @EnableCaching
00016 public class CacheConfig {

```

```

00017
00018     @Bean
00019     public CacheManager cacheManager() {
00020         SimpleCacheManager cacheManager = new SimpleCacheManager();
00021         cacheManager.setCaches(List.of(
00022             new CaffeineCache("matches", Caffeine.newBuilder()
00023                 .expireAfterWrite(Duration.ofSeconds(30))
00024                 .maximumSize(1000)
00025                 .build()),
00026             new CaffeineCache("standings", Caffeine.newBuilder()
00027                 .expireAfterWrite(Duration.ofSeconds(30))
00028                 .maximumSize(1000)
00029                 .build()),
00030             new CaffeineCache("upcomingMatches", Caffeine.newBuilder()
00031                 .expireAfterWrite(Duration.ofMinutes(1))
00032                 .maximumSize(1000)
00033                 .build()),
00034             new CaffeineCache("competitions", Caffeine.newBuilder()
00035                 .expireAfterWrite(Duration.ofMinutes(2))
00036                 .maximumSize(1000)
00037                 .build()),
00038             new CaffeineCache("teams", Caffeine.newBuilder()
00039                 .expireAfterWrite(Duration.ofMinutes(2))
00040                 .maximumSize(1000)
00041                 .build()),
00042             new CaffeineCache("matchResults", Caffeine.newBuilder()
00043                 .expireAfterWrite(Duration.ofSeconds(30))
00044                 .maximumSize(1000)
00045                 .build())
00046         ));
00047         return cacheManager;
00048     }
00049 }

```

9.6 Football_Manager/src/main/java/com/example/football_manager/config/LocaleConfig.java File Reference

```

import java.util.Locale;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.LocaleResolver;
import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.web.servlet.i18n.CookieLocaleResolver;
import org.springframework.web.servlet.i18n.LocaleChangeInterceptor;

```

Classes

- class `com.example.football_manager.config.LocaleConfig`

Packages

- package `com.example.football_manager.config`

9.7 LocaleConfig.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.config;
00002
00003 import java.util.Locale;
00004 import org.springframework.context.annotation.Bean;
00005 import org.springframework.context.annotation.Configuration;
00006 import org.springframework.web.servlet.LocaleResolver;
00007 import org.springframework.web.servlet.config.annotation.InterceptorRegistry;
00008 import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
00009 import org.springframework.web.servlet.i18n.CookieLocaleResolver;
00010 import org.springframework.web.servlet.i18n.LocaleChangeInterceptor;
00011
00012 @Configuration
00013 public class LocaleConfig implements WebMvcConfigurer {
00014

```

```

00015     @Bean
00016     public LocaleResolver localeResolver() {
00017         CookieLocaleResolver resolver = new CookieLocaleResolver();
00018         resolver.setDefaultLocale(new Locale("es"));
00019         return resolver;
00020     }
00021
00022     @Bean
00023     public LocaleChangeInterceptor localeChangeInterceptor() {
00024         LocaleChangeInterceptor interceptor = new LocaleChangeInterceptor();
00025         interceptor.setParamName("lang");
00026         return interceptor;
00027     }
00028
00029     @Override
00030     public void addInterceptors(InterceptorRegistry registry) {
00031         registry.addInterceptor(localeChangeInterceptor());
00032     }
00033 }

```

9.8 Football_Manager/src/main/java/com/example/football_manager/config/OpenApiConfig.java File Reference

```

import io.swagger.v3.oas.annotations.OpenAPIDefinition;
import io.swagger.v3.oas.annotations.info.Info;
import io.swagger.v3.oas.annotations.tags.Tag;
import org.springframework.context.annotation.Configuration;

```

Classes

- class [com.example.football_manager.config.OpenApiConfig](#)

Packages

- package [com.example.football_manager.config](#)

9.9 OpenApiConfig.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.config;
00002
00003 import io.swagger.v3.oas.annotations.OpenAPIDefinition;
00004 import io.swagger.v3.oas.annotations.info.Info;
00005 import io.swagger.v3.oas.annotations.tags.Tag;
00006 import org.springframework.context.annotation.Configuration;
00007
00008 @OpenAPIDefinition(
00009     info = @Info(
00010         title = "Football Manager API",
00011         version = "v1",
00012         description = "REST API for managing teams, matches, competitions, countries, and user authentication."
00013     ),
00014     tags = {
00015         @Tag(name = "Auth", description = "User registration and login"),
00016         @Tag(name = "Teams", description = "Create, update, search, and delete teams"),
00017         @Tag(name = "Matches", description = "Schedule matches and register results"),
00018         @Tag(name = "Competitions", description = "Manage competitions"),
00019         @Tag(name = "Countries", description = "Manage countries"),
00020         @Tag(name = "Favourites", description = "Manage the current user's favourite teams")
00021     }
00022 )
00023 @Configuration
00024 public class OpenApiConfig {
00025 }
00026

```

9.10 Football_Manager/src/main/java/com/example/football_manager/config/SecurityConfig.java File Reference

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
```

Classes

- class [com.example.football_manager.config.SecurityConfig](#)

Packages

- package [com.example.football_manager.config](#)

9.11 SecurityConfig.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.config;
00002
00003 import org.springframework.context.annotation.Bean;
00004 import org.springframework.context.annotation.Configuration;
00005 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
00006 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
00007 import org.springframework.security.crypto.password.PasswordEncoder;
00008 import org.springframework.security.web.SecurityFilterChain;
00009
00010 @Configuration
00011 public class SecurityConfig {
00012
00013     @Bean
00014     public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
00015         http
00016             .csrf(csrf -> csrf.ignoringRequestMatchers("/auth/**", "/api/**"))
00017             .authorizeHttpRequests(auth -> auth
00018                 .anyRequest().permitAll()
00019             );
00020
00021         return http.build();
00022     }
00023
00024     @Bean
00025     public PasswordEncoder passwordEncoder() {
00026         return new BCryptPasswordEncoder();
00027     }
00028 }
00029
```

9.12 Football_Manager/src/main/java/com/example/football_manager/controller/AdminCountryViewController.java File Reference

```
import com.example.football_manager.dto.CountryRequestDTO;
import com.example.football_manager.service.CountryService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;
```

Classes

- class [com.example.football_manager.controller.AdminCountryViewController](#)

Packages

- package [com.example.football_manager.controller](#)

9.13 AdminCountryViewController.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.controller;
00002
00003 import com.example.football_manager.dto.CountryRequestDTO;
00004 import com.example.football_manager.service.CountryService;
00005 import org.springframework.stereotype.Controller;
00006 import org.springframework.ui.Model;
00007 import org.springframework.web.bind.annotation.*;
00008
00009 @Controller
00010 public class AdminCountryViewController {
00011
00012     private final CountryService countryService;
00013
00014     public AdminCountryViewController(CountryService countryService) {
00015         this.countryService = countryService;
00016     }
00017
00018     @GetMapping("/admin/countries")
00019     public String countriesPage(Model model) {
00020         model.addAttribute("countries", countryService.getAllCountries());
00021         model.addAttribute("countryRequest", new CountryRequestDTO());
00022         return "admin-countries";
00023     }
00024
00025     @PostMapping("/admin/countries")
00026     public String createCountry(@ModelAttribute CountryRequestDTO countryRequest) {
00027         countryService.createCountry(countryRequest);
00028         return "redirect:/admin/countries";
00029     }
00030
00031     @PostMapping("/admin/countries/delete/{id}")
00032     public String deleteCountry(@PathVariable Long id) {
00033         countryService.deleteCountry(id);
00034         return "redirect:/admin/countries";
00035     }
00036 }

```

9.14 Football_Manager/src/main/java/com/example/football_manager/controller/AdminPlayerViewController.java File Reference

```

import com.example.football_manager.dto.PlayerRequestDTO;
import com.example.football_manager.model.Player;
import com.example.football_manager.model.PlayerPosition;
import com.example.football_manager.model.Team;
import com.example.football_manager.service.PlayerService;
import com.example.football_manager.service.TeamService;
import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

```

Classes

- class `com.example.football_manager.controller.AdminPlayerViewController`

Packages

- package `com.example.football_manager.controller`

9.15 AdminPlayerViewController.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.controller;
00002
00003 import com.example.football_manager.dto.PlayerRequestDTO;
00004 import com.example.football_manager.model.Player;
00005 import com.example.football_manager.model.PlayerPosition;
00006 import com.example.football_manager.model.Team;
00007 import com.example.football_manager.service.PlayerService;
00008 import com.example.football_manager.service.TeamService;
00009 import jakarta.validation.Valid;
00010 import org.springframework.stereotype.Controller;
00011 import org.springframework.ui.Model;
00012 import org.springframework.validation.BindingResult;
00013 import org.springframework.web.bind.annotation.GetMapping;
00014 import org.springframework.web.bind.annotation.ModelAttribute;
00015 import org.springframework.web.bind.annotation.PathVariable;
00016 import org.springframework.web.bind.annotation.PostMapping;
00017 import org.springframework.web.servlet.mvc.support.RedirectAttributes;
00018
00019 @Controller
00020 public class AdminPlayerViewController {
00021
00022     private final TeamService teamService;
00023     private final PlayerService playerService;
00024
00025     public AdminPlayerViewController(TeamService teamService, PlayerService playerService) {
00026         this.teamService = teamService;
00027         this.playerService = playerService;
00028     }
00029
00030     @GetMapping("/admin/players")
00031     public String managePlayersLanding(Model model) {
00032         return "redirect:/admin/manage-teams";
00033     }
00034
00035     @GetMapping("/admin/teams/{teamId}/players")
00036     public String manageTeamPlayers(@PathVariable Long teamId, Model model, RedirectAttributes
00037         redirectAttributes) {
00038         Team team = teamService.getTeamById(teamId).orElse(null);
00039         if (team == null) {
00040             redirectAttributes.addFlashAttribute("errorMessage", "Team not found.");
00041             return "redirect:/admin/manage-teams";
00042         }
00043         if (!model.containsAttribute("playerRequest")) {
00044             model.addAttribute("playerRequest", new PlayerRequestDTO());
00045         }
00046         populateTeamPlayersModel(model, team);
00047         return "admin-team-players";
00048     }
00049
00050     @PostMapping("/admin/teams/{teamId}/players")
00051     public String createPlayer(
00052         @PathVariable Long teamId,
00053         @Valid @ModelAttribute("playerRequest") PlayerRequestDTO playerRequest,
00054         BindingResult bindingResult,
00055         Model model,
00056         RedirectAttributes redirectAttributes) {
00057         Team team = teamService.getTeamById(teamId).orElse(null);
00058         if (team == null) {
00059             redirectAttributes.addFlashAttribute("errorMessage", "Team not found.");
00060             return "redirect:/admin/manage-teams";
00061         }
00062         if (bindingResult.hasErrors()) {
00063             populateTeamPlayersModel(model, team);
00064             return "admin-team-players";
00065         }
00066     }
00067
00068     }
00069

```

```

00070         try {
00071             playerService.createPlayer(teamId, playerRequest);
00072             redirectAttributes.addFlashAttribute("successMessage", "Player created successfully.");
00073             return "redirect:/admin/teams/" + teamId + "/players";
00074         } catch (IllegalArgumentException ex) {
00075             model.addAttribute("errorMessage", ex.getMessage());
00076             populateTeamPlayersModel(model, team);
00077             return "admin-team-players";
00078         }
00079     }
00080
00081     @GetMapping("/admin/teams/{teamId}/players/edit/{playerId}")
00082     public String showEditPlayerForm(
00083         @PathVariable Long teamId,
00084         @PathVariable Long playerId,
00085         Model model,
00086         RedirectAttributes redirectAttributes) {
00087
00088         Team team = teamService.getTeamById(teamId).orElse(null);
00089         if (team == null) {
00090             redirectAttributes.addFlashAttribute("errorMessage", "Team not found.");
00091             return "redirect:/admin/manage-teams";
00092         }
00093
00094         try {
00095             Player player = playerService.getPlayerByTeamId(teamId, playerId);
00096             if (!model.containsAttribute("playerRequest")) {
00097                 PlayerRequestDTO playerRequest = new PlayerRequestDTO(
00098                     player.getName(),
00099                     player.getNumber(),
00100                     player.getPosition()
00101                 );
00102                 model.addAttribute("playerRequest", playerRequest);
00103             }
00104         } catch (IllegalArgumentException ex) {
00105             redirectAttributes.addFlashAttribute("errorMessage", ex.getMessage());
00106             return "redirect:/admin/teams/" + teamId + "/players";
00107         }
00108
00109         model.addAttribute("team", team);
00110         model.addAttribute("playerId", playerId);
00111         model.addAttribute("positions", PlayerPosition.values());
00112         return "edit-player";
00113     }
00114
00115     @PostMapping("/admin/teams/{teamId}/players/edit/{playerId}")
00116     public String updatePlayer(
00117         @PathVariable Long teamId,
00118         @PathVariable Long playerId,
00119         @Valid @ModelAttribute("playerRequest") PlayerRequestDTO playerRequest,
00120         BindingResult bindingResult,
00121         Model model,
00122         RedirectAttributes redirectAttributes) {
00123
00124         Team team = teamService.getTeamById(teamId).orElse(null);
00125         if (team == null) {
00126             redirectAttributes.addFlashAttribute("errorMessage", "Team not found.");
00127             return "redirect:/admin/manage-teams";
00128         }
00129
00130         if (bindingResult.hasErrors()) {
00131             model.addAttribute("team", team);
00132             model.addAttribute("playerId", playerId);
00133             model.addAttribute("positions", PlayerPosition.values());
00134             return "edit-player";
00135         }
00136
00137         try {
00138             playerService.updatePlayer(teamId, playerId, playerRequest);
00139             redirectAttributes.addFlashAttribute("successMessage", "Player updated successfully.");
00140             return "redirect:/admin/teams/" + teamId + "/players";
00141         } catch (IllegalArgumentException ex) {
00142             model.addAttribute("team", team);
00143             model.addAttribute("playerId", playerId);
00144             model.addAttribute("positions", PlayerPosition.values());
00145             model.addAttribute("errorMessage", ex.getMessage());
00146             return "edit-player";
00147         }
00148     }
00149
00150     @PostMapping("/admin/teams/{teamId}/players/delete/{playerId}")
00151     public String deletePlayer(
00152         @PathVariable Long teamId,
00153         @PathVariable Long playerId,
00154         RedirectAttributes redirectAttributes) {
00155         try {
00156             playerService.deletePlayer(teamId, playerId);

```


9.16

Football_Manager/src/main/java/com/example/football_manager/controller/AdminUserController.java File Reference

85

```
00157         redirectAttributes.addFlashAttribute("successMessage", "Player deleted successfully.");
00158     } catch (IllegalArgumentException ex) {
00159         redirectAttributes.addFlashAttribute("errorMessage", ex.getMessage());
00160     }
00161 }
00162     return "redirect:/admin/teams/" + teamId + "/players";
00163 }
00164 }
00165 private void populateTeamPlayersModel(Model model, Team team) {
00166     model.addAttribute("team", team);
00167     model.addAttribute("players", playerService.getPlayersByTeamId(team.getId()));
00168     model.addAttribute("positions", PlayerPosition.values());
00169 }
00170 }
```

9.16 Football_Manager/src/main/java/com/example/football_manager/controller/AdminUserController.java File Reference

```
import com.example.football_manager.service.UserService;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.*;
```

Classes

- class [com.example.football_manager.controller.AdminUserController](#)

Packages

- package [com.example.football_manager.controller](#)

9.17 AdminUserController.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.controller;
00002
00003 import com.example.football_manager.service.UserService;
00004 import org.springframework.stereotype.Controller;
00005 import org.springframework.web.bind.annotation.*;
00006
00007 @Controller
00008 public class AdminUserController {
00009
00010     private final UserService userService;
00011
00012     public AdminUserController(UserService userService) {
00013         this.userService = userService;
00014     }
00015
00016     @GetMapping("/admin/users")
00017     public String usersPage() {
00018         return "admin-users";
00019     }
00020
00021     @PostMapping("/admin/users")
00022     public String createUser(@RequestParam String username,
00023                             @RequestParam String email,
00024                             @RequestParam String password,
00025                             @RequestParam(defaultValue = "false") Boolean isAdmin) {
00026         userService.registerUser(username, email, password, isAdmin);
00027         return "redirect:/admin/users";
00028     }
00029 }
```

9.18 Football_Manager/src/main/java/com/example/football_manager/controller/AdminViewController.java File Reference

```
import com.example.football_manager.service.TeamService;
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
```

Classes

- class [com.example.football_manager.controller.AdminViewController](#)

Packages

- package [com.example.football_manager.controller](#)

9.19 AdminViewController.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.controller;
00002
00003 import com.example.football_manager.service.TeamService;
00004 import org.springframework.beans.factory.annotation.Autowired;
00005 import org.springframework.stereotype.Controller;
00006 import org.springframework.ui.Model;
00007 import org.springframework.web.bind.annotation.GetMapping;
00008
00009 @Controller
00010 public class AdminViewController {
00011
00012     @Autowired
00013     private TeamService teamService;
00014
00015     @GetMapping("/admin")
00016     public String adminPanel() {
00017         return "admin-panel";
00018     }
00019
00020     @GetMapping("/admin/manage-teams")
00021     public String manageTeams(Model model) {
00022         model.addAttribute("teams", teamService.getAllTeams());
00023         return "admin-manage-teams";
00024     }
00025 }
```

9.20 Football_Manager/src/main/java/com/example/football_manager/controller/AuthController.java File Reference ↩

```
import com.example.football_manager.dto.AuthResponseDTO;
import com.example.football_manager.dto.LoginDTO;
import com.example.football_manager.dto.RegisterDTO;
import com.example.football_manager.model.User;
import com.example.football_manager.service.UserService;
import jakarta.servlet.http.HttpSession;
import jakarta.validation.Valid;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
```

Classes

- class [com.example.football_manager.controller.AuthController](#)

Packages

- package [com.example.football_manager.controller](#)

9.21 AuthController.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.controller;
00002
00003 import com.example.football_manager.dto.AuthResponseDTO;
00004 import com.example.football_manager.dto.LoginDTO;
00005 import com.example.football_manager.dto.RegisterDTO;
00006 import com.example.football_manager.model.User;
00007 import com.example.football_manager.service.UserService;
00008 import jakarta.servlet.http.HttpSession;
00009 import jakarta.validation.Valid;
00010 import io.swagger.v3.oas.annotations.Operation;
00011 import io.swagger.v3.oas.annotations.media.Content;
00012 import io.swagger.v3.oas.annotations.media.Schema;
00013 import io.swagger.v3.oas.annotations.responses.ApiResponse;
00014 import io.swagger.v3.oas.annotations.responses.ApiResponses;
00015 import io.swagger.v3.oas.annotations.tags.Tag;
00016 import org.springframework.beans.factory.annotation.Autowired;
00017 import org.springframework.http.HttpStatus;
00018 import org.springframework.http.ResponseEntity;
00019 import org.springframework.web.bind.annotation.*;
00020
00021 @RestController
00022 @RequestMapping("/auth")
00023 @Tag(name = "Auth")
00024 public class AuthController {
00025
00026     @Autowired
00027     private UserService userService;
00028
00029     @PostMapping("/register")
00030     @Operation(
00031         summary = "Register a user",
00032         description = "Creates a new user account. Returns a message indicating success or
00033 conflict.",
00034         requestBody = @io.swagger.v3.oas.annotations.parameters.RequestBody(
00035             required = true,
00036             description = "Registration details",
00037             content = @Content(schema = @Schema(implementation = RegisterDTO.class))
00038         )
00039     )
00040     @ApiResponses({
00041         @ApiResponse(responseCode = "201", description = "User registered"),
00042         @ApiResponse(responseCode = "409", description = "Username or email already exists"),
00043         @ApiResponse(responseCode = "400", description = "Validation error")
00044     })
00045     public ResponseEntity<AuthResponseDTO> register(@Valid @RequestBody RegisterDTO registerDTO) {
00046         try {
00047             User user = userService.registerUser(
00048                 registerDTO.getUsername(),
00049                 registerDTO.getEmail(),
00050                 registerDTO.getPassword(),
00051                 registerDTO.getIsAdmin()
00052             );
00053             return ResponseEntity.status(HttpStatus.CREATED)
00054                 .body(new AuthResponseDTO(user.getId(), user.getUsername(), user.getEmail(),
00055 user.getIsAdmin(), "User registered"));
00056         } catch (IllegalArgumentException ex) {
00057             return ResponseEntity.status(HttpStatus.CONFLICT)
00058                 .body(new AuthResponseDTO(null, null, null, null, ex.getMessage()));
00059         }
00060     }
00061
00062     @PostMapping("/login")
00063     @Operation(
00064         summary = "Login",
00065         description = "Authenticates the user and stores the session attributes for subsequent
00066 requests.",
00067         requestBody = @io.swagger.v3.oas.annotations.parameters.RequestBody(
00068             required = true,
00069             description = "Login credentials",

```

```

00068             content = @Content(schema = @Schema(implementation = LoginDTO.class))
00069         )
00070     )
00071     @ApiResponses({
00072         @ApiResponse(responseCode = "200", description = "Login successful"),
00073         @ApiResponse(responseCode = "401", description = "Invalid credentials"),
00074         @ApiResponse(responseCode = "400", description = "Validation error")
00075     })
00076     public ResponseEntity<AuthResponseDTO> login(@Valid @RequestBody LoginDTO loginDTO, HttpSession
session) {
00077         try {
00078             User user = userService.login(loginDTO.getUsername(), loginDTO.getPassword());
00079
00080             session.setAttribute("userId", user.getId());
00081             session.setAttribute("isAdmin", user.getIsAdmin());
00082
00083             return ResponseEntity.ok(
00084                 new AuthResponseDTO(user.getId(), user.getUsername(), user.getEmail(),
user.getIsAdmin(), "Login successful")
00085             );
00086         } catch (IllegalArgumentException ex) {
00087             return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
00088                 .body(new AuthResponseDTO(null, null, null, null, ex.getMessage()));
00089         }
00090     }
00091 }

```

9.22 Football_Manager/src/main/java/com/example/football_ manager/controller/AuthViewController.java File Reference

```

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

```

Classes

- class [com.example.football_manager.controller.AuthViewController](#)

Packages

- package [com.example.football_manager.controller](#)

9.23 AuthViewController.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.controller;
00002
00003 import org.springframework.stereotype.Controller;
00004 import org.springframework.web.bind.annotation.GetMapping;
00005
00006 @Controller
00007 public class AuthViewController {
00008
00009
00010     @GetMapping("/")
00011     public String index() {
00012         return "redirect:/login";
00013     }
00014
00015     @GetMapping("/login")
00016     public String loginPage() {
00017         return "login"; //
00018     }
00019
00020     @GetMapping("/register")
00021     public String registerPage() {
00022         return "register"; //
00023     }
00024 }

```

9.24 Football_Manager/src/main/java/com/example/football_manager/controller/CompetitionController.java File Reference

```
import com.example.football_manager.dto.CompetitionRequestDTO;
import com.example.football_manager.model.Competition;
import com.example.football_manager.service.CompetitionService;
import jakarta.validation.Valid;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
```

Classes

- class [com.example.football_manager.controller.CompetitionController](#)

Packages

- package [com.example.football_manager.controller](#)

9.25 CompetitionController.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.controller;
00002
00003 import com.example.football_manager.dto.CompetitionRequestDTO;
00004 import com.example.football_manager.model.Competition;
00005 import com.example.football_manager.service.CompetitionService;
00006 import jakarta.validation.Valid;
00007 import io.swagger.v3.oas.annotations.Operation;
00008 import io.swagger.v3.oas.annotations.media.Content;
00009 import io.swagger.v3.oas.annotations.media.Schema;
00010 import io.swagger.v3.oas.annotations.responses.ApiResponse;
00011 import io.swagger.v3.oas.annotations.responses.ApiResponses;
00012 import io.swagger.v3.oas.annotations.tags.Tag;
00013 import org.springframework.http.HttpStatus;
00014 import org.springframework.http.ResponseEntity;
00015 import org.springframework.web.bind.annotation.*;
00016
00017 import java.util.List;
00018
00019 @RestController
00020 @RequestMapping("/api/competitions")
00021 @Tag(name = "Competitions")
00022 public class CompetitionController {
00023
00024     private final CompetitionService competitionService;
00025
00026     public CompetitionController(CompetitionService competitionService) {
00027         this.competitionService = competitionService;
00028     }
00029
00030     @PostMapping
00031     @Operation(
00032         summary = "Create a competition",
00033         description = "Creates a new competition entry.",
00034         requestBody = @io.swagger.v3.oas.annotations.parameters.RequestBody(
00035             required = true,
00036             description = "Competition details",
00037             content = @Content(schema = @Schema(implementation = CompetitionRequestDTO.class))
00038         )
00039     )
00040     @ApiResponses({
00041         @ApiResponse(responseCode = "201", description = "Competition created"),
```

```

00042         @ApiResponse(responseCode = "400", description = "Validation error")
00043     })
00044     public ResponseEntity<Competition> createCompetition(@Valid @RequestBody CompetitionRequestDTO
dto) {
00045         Competition createdCompetition = competitionService.createCompetition(dto);
00046         return new ResponseEntity<>(createdCompetition, HttpStatus.CREATED);
00047     }
00048
00049     @PutMapping("/{id}")
00050     @Operation(
00051         summary = "Update a competition",
00052         description = "Updates the competition name.",
00053         requestBody = @io.swagger.v3.oas.annotations.parameters.RequestBody(
00054             required = true,
00055             description = "Updated competition details",
00056             content = @Content(schema = @Schema(implementation = CompetitionRequestDTO.class))
00057         )
00058     )
00059     @ApiResponses({
00060         @ApiResponse(responseCode = "200", description = "Competition updated"),
00061         @ApiResponse(responseCode = "400", description = "Validation error"),
00062         @ApiResponse(responseCode = "404", description = "Competition not found")
00063     })
00064     public ResponseEntity<Competition> updateCompetition(@PathVariable Long id,
00065         @Valid @RequestBody CompetitionRequestDTO
dto) {
00066         Competition updatedCompetition = competitionService.updateCompetition(id, dto);
00067         return ResponseEntity.ok(updatedCompetition);
00068     }
00069
00070     @GetMapping("/{id}")
00071     @Operation(
00072         summary = "Get competition by id",
00073         description = "Returns competition details for the specified id."
00074     )
00075     @ApiResponses({
00076         @ApiResponse(responseCode = "200", description = "Competition found"),
00077         @ApiResponse(responseCode = "404", description = "Competition not found")
00078     })
00079     public ResponseEntity<Competition> getCompetitionById(@PathVariable Long id) {
00080         return ResponseEntity.of(competitionService.getCompetitionById(id));
00081     }
00082
00083     @GetMapping
00084     @Operation(
00085         summary = "List all competitions",
00086         description = "Returns every competition in the system."
00087     )
00088     @ApiResponses({
00089         @ApiResponse(responseCode = "200", description = "Competition list")
00090     })
00091     public ResponseEntity<List<Competition>> getAllCompetitions() {
00092         return ResponseEntity.ok(competitionService.getAllCompetitions());
00093     }
00094
00095     @DeleteMapping("/{id}")
00096     @Operation(
00097         summary = "Delete a competition",
00098         description = "Deletes a competition by its identifier."
00099     )
00100     @ApiResponses({
00101         @ApiResponse(responseCode = "204", description = "Competition deleted"),
00102         @ApiResponse(responseCode = "404", description = "Competition not found")
00103     })
00104     public ResponseEntity<Void> deleteCompetition(@PathVariable Long id) {
00105         competitionService.deleteCompetition(id);
00106         return ResponseEntity.noContent().build();
00107     }
00108 }

```

9.26 Football_Manager/src/main/java/com/example/football_↵ manager/controller/CountryController.java File Reference

```

import com.example.football_manager.dto.CountryRequestDTO;
import com.example.football_manager.model.Country;
import com.example.football_manager.service.CountryService;
import jakarta.validation.Valid;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;

```

```
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
```

Classes

- class [com.example.football_manager.controller.CountryController](#)

Packages

- package [com.example.football_manager.controller](#)

9.27 CountryController.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.controller;
00002
00003 import com.example.football_manager.dto.CountryRequestDTO;
00004 import com.example.football_manager.model.Country;
00005 import com.example.football_manager.service.CountryService;
00006 import jakarta.validation.Valid;
00007 import io.swagger.v3.oas.annotations.Operation;
00008 import io.swagger.v3.oas.annotations.media.Content;
00009 import io.swagger.v3.oas.annotations.media.Schema;
00010 import io.swagger.v3.oas.annotations.responses.ApiResponse;
00011 import io.swagger.v3.oas.annotations.responses.ApiResponses;
00012 import io.swagger.v3.oas.annotations.tags.Tag;
00013 import org.springframework.http.HttpStatus;
00014 import org.springframework.http.ResponseEntity;
00015 import org.springframework.web.bind.annotation.*;
00016
00017 import java.util.List;
00018
00019 @RestController
00020 @RequestMapping("/api/countries")
00021 @Tag(name = "Countries")
00022 public class CountryController {
00023
00024     private final CountryService countryService;
00025
00026     public CountryController(CountryService countryService) {
00027         this.countryService = countryService;
00028     }
00029
00030     @PostMapping
00031     @Operation(
00032         summary = "Create a country",
00033         description = "Creates a new country entry.",
00034         requestBody = @io.swagger.v3.oas.annotations.parameters.RequestBody(
00035             required = true,
00036             description = "Country details",
00037             content = @Content(schema = @Schema(implementation = CountryRequestDTO.class))
00038         )
00039     )
00040     @ApiResponses({
00041         @ApiResponse(responseCode = "201", description = "Country created"),
00042         @ApiResponse(responseCode = "400", description = "Validation error")
00043     })
00044     public ResponseEntity<Country> createCountry(@Valid @RequestBody CountryRequestDTO dto) {
00045         Country createdCountry = countryService.createCountry(dto);
00046         return new ResponseEntity<>(createdCountry, HttpStatus.CREATED);
00047     }
00048
00049     @GetMapping
00050     @Operation(
00051         summary = "List all countries",
00052         description = "Returns every country currently in the system."
00053     )
00054     @ApiResponses({
00055         @ApiResponse(responseCode = "200", description = "Country list")
00056     })
```

```

00057     public ResponseEntity<List<Country>> getAllCountries() {
00058         List<Country> countries = countryService.getAllCountries();
00059         return ResponseEntity.ok(countries);
00060     }
00061
00062     @DeleteMapping("/{id}")
00063     @Operation(
00064         summary = "Delete a country",
00065         description = "Deletes a country by its identifier."
00066     )
00067     @ApiResponses({
00068         @ApiResponse(responseCode = "204", description = "Country deleted"),
00069         @ApiResponse(responseCode = "404", description = "Country not found")
00070     })
00071     public ResponseEntity<Void> deleteCountry(@PathVariable Long id) {
00072         countryService.deleteCountry(id);
00073         return ResponseEntity.noContent().build();
00074     }
00075 }

```

9.28 Football_Manager/src/main/java/com/example/football_manager/controller/FavouriteTeamController.java File Reference

```

import com.example.football_manager.service.UserService;
import jakarta.servlet.http.HttpSession;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

```

Classes

- class [com.example.football_manager.controller.FavouriteTeamController](#)

Packages

- package [com.example.football_manager.controller](#)

9.29 FavouriteTeamController.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.controller;
00002
00003 import com.example.football_manager.service.UserService;
00004 import jakarta.servlet.http.HttpSession;
00005 import io.swagger.v3.oas.annotations.Operation;
00006 import io.swagger.v3.oas.annotations.responses.ApiResponse;
00007 import io.swagger.v3.oas.annotations.responses.ApiResponses;
00008 import io.swagger.v3.oas.annotations.tags.Tag;
00009 import org.springframework.http.HttpStatus;
00010 import org.springframework.http.ResponseEntity;
00011 import org.springframework.web.bind.annotation.DeleteMapping;
00012 import org.springframework.web.bind.annotation.PathVariable;
00013 import org.springframework.web.bind.annotation.PostMapping;
00014 import org.springframework.web.bind.annotation.RequestMapping;
00015 import org.springframework.web.bind.annotation.RestController;
00016
00017 @RestController
00018 @RequestMapping("/api/users/me/favourites/teams")
00019 @Tag(name = "Favourites")
00020 public class FavouriteTeamController {

```



```
00021
00022     private final UserService userService;
00023
00024     public FavouriteTeamController(UserService userService) {
00025         this.userService = userService;
00026     }
00027
00028     @PostMapping("/{teamId}")
00029     @Operation(
00030         summary = "Add favourite team",
00031         description = "Adds the team to the current user's favourites. Requires a logged-in
session."
00032     )
00033     @ApiResponses({
00034         @ApiResponse(responseCode = "201", description = "Team added to favourites"),
00035         @ApiResponse(responseCode = "401", description = "User not logged in"),
00036         @ApiResponse(responseCode = "404", description = "User or team not found"),
00037         @ApiResponse(responseCode = "409", description = "Team already in favourites")
00038     })
00039     public ResponseEntity<String> addFavouriteTeam(@PathVariable Long teamId, HttpSession session) {
00040         Long userId = (Long) session.getAttribute("userId");
00041         if (userId == null) {
00042             return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("You must be logged in");
00043         }
00044
00045         try {
00046             userService.addFavouriteTeam(userId, teamId);
00047             return ResponseEntity.status(HttpStatus.CREATED).body("Team added to favourites");
00048         } catch (IllegalArgumentException ex) {
00049             return mapError(ex.getMessage());
00050         }
00051     }
00052
00053     @DeleteMapping("/{teamId}")
00054     @Operation(
00055         summary = "Remove favourite team",
00056         description = "Removes the team from the current user's favourites. Requires a logged-in
session."
00057     )
00058     @ApiResponses({
00059         @ApiResponse(responseCode = "200", description = "Team removed from favourites"),
00060         @ApiResponse(responseCode = "401", description = "User not logged in"),
00061         @ApiResponse(responseCode = "404", description = "User or team not found"),
00062         @ApiResponse(responseCode = "409", description = "Team not in favourites")
00063     })
00064     public ResponseEntity<String> removeFavouriteTeam(@PathVariable Long teamId, HttpSession session)
{
00065         Long userId = (Long) session.getAttribute("userId");
00066         if (userId == null) {
00067             return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("You must be logged in");
00068         }
00069
00070         try {
00071             userService.removeFavouriteTeam(userId, teamId);
00072             return ResponseEntity.ok("Team removed from favourites");
00073         } catch (IllegalArgumentException ex) {
00074             return mapError(ex.getMessage());
00075         }
00076     }
00077
00078     private ResponseEntity<String> mapError(String message) {
00079         if ("User not found".equals(message) || "Team not found".equals(message)) {
00080             return ResponseEntity.status(HttpStatus.NOT_FOUND).body(message);
00081         }
00082
00083         if ("Team already in favourites".equals(message) || "Team not in favourites".equals(message))
{
00084             return ResponseEntity.status(HttpStatus.CONFLICT).body(message);
00085         }
00086
00087         return ResponseEntity.badRequest().body(message);
00088     }
00089 }
00090
```

9.30 Football_Manager/src/main/java/com/example/football_ manager/controller/GlobalModelAttributes.java File Reference

```
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpSession;
import org.springframework.ui.Model;
```

```
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ModelAttribute;
```

Classes

- class [com.example.football_manager.controller.GlobalModelAttributes](#)

Packages

- package [com.example.football_manager.controller](#)

9.31 GlobalModelAttributes.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.controller;
00002
00003 import jakarta.servlet.http.HttpServletRequest;
00004 import jakarta.servlet.http.HttpSession;
00005 import org.springframework.ui.Model;
00006 import org.springframework.web.bind.annotation.ControllerAdvice;
00007 import org.springframework.web.bind.annotation.ModelAttribute;
00008
00009 @ControllerAdvice
00010 public class GlobalModelAttributes {
00011
00012     @ModelAttribute
00013     public void addIsAdmin(Model model, HttpServletRequest request) {
00014         HttpSession session = request.getSession(false);
00015         Boolean isAdmin = session != null ? (Boolean) session.getAttribute("isAdmin") : null;
00016         model.addAttribute("isAdmin", isAdmin != null && isAdmin);
00017     }
00018 }
00019
```

9.32 Football_Manager/src/main/java/com/example/football_manager/controller/MatchController.java File Reference

```
import com.example.football_manager.dto.MatchRequestDTO;
import com.example.football_manager.dto.MatchResultDTO;
import com.example.football_manager.dto.MatchResultRequestDTO;
import com.example.football_manager.model.Match;
import com.example.football_manager.service.MatchService;
import jakarta.validation.Valid;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.server.ResponseStatusException;
import org.springframework.web.bind.annotation.*;
import java.util.List;
```

Classes

- class [com.example.football_manager.controller.MatchController](#)

Packages

- package [com.example.football_manager.controller](#)

9.33 MatchController.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.controller;
00002
00003 import com.example.football_manager.dto.MatchRequestDTO;
00004 import com.example.football_manager.dto.MatchResultDTO;
00005 import com.example.football_manager.dto.MatchResultRequestDTO;
00006 import com.example.football_manager.model.Match;
00007 import com.example.football_manager.service.MatchService;
00008 import jakarta.validation.Valid;
00009 import io.swagger.v3.oas.annotations.Operation;
00010 import io.swagger.v3.oas.annotations.media.Content;
00011 import io.swagger.v3.oas.annotations.media.Schema;
00012 import io.swagger.v3.oas.annotations.responses.ApiResponse;
00013 import io.swagger.v3.oas.annotations.responses.ApiResponses;
00014 import io.swagger.v3.oas.annotations.tags.Tag;
00015 import org.springframework.beans.factory.annotation.Autowired;
00016 import org.springframework.http.HttpStatus;
00017 import org.springframework.http.MediaType;
00018 import org.springframework.http.ResponseEntity;
00019 import org.springframework.web.server.ResponseStatusException;
00020 import org.springframework.web.bind.annotation.*;
00021
00022 import java.util.List;
00023
00024 @RestController
00025 @RequestMapping("/api/matches")
00026 @Tag(name = "Matches")
00027 public class MatchController {
00028
00029     @Autowired
00030     private MatchService matchService;
00031
00032     @GetMapping
00033     @Operation(
00034         summary = "List matches",
00035         description = "Returns matches, optionally filtered by team, status, or competition."
00036     )
00037     @ApiResponses({
00038         @ApiResponse(responseCode = "200", description = "Match list"),
00039         @ApiResponse(responseCode = "400", description = "Unsupported filter value")
00040     })
00041     public ResponseEntity<List<Match>> getMatches(
00042         @RequestParam(required = false) Long teamId,
00043         @RequestParam(required = false) MatchRequestDTO.MatchStatus status,
00044         @RequestParam(required = false) Long competitionId) {
00045         try {
00046             return ResponseEntity.ok(matchService.getMatches(teamId, status, competitionId));
00047         } catch (IllegalArgumentException e) {
00048             throw new ResponseStatusException(HttpStatus.BAD_REQUEST, e.getMessage(), e);
00049         }
00050     }
00051
00052     @GetMapping("/upcoming")
00053     @Operation(
00054         summary = "List all upcoming matches",
00055         description = "Returns matches that have not finished yet, sorted by date ascending."
00056     )
00057     public ResponseEntity<List<Match>> getUpcomingMatches() {
00058         return ResponseEntity.ok(matchService.getUpcomingMatches());
00059     }
00060
00061     @PostMapping(consumes = MediaType.APPLICATION_JSON_VALUE)
00062     @Operation(
00063         summary = "Create a match (JSON)",
00064         description = "Creates a match using a JSON body. Use this for API clients.",
00065         requestBody = @io.swagger.v3.oas.annotations.parameters.RequestBody(
00066             required = true,
00067             description = "Match details",
00068             content = @Content(schema = @Schema(implementation = MatchRequestDTO.class))
00069         )
00070     )
00071     @ApiResponses({
00072         @ApiResponse(responseCode = "201", description = "Match created"),
00073         @ApiResponse(responseCode = "400", description = "Validation or business rule error")
00074     })
00075     public ResponseEntity<String> createMatchFromJson(@Valid @RequestBody MatchRequestDTO matchDTO) {
00076         return createMatchResponse(matchDTO);

```

```

00077     }
00078
00079     @PostMapping(consumes = MediaType.APPLICATION_FORM_URLENCODED_VALUE)
00080     @Operation(
00081         summary = "Create a match (form)",
00082         description = "Creates a match using form-encoded fields. Useful for HTML form
submissions.",
00083         requestBody = @io.swagger.v3.oas.annotations.parameters.RequestBody(
00084             required = true,
00085             description = "Match form fields",
00086             content = @Content(schema = @Schema(implementation = MatchRequestDTO.class))
00087         )
00088     )
00089     @ApiResponse({
00090         @ApiResponse(responseCode = "201", description = "Match created"),
00091         @ApiResponse(responseCode = "400", description = "Validation or business rule error")
00092     })
00093     public ResponseEntity<String> createMatchFromForm(@Valid @ModelAttribute MatchRequestDTO matchDTO)
{
00094         return createMatchResponse(matchDTO);
00095     }
00096
00097     private ResponseEntity<String> createMatchResponse(MatchRequestDTO matchDTO) {
00098         try {
00099             String response = matchService.createMatch(matchDTO);
00100             return new ResponseEntity<>(response, HttpStatus.CREATED);
00101         } catch (IllegalArgumentException e) {
00102             return new ResponseEntity<>(e.getMessage(), HttpStatus.BAD_REQUEST);
00103         }
00104     }
00105
00106     @GetMapping("/results")
00107     @Operation(
00108         summary = "List finished match results",
00109         description = "Returns results for matches with registered scores."
00110     )
00111     @ApiResponse({
00112         @ApiResponse(responseCode = "200", description = "Match results")
00113     })
00114     public ResponseEntity<List<MatchResultDTO>> getMatchResults() {
00115         return ResponseEntity.ok(matchService.getFinishedMatchResults());
00116     }
00117
00118     // Edit match details
00119     @PostMapping("/{id}")
00120     @Operation(
00121         summary = "Update match details",
00122         description = "Edits match details such as kickoff time, venue, or participants.",
00123         requestBody = @io.swagger.v3.oas.annotations.parameters.RequestBody(
00124             required = true,
00125             description = "Match updates",
00126             content = @Content(schema = @Schema(implementation = MatchRequestDTO.class))
00127         )
00128     )
00129     @ApiResponse({
00130         @ApiResponse(responseCode = "200", description = "Match updated"),
00131         @ApiResponse(responseCode = "400", description = "Validation or business rule error")
00132     })
00133     public ResponseEntity<String> updateMatch(@PathVariable Long id, @RequestBody MatchRequestDTO
matchDTO) {
00134         try {
00135             return ResponseEntity.ok(matchService.updateMatch(id, matchDTO));
00136         } catch (IllegalArgumentException e) {
00137             return new ResponseEntity<>(e.getMessage(), HttpStatus.BAD_REQUEST);
00138         }
00139     }
00140
00141     @DeleteMapping("/{id}")
00142     @Operation(
00143         summary = "Delete a match",
00144         description = "Removes a match by its identifier."
00145     )
00146     @ApiResponse({
00147         @ApiResponse(responseCode = "200", description = "Match deleted"),
00148         @ApiResponse(responseCode = "400", description = "Match not found or cannot be deleted")
00149     })
00150     public ResponseEntity<String> deleteMatch(@PathVariable Long id) {
00151         try {
00152             return ResponseEntity.ok(matchService.deleteMatch(id));
00153         } catch (IllegalArgumentException e) {
00154             return new ResponseEntity<>(e.getMessage(), HttpStatus.BAD_REQUEST);
00155         }
00156     }
00157
00158     @PatchMapping("/{id}/result")
00159     @Operation(
00160         summary = "Register match result",

```

```
00161         description = "Registers goals and final score for a match.",
00162         requestBody = @io.swagger.v3.oas.annotations.parameters.RequestBody(
00163             required = true,
00164             description = "List of goals to register",
00165             content = @Content(schema = @Schema(implementation = MatchResultRequestDTO.class))
00166         )
00167     )
00168     @ApiResponses({
00169         @ApiResponse(responseCode = "200", description = "Result registered"),
00170         @ApiResponse(responseCode = "400", description = "Validation or business rule error")
00171     })
00172     public ResponseEntity<String> registerResult(
00173         @PathVariable Long id,
00174         @Valid @RequestBody MatchResultRequestDTO resultDTO) {
00175         try {
00176             return ResponseEntity.ok(matchService.registerResult(id, resultDTO));
00177         } catch (IllegalArgumentException e) {
00178             return new ResponseEntity<>(e.getMessage(), HttpStatus.BAD_REQUEST);
00179         }
00180     }
00181 }
```

9.34 Football_Manager/src/main/java/com/example/football_manager/controller/MatchViewController.java File Reference

```
import com.example.football_manager.dto.MatchRequestDTO;
import com.example.football_manager.dto.MatchResultRequestDTO;
import com.example.football_manager.model.Match;
import com.example.football_manager.model.MatchGoal;
import com.example.football_manager.service.MatchService;
import com.example.football_manager.service.TeamService;
import jakarta.servlet.http.HttpSession;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.server.ResponseStatusException;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;
import java.util.ArrayList;
import java.util.List;
```

Classes

- class [com.example.football_manager.controller.MatchViewController](#)

Packages

- package [com.example.football_manager.controller](#)

9.35 MatchViewController.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.controller;
00002
00003 import com.example.football_manager.dto.MatchRequestDTO;
00004 import com.example.football_manager.dto.MatchResultRequestDTO;
00005 import com.example.football_manager.model.Match;
00006 import com.example.football_manager.model.MatchGoal;
00007 import com.example.football_manager.service.MatchService;
00008 import com.example.football_manager.service.TeamService;
00009
```

```

00010 import jakarta.servlet.http.HttpSession;
00011 import org.springframework.http.HttpStatus;
00012 import org.springframework.stereotype.Controller;
00013 import org.springframework.ui.Model;
00014 import org.springframework.web.bind.annotation.GetMapping;
00015 import org.springframework.web.bind.annotation.ModelAttribute;
00016 import org.springframework.web.bind.annotation.PathVariable;
00017 import org.springframework.web.bind.annotation.PostMapping;
00018 import org.springframework.web.bind.annotation.RequestParam;
00019 import org.springframework.web.server.ResponseStatusException;
00020 import org.springframework.web.servlet.mvc.support.RedirectAttributes;
00021
00022 import java.util.ArrayList;
00023 import java.util.List;
00024
00025 @Controller
00026 public class MatchViewController {
00027
00028     private final MatchService matchService;
00029     private final TeamService teamService;
00030
00031     public MatchViewController(MatchService matchService, TeamService teamService) {
00032         this.matchService = matchService;
00033         this.teamService = teamService;
00034     }
00035
00036     @GetMapping("/matches")
00037     public String showMatchesPage(
00038         @RequestParam(required = false) Long teamId,
00039         @RequestParam(required = false) MatchRequestDTO.MatchStatus status,
00040         @RequestParam(required = false) Long competitionId,
00041         Model model,
00042         HttpSession session) {
00043         Boolean isAdmin = (Boolean) session.getAttribute("isAdmin");
00044
00045         model.addAttribute("isAdmin", isAdmin != null && isAdmin);
00046         try {
00047             model.addAttribute("matches", matchService.getMatches(teamId, status, competitionId));
00048         } catch (IllegalArgumentException e) {
00049             throw new ResponseStatusException(HttpStatus.BAD_REQUEST, e.getMessage(), e);
00050         }
00051         model.addAttribute("teams", teamService.getAllTeams());
00052         model.addAttribute("selectedTeamId", teamId);
00053
00054         return "matches";
00055     }
00056
00057     @GetMapping("/matches/schedule")
00058     public String showScheduleForm(Model model) {
00059         model.addAttribute("matchRequest", new MatchRequestDTO());
00060         model.addAttribute("teams", teamService.getAllTeams());
00061         return "schedule-match";
00062     }
00063
00064     @GetMapping("/matches/results")
00065     public String showResultsPage(Model model) {
00066         model.addAttribute("results", matchService.getFinishedMatchResults());
00067         return "match-results";
00068     }
00069
00070     @GetMapping("/matches/edit/{id}")
00071     public String showEditForm(@PathVariable Long id, Model model) {
00072         MatchRequestDTO matchRequestDTO = matchService.getMatchForEdit(id)
00073             .orElseThrow(() -> new IllegalArgumentException("Match not found."));
00074
00075         model.addAttribute("matchRequest", matchRequestDTO);
00076         model.addAttribute("matchId", id);
00077         // PASAR LA LISTA DE EQUIPOS PARA LOS DESPLEGABLES
00078         model.addAttribute("teams", teamService.getAllTeams());
00079         return "edit-match";
00080     }
00081
00082     @PostMapping("/matches/edit/{id}")
00083     public String updateMatch(@PathVariable Long id, @ModelAttribute("matchRequest") MatchRequestDTO
matchDTO) {
00084         matchService.updateMatch(id, matchDTO);
00085         return "redirect:/matches/results";
00086     }
00087
00088
00089     @GetMapping("/matches/{id}/score")
00090     public String showScoreForm(@PathVariable Long id, Model model) {
00091         Match match = matchService.getMatchById(id);
00092
00093         if (!model.containsAttribute("resultRequest")) {
00094             model.addAttribute("resultRequest", toResultRequest(matchService.getGoalsByMatchId(id)));
00095         }

```

```

00096
00097     model.addAttribute("match", match);
00098     return "enter-match-score";
00099 }
00100
00101 @PostMapping("/matches/{id}/score")
00102 public String submitScoreForm(
00103     @PathVariable Long id,
00104     @ModelAttribute("resultRequest") MatchResultRequestDTO resultRequest,
00105     Model model,
00106     RedirectAttributes redirectAttributes) {
00107
00108     Match match = matchService.getMatchById(id);
00109     model.addAttribute("match", match);
00110
00111     if (resultRequest.getGoals() == null) {
00112         resultRequest.setGoals(new ArrayList<>());
00113     }
00114
00115     try {
00116         matchService.registerResult(id, resultRequest);
00117         redirectAttributes.addFlashAttribute("successMessage", "Result saved successfully.");
00118         return "redirect:/matches/" + id + "/score";
00119     } catch (IllegalArgumentException e) {
00120         model.addAttribute("errorMessage", e.getMessage());
00121         return "enter-match-score";
00122     }
00123 }
00124
00125 private MatchResultRequestDTO toResultRequest(List<MatchGoal> goals) {
00126     MatchResultRequestDTO resultRequest = new MatchResultRequestDTO();
00127     List<MatchResultRequestDTO.GoalDTO> goalDTOs = new ArrayList<>();
00128
00129     for (MatchGoal goal : goals) {
00130         MatchResultRequestDTO.GoalDTO goalDTO = new MatchResultRequestDTO.GoalDTO();
00131         goalDTO.setTeamId(goal.getTeam().getId());
00132         goalDTO.setMinute((int) goal.getMinute());
00133         goalDTO.setStoppageMinute(
00134             goal.getStoppageMinute() == null ? null : (int) goal.getStoppageMinute()
00135         );
00136         goalDTOs.add(goalDTO);
00137     }
00138
00139     resultRequest.setGoals(goalDTOs);
00140     return resultRequest;
00141 }
00142
00143 @PostMapping("/matches/{id}/results")
00144 public String saveMatchResult(@PathVariable Long id, @ModelAttribute MatchResultRequestDTO
requestDTO) {
00145     try {
00146         matchService.registerResult(id, requestDTO);
00147
00148         return "redirect:/admin/matches";
00149     } catch (IllegalArgumentException e) {
00150         return "redirect:/matches/" + id + "/results?error=true";
00151     }
00152 }
00153
00154 @GetMapping("/matches/upcoming")
00155 public String showUpcomingMatchesPage(Model model, HttpSession session) {
00156     Boolean isAdmin = (Boolean) session.getAttribute("isAdmin");
00157
00158     model.addAttribute("isAdmin", isAdmin != null && isAdmin);
00159     model.addAttribute("matches", matchService.getUpcomingMatches());
00160     return "upcoming-matches";
00161 }
00162 }

```

9.36 Football_Manager/src/main/java/com/example/football_manager/controller/PlayerController.java File Reference

```

import com.example.football_manager.dto.PlayerBulkRequestDTO;
import com.example.football_manager.dto.PlayerRequestDTO;
import com.example.football_manager.model.Player;
import com.example.football_manager.service.PlayerService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;

```

```
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
```

Classes

- class [com.example.football_manager.controller.PlayerController](#)

Packages

- package [com.example.football_manager.controller](#)

9.37 PlayerController.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.controller;
00002
00003 import com.example.football_manager.dto.PlayerBulkRequestDTO;
00004 import com.example.football_manager.dto.PlayerRequestDTO;
00005 import com.example.football_manager.model.Player;
00006 import com.example.football_manager.service.PlayerService;
00007 import io.swagger.v3.oas.annotations.Operation;
00008 import io.swagger.v3.oas.annotations.media.Content;
00009 import io.swagger.v3.oas.annotations.media.Schema;
00010 import io.swagger.v3.oas.annotations.responses.ApiResponse;
00011 import io.swagger.v3.oas.annotations.responses.ApiResponses;
00012 import io.swagger.v3.oas.annotations.tags.Tag;
00013 import jakarta.validation.Valid;
00014 import org.springframework.http.HttpStatus;
00015 import org.springframework.http.ResponseEntity;
00016 import org.springframework.web.bind.annotation.*;
00017
00018 import java.util.List;
00019
00020 @RestController
00021 @RequestMapping("/api/teams/{teamId}/players")
00022 @Tag(name = "Players")
00023 public class PlayerController {
00024
00025     private final PlayerService playerService;
00026
00027     public PlayerController(PlayerService playerService) {
00028         this.playerService = playerService;
00029     }
00030
00031     @PostMapping
00032     @Operation(
00033         summary = "Create a player for a team",
00034         description = "Adds a new player to the given team squad."
00035     )
00036     @ApiResponses({
00037         @ApiResponse(responseCode = "201", description = "Player created"),
00038         @ApiResponse(responseCode = "400", description = "Validation or business rule error")
00039     })
00040     public ResponseEntity<?> createPlayer(
00041         @PathVariable Long teamId,
00042         @Valid @RequestBody PlayerRequestDTO dto) {
00043         try {
00044             Player createdPlayer = playerService.createPlayer(teamId, dto);
00045             return new ResponseEntity<>(createdPlayer, HttpStatus.CREATED);
00046         } catch (IllegalArgumentException ex) {
00047             return ResponseEntity.badRequest().body(ex.getMessage());
00048         }
00049     }
00050
00051     @PostMapping("/bulk")
00052     @Operation(
00053         summary = "Create many players for a team",
00054         description = "Adds multiple players to the given team squad using a single JSON payload."
00055     )
```



```

00056     @ApiResponses({
00057         @ApiResponse(responseCode = "201", description = "Players created"),
00058         @ApiResponse(responseCode = "400", description = "Validation or business rule error")
00059     })
00060     public ResponseEntity<?> createPlayers(
00061         @PathVariable Long teamId,
00062         @Valid @RequestBody PlayerBulkRequestDTO dto) {
00063         try {
00064             List<Player> createdPlayers = playerService.createPlayers(teamId, dto.getPlayers());
00065             return new ResponseEntity<>(createdPlayers, HttpStatus.CREATED);
00066         } catch (IllegalArgumentException ex) {
00067             return ResponseEntity.badRequest().body(ex.getMessage());
00068         }
00069     }
00070
00071     @GetMapping
00072     @Operation(
00073         summary = "List players of a team",
00074         description = "Returns the squad players for the given team ordered by number."
00075     )
00076     @ApiResponses({
00077         @ApiResponse(responseCode = "200", description = "Player list"),
00078         @ApiResponse(responseCode = "400", description = "Validation or business rule error")
00079     })
00080     public ResponseEntity<?> getPlayersByTeamId(@PathVariable Long teamId) {
00081         try {
00082             List<Player> players = playerService.getPlayersByTeamId(teamId);
00083             return ResponseEntity.ok(players);
00084         } catch (IllegalArgumentException ex) {
00085             return ResponseEntity.badRequest().body(ex.getMessage());
00086         }
00087     }
00088
00089     @PutMapping("/{playerId}")
00090     @Operation(
00091         summary = "Update a player of a team",
00092         description = "Updates the details of a player that belongs to the given team."
00093     )
00094     @ApiResponses({
00095         @ApiResponse(responseCode = "200", description = "Player updated"),
00096         @ApiResponse(responseCode = "400", description = "Validation or business rule error")
00097     })
00098     public ResponseEntity<?> updatePlayer(
00099         @PathVariable Long teamId,
00100         @PathVariable Long playerId,
00101         @Valid @RequestBody PlayerRequestDTO dto) {
00102         try {
00103             Player updatedPlayer = playerService.updatePlayer(teamId, playerId, dto);
00104             return ResponseEntity.ok(updatedPlayer);
00105         } catch (IllegalArgumentException ex) {
00106             return ResponseEntity.badRequest().body(ex.getMessage());
00107         }
00108     }
00109
00110     @DeleteMapping("/{playerId}")
00111     @Operation(
00112         summary = "Delete a player from a team",
00113         description = "Deletes a player that belongs to the given team."
00114     )
00115     @ApiResponses({
00116         @ApiResponse(responseCode = "204", description = "Player deleted"),
00117         @ApiResponse(responseCode = "400", description = "Validation or business rule error")
00118     })
00119     public ResponseEntity<?> deletePlayer(@PathVariable Long teamId, @PathVariable Long playerId) {
00120         try {
00121             playerService.deletePlayer(teamId, playerId);
00122             return ResponseEntity.noContent().build();
00123         } catch (IllegalArgumentException ex) {
00124             return ResponseEntity.badRequest().body(ex.getMessage());
00125         }
00126     }
00127 }

```

9.38 Football_Manager/src/main/java/com/example/football_manager/controller/ProfileViewController.java File Reference

```

import com.example.football_manager.dto.ProfileUpdatedDTO;
import com.example.football_manager.model.User;
import com.example.football_manager.service.UserService;
import jakarta.servlet.http.HttpSession;
import jakarta.validation.Valid;

```

```
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
import java.util.Set;
```

Classes

- class [com.example.football_manager.controller.ProfileViewController](#)

Packages

- package [com.example.football_manager.controller](#)

9.39 ProfileViewController.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.controller;
00002
00003 import com.example.football_manager.dto.ProfileUpdateDTO;
00004 import com.example.football_manager.model.User;
00005 import com.example.football_manager.service.UserService;
00006 import jakarta.servlet.http.HttpSession;
00007 import jakarta.validation.Valid;
00008 import org.springframework.stereotype.Controller;
00009 import org.springframework.ui.Model;
00010 import org.springframework.validation.BindingResult;
00011 import org.springframework.web.bind.annotation.GetMapping;
00012 import org.springframework.web.bind.annotation.ModelAttribute;
00013 import org.springframework.web.bind.annotation.PostMapping;
00014
00015 import java.util.Set;
00016
00017 @Controller
00018 public class ProfileViewController {
00019
00020     private final UserService userService;
00021
00022     public ProfileViewController(UserService userService) {
00023         this.userService = userService;
00024     }
00025
00026     @GetMapping("/profile")
00027     public String profilePage(Model model, HttpSession session) {
00028         Long userId = (Long) session.getAttribute("userId");
00029         if (userId == null) {
00030             return "redirect:/login";
00031         }
00032
00033         User user;
00034         try {
00035             user = userService.getUserProfile(userId);
00036         } catch (IllegalArgumentException ex) {
00037             session.invalidate();
00038             return "redirect:/login";
00039         }
00040
00041         populateProfileModel(model, user);
00042         return "profile";
00043     }
00044
00045     @PostMapping("/profile")
00046     public String updateProfile(@Valid @ModelAttribute("profileForm") ProfileUpdateDTO profileForm,
00047                                BindingResult bindingResult,
00048                                Model model,
00049                                HttpSession session) {
00050         Long userId = (Long) session.getAttribute("userId");
00051         if (userId == null) {
00052             return "redirect:/login";
00053         }
00054
00055         User currentUser;
00056         try {
```

```
00057         currentUser = userService.getUserProfile(userId);
00058     } catch (IllegalArgumentException ex) {
00059         session.invalidate();
00060         return "redirect:/login";
00061     }
00062
00063     if (bindingResult.hasErrors()) {
00064         populateProfileModel(model, currentUser);
00065         return "profile";
00066     }
00067
00068     try {
00069         User updatedUser = userService.updateProfile(
00070             userId,
00071             profileForm.getUsername(),
00072             profileForm.getEmail(),
00073             profileForm.getCurrentPassword(),
00074             profileForm.getNewPassword(),
00075             profileForm.getConfirmNewPassword()
00076         );
00077
00078         ProfileUpdateDTO refreshedForm = new ProfileUpdateDTO();
00079         refreshedForm.setUsername(updatedUser.getUsername());
00080         refreshedForm.setEmail(updatedUser.getEmail());
00081         model.addAttribute("profileForm", refreshedForm);
00082         model.addAttribute("favouriteTeams", updatedUser.getFavouriteTeams() != null ?
updatedUser.getFavouriteTeams() : Set.of());
00083         model.addAttribute("successMessage", "Profile updated successfully");
00084         return "profile";
00085     } catch (IllegalArgumentException ex) {
00086         populateProfileModel(model, currentUser);
00087         model.addAttribute("errorMessage", ex.getMessage());
00088         return "profile";
00089     }
00090 }
00091
00092 private void populateProfileModel(Model model, User user) {
00093     if (!model.containsAttribute("profileForm")) {
00094         ProfileUpdateDTO profileForm = new ProfileUpdateDTO();
00095         profileForm.setUsername(user.getUsername());
00096         profileForm.setEmail(user.getEmail());
00097         model.addAttribute("profileForm", profileForm);
00098     }
00099
00100     model.addAttribute("favouriteTeams", user.getFavouriteTeams() != null ?
user.getFavouriteTeams() : Set.of());
00101 }
00102 }
00103
00104
00105
00106
00107
00108
```

9.40 Football_Manager/src/main/java/com/example/football_manager/controller/StandingsController.java File Reference

```
import com.example.football_manager.dto.StandingDTO;
import com.example.football_manager.repository.CompetitionRepository;
import com.example.football_manager.service.StandingsService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.server.ResponseStatusException;
import java.util.List;
```

Classes

- class [com.example.football_manager.controller.StandingsController](#)

Packages

- package [com.example.football_manager.controller](#)

9.41 StandingsController.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.controller;
00002
00003 import com.example.football_manager.dto.StandingDTO;
00004 import com.example.football_manager.repository.CompetitionRepository;
00005 import com.example.football_manager.service.StandingsService;
00006 import io.swagger.v3.oas.annotations.Operation;
00007 import io.swagger.v3.oas.annotations.responses.ApiResponse;
00008 import io.swagger.v3.oas.annotations.responses.ApiResponses;
00009 import io.swagger.v3.oas.annotations.tags.Tag;
00010 import org.springframework.http.HttpStatus;
00011 import org.springframework.http.ResponseEntity;
00012 import org.springframework.web.bind.annotation.GetMapping;
00013 import org.springframework.web.bind.annotation.RequestMapping;
00014 import org.springframework.web.bind.annotation.RequestParam;
00015 import org.springframework.web.bind.annotation.RestController;
00016 import org.springframework.web.server.ResponseStatusException;
00017
00018 import java.util.List;
00019
00020 @RestController
00021 @RequestMapping("/api/standings")
00022 @Tag(name = "Standings")
00023 public class StandingsController {
00024
00025     private final StandingsService standingsService;
00026     private final CompetitionRepository competitionRepository;
00027
00028     public StandingsController(StandingsService standingsService,
00029                               CompetitionRepository competitionRepository) {
00030         this.standingsService = standingsService;
00031         this.competitionRepository = competitionRepository;
00032     }
00033
00034     @GetMapping
00035     @Operation(
00036         summary = "Get standings by competition",
00037         description = "Returns the standings table for the specified competition."
00038     )
00039     @ApiResponses({
00040         @ApiResponse(responseCode = "200", description = "Standings list"),
00041         @ApiResponse(responseCode = "404", description = "Competition not found")
00042     })
00043     public ResponseEntity<List<StandingDTO> getStandings(@RequestParam Long competitionId) {
00044         if (!competitionRepository.existsById(competitionId)) {
00045             throw new ResponseStatusException(
00046                 HttpStatus.NOT_FOUND, "Competition not found with id: " + competitionId);
00047         }
00048
00049         return ResponseEntity.ok(standingsService.getStandingsForCompetition(competitionId));
00050     }
00051 }

```

9.42 Football_Manager/src/main/java/com/example/football_manager/controller/StandingsViewController.java File Reference

```

import com.example.football_manager.dto.StandingDTO;
import com.example.football_manager.model.Competition;
import com.example.football_manager.service.StandingsService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;

```

```
import java.util.List;
```

Classes

- class [com.example.football_manager.controller.StandingsViewController](#)

Packages

- package [com.example.football_manager.controller](#)

9.43 StandingsViewController.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.controller;
00002
00003 import com.example.football_manager.dto.StandingDTO;
00004 import com.example.football_manager.model.Competition;
00005 import com.example.football_manager.service.StandingsService;
00006 import org.springframework.stereotype.Controller;
00007 import org.springframework.ui.Model;
00008 import org.springframework.web.bind.annotation.GetMapping;
00009 import org.springframework.web.bind.annotation.RequestParam;
00010
00011 import java.util.List;
00012
00013 @Controller
00014 public class StandingsViewController {
00015     private final StandingsService standingsService;
00016
00017     public StandingsViewController(StandingsService standingsService) {
00018         this.standingsService = standingsService;
00019     }
00020
00021     @GetMapping("/standings")
00022     public String standingsPage(@RequestParam(required = false) Long competitionId,
00023                               Model model) {
00024
00025         List<Competition> competitions = standingsService.getCompetitions();
00026
00027         Long selectedCompetitionId = null;
00028         List<StandingDTO> standings = List.of();
00029
00030         if (!competitions.isEmpty()) {
00031             // Fall back to the first available competition when no valid
00032             // selection is provided. Keeps the view stable on first load and
00033             // against unknown query values.
00034             boolean requestedIsValid = competitionId != null
00035                                     && competitions.stream().anyMatch(c -> c.getId().equals(competitionId));
00036             selectedCompetitionId = requestedIsValid ? competitionId : competitions.get(0).getId();
00037             standings = standingsService.getStandingsForCompetition(selectedCompetitionId);
00038         }
00039
00040         model.addAttribute("standings", standings);
00041         model.addAttribute("competitions", competitions);
00042         model.addAttribute("selectedCompetitionId", selectedCompetitionId);
00043         return "standings";
00044     }
00045 }
00046 }
```

9.44 Football_Manager/src/main/java/com/example/football_↵ manager/controller/TeamController.java File Reference

```
import com.example.football_manager.dto.TeamRequestDTO;
import com.example.football_manager.model.Team;
import com.example.football_manager.service.TeamService;
import jakarta.validation.Valid;
import java.util.List;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.media.Content;
import io.swagger.v3.oas.annotations.media.Schema;
```

```

import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

```

Classes

- class [com.example.football_manager.controller.TeamController](#)

Packages

- package [com.example.football_manager.controller](#)

9.45 TeamController.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.controller;
00002
00003 import com.example.football_manager.dto.TeamRequestDTO;
00004 import com.example.football_manager.model.Team;
00005 import com.example.football_manager.service.TeamService;
00006 import jakarta.validation.Valid;
00007
00008 import java.util.List;
00009
00010 import io.swagger.v3.oas.annotations.Operation;
00011 import io.swagger.v3.oas.annotations.media.Content;
00012 import io.swagger.v3.oas.annotations.media.Schema;
00013 import io.swagger.v3.oas.annotations.responses.ApiResponse;
00014 import io.swagger.v3.oas.annotations.responses.ApiResponses;
00015 import io.swagger.v3.oas.annotations.tags.Tag;
00016 import org.springframework.http.HttpStatus;
00017 import org.springframework.http.ResponseEntity;
00018 import org.springframework.web.bind.annotation.*;
00019
00020 @RestController
00021 @RequestMapping("/api/teams")
00022 @Tag(name = "Teams")
00023 public class TeamController {
00024
00025     private final TeamService teamService;
00026
00027     public TeamController(TeamService teamService) {
00028         this.teamService = teamService;
00029     }
00030
00031     @PostMapping
00032     @Operation(
00033         summary = "Create a team",
00034         description = "Creates a new team linked to a country.",
00035         requestBody = @io.swagger.v3.oas.annotations.parameters.RequestBody(
00036             required = true,
00037             description = "Team details",
00038             content = @Content(schema = @Schema(implementation = TeamRequestDTO.class))
00039         )
00040     )
00041     @ApiResponses({
00042         @ApiResponse(responseCode = "201", description = "Team created"),
00043         @ApiResponse(responseCode = "400", description = "Validation error")
00044     })
00045     public ResponseEntity<Team> createTeam(@Valid @RequestBody TeamRequestDTO dto) {
00046         Team createdTeam = teamService.createTeam(dto);
00047         return new ResponseEntity<>(createdTeam, HttpStatus.CREATED);
00048     }
00049
00050     @PutMapping("/{id}")
00051     @Operation(
00052         summary = "Update a team",
00053         description = "Updates the name, logo, or country of an existing team.",
00054         requestBody = @io.swagger.v3.oas.annotations.parameters.RequestBody(
00055             required = true,
00056             description = "Updated team details",
00057             content = @Content(schema = @Schema(implementation = TeamRequestDTO.class))
00058         )

```

```
00059     )
00060     @ApiResponse({
00061         @ApiResponse(responseCode = "200", description = "Team updated"),
00062         @ApiResponse(responseCode = "400", description = "Validation error"),
00063         @ApiResponse(responseCode = "404", description = "Team not found")
00064     })
00065     public ResponseEntity<Team> updateTeam(@PathVariable Long id,
00066                                           @Valid @RequestBody TeamRequestDTO dto) {
00067         Team updatedTeam = teamService.updateTeam(id, dto);
00068         return ResponseEntity.ok(updatedTeam);
00069     }
00070
00071     @DeleteMapping("/{id}")
00072     @Operation(
00073         summary = "Delete a team",
00074         description = "Deletes a team by its identifier."
00075     )
00076     @ApiResponse({
00077         @ApiResponse(responseCode = "204", description = "Team deleted"),
00078         @ApiResponse(responseCode = "404", description = "Team not found")
00079     })
00080     public ResponseEntity<Void> deleteTeam(@PathVariable Long id) {
00081         teamService.deleteTeam(id);
00082         return ResponseEntity.noContent().build();
00083     }
00084
00085     @GetMapping("/{id}")
00086     @Operation(
00087         summary = "Get team by id",
00088         description = "Returns a team with its persisted details."
00089     )
00090     @ApiResponse({
00091         @ApiResponse(responseCode = "200", description = "Team found"),
00092         @ApiResponse(responseCode = "404", description = "Team not found")
00093     })
00094     public ResponseEntity<Team> getTeamById(@PathVariable Long id) {
00095         return ResponseEntity.of(teamService.getTeamById(id));
00096     }
00097
00098     @GetMapping("/search")
00099     @Operation(
00100         summary = "Search teams by name",
00101         description = "Returns teams whose name contains the provided text (case-insensitive)."
00102     )
00103     @ApiResponse({
00104         @ApiResponse(responseCode = "200", description = "Teams matching query")
00105     })
00106     public ResponseEntity<List<Team>> getTeamsContainingName(@RequestParam String name) {
00107         return ResponseEntity.ok(teamService.getTeamsContainingName(name));
00108     }
00109
00110     @GetMapping
00111     @Operation(
00112         summary = "List all teams",
00113         description = "Returns all teams currently registered in the system."
00114     )
00115     @ApiResponse({
00116         @ApiResponse(responseCode = "200", description = "Team list")
00117     })
00118     public ResponseEntity<List<Team>> getAllTeams() {
00119         List<Team> teams = teamService.getAllTeams();
00120         return ResponseEntity.ok(teams);
00121     }
00122 }
```

9.46 Football_Manager/src/main/java/com/example/football_manager/controller/TeamViewController.java File Reference

```
import com.example.football_manager.dto.TeamRequestDTO;
import com.example.football_manager.model.Team;
import com.example.football_manager.service.CountryService;
import com.example.football_manager.service.MatchService;
import com.example.football_manager.service.PlayerService;
import com.example.football_manager.service.TeamService;
import com.example.football_manager.service.UserService;
import jakarta.servlet.http.HttpSession;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
```

```
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import java.util.Set;
```

Classes

- class [com.example.football_manager.controller.TeamViewController](#)

Packages

- package [com.example.football_manager.controller](#)

9.47 TeamViewController.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.controller;
00002
00003 import com.example.football_manager.dto.TeamRequestDTO;
00004 import com.example.football_manager.model.Team;
00005 import com.example.football_manager.service.CountryService;
00006 import com.example.football_manager.service.MatchService;
00007 import com.example.football_manager.service.PlayerService;
00008 import com.example.football_manager.service.TeamService;
00009 import com.example.football_manager.service.UserService;
00010 import jakarta.servlet.http.HttpSession;
00011 import org.springframework.beans.factory.annotation.Autowired;
00012 import org.springframework.stereotype.Controller;
00013 import org.springframework.ui.Model;
00014 import org.springframework.web.bind.annotation.GetMapping;
00015 import org.springframework.web.bind.annotation.ModelAttribute;
00016 import org.springframework.web.bind.annotation.PathVariable;
00017 import org.springframework.web.bind.annotation.PostMapping;
00018
00019 import java.util.Set;
00020
00021 @Controller
00022 public class TeamViewController {
00023
00024     @Autowired
00025     private TeamService teamService;
00026
00027     @Autowired
00028     private CountryService countryService;
00029
00030     @Autowired
00031     private UserService userService;
00032
00033     @Autowired
00034     private MatchService matchService;
00035
00036     @Autowired
00037     private PlayerService playerService;
00038
00039     @GetMapping("/teams")
00040     public String teamsPage(Model model, HttpSession session) {
00041         Boolean isAdmin = (Boolean) session.getAttribute("isAdmin");
00042         Long userId = (Long) session.getAttribute("userId");
00043
00044         Set<Long> favouriteTeamIds = Set.of();
00045
00046         try {
00047             favouriteTeamIds = userService.getFavouriteTeamIdsByUserId(userId);
00048         } catch (IllegalArgumentException ignored) {
00049         }
00050
00051         model.addAttribute("isAdmin", isAdmin != null && isAdmin);
00052         model.addAttribute("teams", teamService.getAllTeams());
00053         model.addAttribute("favouriteTeamIds", favouriteTeamIds);
00054         model.addAttribute("favouriteUpcomingMatches",
00055             matchService.getFutureMatchesByFavouriteTeamIds(favouriteTeamIds));
00056
00056         return "teams";
00057     }
}
```



```

00058     @GetMapping("/teams/{id}")
00059     public String teamDetailsPage(@PathVariable Long id, Model model, HttpSession session) {
00060         Boolean isAdmin = (Boolean) session.getAttribute("isAdmin");
00061
00062         Team team = teamService.getTeamById(id)
00063             .orElseThrow(() -> new RuntimeException("Team not found"));
00064
00065         model.addAttribute("isAdmin", isAdmin != null && isAdmin);
00066         model.addAttribute("team", team);
00067         model.addAttribute("players", playerService.getPlayersByTeamId(id));
00068         model.addAttribute("recentMatches", matchService.getRecentMatchesByTeamId(id));
00069         model.addAttribute("futureMatches", matchService.getFutureMatchesByTeamId(id));
00070
00071         return "team-details";
00072     }
00073
00074     @GetMapping("/teams/add")
00075     public String showAddTeamForm(Model model) {
00076         model.addAttribute("teamRequest", new TeamRequestDTO());
00077         model.addAttribute("countries", countryService.getAllCountries());
00078
00079         return "add-team";
00080     }
00081
00082     @PostMapping("/teams/add")
00083     public String createTeam(@ModelAttribute TeamRequestDTO teamDTO) {
00084         teamService.createTeam(teamDTO);
00085
00086         return "redirect:/teams";
00087     }
00088
00089     @GetMapping("/teams/edit/{id}")
00090     public String showEditTeamForm(@PathVariable Long id, Model model) {
00091         Team existingTeam = teamService.getTeamById(id)
00092             .orElseThrow(() -> new RuntimeException("Team not found"));
00093
00094         TeamRequestDTO teamDTO = new TeamRequestDTO();
00095         teamDTO.setName(existingTeam.getName());
00096         teamDTO.setLogoUrl(existingTeam.getLogoUrl());
00097
00098         if (existingTeam.getCountry() != null) {
00099             teamDTO.setCountryId(existingTeam.getCountry().getId());
00100         }
00101
00102         model.addAttribute("teamRequest", teamDTO);
00103         model.addAttribute("teamId", id);
00104         model.addAttribute("countries", countryService.getAllCountries());
00105
00106         return "edit-team";
00107     }
00108
00109     @PostMapping("/teams/edit/{id}")
00110     public String updateTeam(@PathVariable Long id, @ModelAttribute TeamRequestDTO teamDTO) {
00111         teamService.updateTeam(id, teamDTO);
00112
00113         return "redirect:/teams";
00114     }
00115
00116     @PostMapping("/teams/delete/{id}")
00117     public String deleteTeam(@PathVariable Long id) {
00118         teamService.deleteTeam(id);
00119
00120         return "redirect:/teams";
00121     }
00122 }
00123

```

9.48 Football_Manager/src/main/java/com/example/football_manager/dto/AuthResponseDTO.java File Reference

```
import io.swagger.v3.oas.annotations.media.Schema;
```

Classes

- class [com.example.football_manager.dto.AuthResponseDTO](#)

Packages

- package [com.example.football_manager.dto](#)

9.49 AuthResponseDTO.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.dto;
00002
00003 import io.swagger.v3.oas.annotations.media.Schema;
00004
00005 public class AuthResponseDTO {
00006
00007     @Schema(description = "User identifier", example = "101")
00008     private Long id;
00009     @Schema(description = "Username", example = "user123")
00010     private String username;
00011     @Schema(description = "Email address", example = "user123@example.com")
00012     private String email;
00013     @Schema(description = "Whether the user is an administrator", example = "false")
00014     private Boolean isAdmin;
00015     @Schema(description = "Status message")
00016     private String message;
00017
00018     public AuthResponseDTO(Long id, String username, String email, Boolean isAdmin, String message) {
00019         this.id = id;
00020         this.username = username;
00021         this.email = email;
00022         this.isAdmin = isAdmin;
00023         this.message = message;
00024     }
00025
00026     public Long getId() {
00027         return id;
00028     }
00029
00030     public String getUsername() {
00031         return username;
00032     }
00033
00034     public String getEmail() {
00035         return email;
00036     }
00037
00038     public Boolean getIsAdmin() {
00039         return isAdmin;
00040     }
00041
00042     public String getMessage() {
00043         return message;
00044     }
00045 }
00046
```

9.50 Football_Manager/src/main/java/com/example/football_↵ manager/dto/CompetitionRequestDTO.java File Reference

```
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import io.swagger.v3.oas.annotations.media.Schema;
```

Classes

- class [com.example.football_manager.dto.CompetitionRequestDTO](#)

Packages

- package [com.example.football_manager.dto](#)

9.51 CompetitionRequestDTO.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.dto;
00002
00003 import jakarta.validation.constraints.NotBlank;
00004 import jakarta.validation.constraints.Size;
00005 import io.swagger.v3.oas.annotations.media.Schema;
00006
00007 public class CompetitionRequestDTO {
00008
00009     @NotBlank(message = "Competition name is required")
00010     @Size(max = 50, message = "Competition name must be at most 50 characters")
00011     @Schema(description = "Competition name", example = "LaLiga")
00012     private String name;
00013
00014     public CompetitionRequestDTO() {
00015     }
00016
00017     public CompetitionRequestDTO(String name) {
00018         this.name = name;
00019     }
00020
00021     public String getName() {
00022         return name;
00023     }
00024
00025     public void setName(String name) {
00026         this.name = name;
00027     }
00028 }

```

9.52 Football_Manager/src/main/java/com/example/football_manager/dto/CountryRequestDTO.java File Reference

```

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import io.swagger.v3.oas.annotations.media.Schema;

```

Classes

- class [com.example.football_manager.dto.CountryRequestDTO](#)

Packages

- package [com.example.football_manager.dto](#)

9.53 CountryRequestDTO.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.dto;
00002
00003 import jakarta.validation.constraints.NotBlank;
00004 import jakarta.validation.constraints.Size;
00005 import io.swagger.v3.oas.annotations.media.Schema;
00006
00007 public class CountryRequestDTO {
00008
00009     @NotBlank(message = "Country name is required")
00010     @Size(max = 100, message = "Country name must be at most 100 characters")
00011     @Schema(description = "Country name", example = "Spain")
00012     private String name;
00013
00014     public CountryRequestDTO() {
00015     }
00016
00017     public CountryRequestDTO(String name) {
00018         this.name = name;
00019     }
00020
00021     public String getName() {
00022         return name;

```

```

00023     }
00024
00025     public void setName(String name) {
00026         this.name = name;
00027     }
00028 }

```

9.54 Football_Manager/src/main/java/com/example/football_manager/dto/LoginDTO.java File Reference ↩

```

import jakarta.validation.constraints.NotBlank;
import io.swagger.v3.oas.annotations.media.Schema;

```

Classes

- class [com.example.football_manager.dto.LoginDTO](#)

Packages

- package [com.example.football_manager.dto](#)

9.55 LoginDTO.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.dto;
00002
00003 import jakarta.validation.constraints.NotBlank;
00004 import io.swagger.v3.oas.annotations.media.Schema;
00005
00006 public class LoginDTO {
00007
00008     @NotBlank(message = "Username is required")
00009     @Schema(description = "Username", example = "user123")
00010     private String username;
00011
00012     @NotBlank(message = "Password is required")
00013     @Schema(description = "User password", example = "P@ssw0rd123")
00014     private String password;
00015
00016     public String getUsername() {
00017         return username;
00018     }
00019
00020     public void setUsername(String username) {
00021         this.username = username;
00022     }
00023
00024     public String getPassword() {
00025         return password;
00026     }
00027
00028     public void setPassword(String password) {
00029         this.password = password;
00030     }
00031 }
00032

```

9.56 Football_Manager/src/main/java/com/example/football_manager/dto/MatchRequestDTO.java File Reference ↩

```

import java.time.LocalDateTime;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import io.swagger.v3.oas.annotations.media.Schema;

```

Classes

- class [com.example.football_manager.dto.MatchRequestDTO](#)
- enum [com.example.football_manager.dto.MatchRequestDTO.MatchStatus](#)

Packages

- package [com.example.football_manager.dto](#)

9.57 MatchRequestDTO.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.dto;
00002
00003 import java.time.LocalDateTime;
00004 import jakarta.validation.constraints.Min;
00005 import jakarta.validation.constraints.NotBlank;
00006 import jakarta.validation.constraints.NotNull;
00007 import io.swagger.v3.oas.annotations.media.Schema;
00008
00009 public class MatchRequestDTO {
00010
00011     public enum MatchStatus {
00012         SCHEDULED,
00013         IN_PROGRESS,
00014         FINISHED,
00015         CANCELLED
00016     }
00017
00018     @Schema(description = "Home team identifier", example = "12")
00019     private Long homeTeamId;
00020     @Schema(description = "Away team identifier", example = "45")
00021     private Long awayTeamId;
00022     @Schema(description = "Competition identifier", example = "3")
00023     private Long competitionId;
00024     @Schema(description = "Kickoff time (local server time)", example = "2026-04-28T18:30:00")
00025     private LocalDateTime kickoffTime;
00026     @Schema(description = "Match venue or stadium name", example = "Anoeta Stadium")
00027     private String venue;
00028
00029     @Schema(description = "Match status", example = "SCHEDULED")
00030     private MatchStatus status;
00031
00032     @Schema(description = "Home team score", example = "2")
00033     private Integer homeScore;
00034     @Schema(description = "Away team score", example = "1")
00035     private Integer awayScore;
00036
00037
00038     public Long getHomeTeamId() { return homeTeamId; }
00039     public void setHomeTeamId(Long homeTeamId) { this.homeTeamId = homeTeamId; }
00040
00041     public Long getAwayTeamId() { return awayTeamId; }
00042     public void setAwayTeamId(Long awayTeamId) { this.awayTeamId = awayTeamId; }
00043
00044     public Long getCompetitionId() { return competitionId; }
00045     public void setCompetitionId(Long competitionId) { this.competitionId = competitionId; }
00046
00047     public LocalDateTime getKickoffTime() { return kickoffTime; }
00048     public void setKickoffTime(LocalDateTime kickoffTime) { this.kickoffTime = kickoffTime; }
00049
00050     public String getVenue() { return venue; }
00051     public void setVenue(String venue) { this.venue = venue; }
00052
00053     public MatchStatus getStatus() { return status; }
00054     public void setStatus(MatchStatus status) { this.status = status; }
00055
00056     public Integer getHomeScore() { return homeScore; }
00057     public void setHomeScore(Integer homeScore) { this.homeScore = homeScore; }
00058
00059     public Integer getAwayScore() { return awayScore; }
00060     public void setAwayScore(Integer awayScore) { this.awayScore = awayScore; }
00061 }

```

9.58 Football_Manager/src/main/java/com/example/football_manager/dto/MatchResultDTO.java File Reference

```
import java.time.OffsetDateTime;
import io.swagger.v3.oas.annotations.media.Schema;
```

Packages

- package [com.example.football_manager.dto](#)

Functions

- record [com.example.football_manager.dto.MatchResultDTO](#) (@Schema(description="Match identifier", example="220") Long matchId, @Schema(description="Home team name", example="Real Sociedad") String homeTeamName, @Schema(description="Away team name", example="Athletic Club") String awayTeamName, @Schema(description="Home team score", example="2") short homeScore, @Schema(description="Away team score", example="1") short awayScore, @Schema(description="Kickoff time (UTC)", example="2026-04-28T18:30:00Z") OffsetDateTime kickoffTime)

9.59 MatchResultDTO.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.dto;
00002
00003 import java.time.OffsetDateTime;
00004 import io.swagger.v3.oas.annotations.media.Schema;
00005
00006 public record MatchResultDTO(
00007     @Schema(description = "Match identifier", example = "220") Long matchId,
00008     @Schema(description = "Home team name", example = "Real Sociedad") String homeTeamName,
00009     @Schema(description = "Away team name", example = "Athletic Club") String awayTeamName,
00010     @Schema(description = "Home team score", example = "2") short homeScore,
00011     @Schema(description = "Away team score", example = "1") short awayScore,
00012     @Schema(description = "Kickoff time (UTC)", example = "2026-04-28T18:30:00Z") OffsetDateTime
    kickoffTime
00013 ) {
00014 }
```

9.60 Football_Manager/src/main/java/com/example/football_manager/dto/MatchResultRequestDTO.java File Reference

```
import jakarta.validation.Valid;
import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotNull;
import io.swagger.v3.oas.annotations.media.Schema;
import java.util.List;
```

Classes

- class [com.example.football_manager.dto.MatchResultRequestDTO](#)
- class [com.example.football_manager.dto.MatchResultRequestDTO.GoalDTO](#)

Packages

- package [com.example.football_manager.dto](#)

9.61 MatchResultRequestDTO.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.dto;
00002
00003 import jakarta.validation.Valid;
00004 import jakarta.validation.constraints.Max;
00005 import jakarta.validation.constraints.Min;
00006 import jakarta.validation.constraints.NotNull;
00007 import io.swagger.v3.oas.annotations.media.Schema;
00008
00009 import java.util.List;
00010
00011 public class MatchResultRequestDTO {
00012
00013     @NotNull
00014     @Schema(description = "List of goals scored during the match")
00015     private List<@Valid GoalDTO> goals;
00016
00017     public List<GoalDTO> getGoals() {
00018         return goals;
00019     }
00020
00021     public void setGoals(List<GoalDTO> goals) {
00022         this.goals = goals;
00023     }
00024
00025     public static class GoalDTO {
00026
00027         @NotNull
00028         @Schema(description = "Identifier of the scoring team", example = "12")
00029         private Long teamId;
00030
00031         @NotNull
00032         @Min(1)
00033         @Max(120)
00034         @Schema(description = "Minute of the goal (1-120)", example = "67")
00035         private Integer minute;
00036
00037         @Min(0)
00038         @Max(30)
00039         @Schema(description = "Stoppage minute (optional)", example = "2")
00040         private Integer stoppageMinute;
00041
00042         public Long getTeamId() {
00043             return teamId;
00044         }
00045
00046         public void setTeamId(Long teamId) {
00047             this.teamId = teamId;
00048         }
00049
00050         public Integer getMinute() {
00051             return minute;
00052         }
00053
00054         public void setMinute(Integer minute) {
00055             this.minute = minute;
00056         }
00057
00058         public Integer getStoppageMinute() {
00059             return stoppageMinute;
00060         }
00061
00062         public void setStoppageMinute(Integer stoppageMinute) {
00063             this.stoppageMinute = stoppageMinute;
00064         }
00065     }
00066 }

```

9.62 Football_Manager/src/main/java/com/example/football_↵ manager/dto/PlayerBulkRequestDTO.java File Reference

```

import jakarta.validation.Valid;
import jakarta.validation.constraints.NotEmpty;
import java.util.List;

```

Classes

- class [com.example.football_manager.dto.PlayerBulkRequestDTO](#)

Packages

- package [com.example.football_manager.dto](#)

9.63 PlayerBulkRequestDTO.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.dto;
00002
00003 import jakarta.validation.Valid;
00004 import jakarta.validation.constraints.NotEmpty;
00005
00006 import java.util.List;
00007
00008 public class PlayerBulkRequestDTO {
00009     @NotEmpty(message = "Player list is required")
00010     private List<@Valid PlayerRequestDTO> players;
00011
00012     public PlayerBulkRequestDTO() {
00013     }
00014
00015     public PlayerBulkRequestDTO(List<PlayerRequestDTO> players) {
00016         this.players = players;
00017     }
00018
00019     public List<PlayerRequestDTO> getPlayers() {
00020         return players;
00021     }
00022
00023     public void setPlayers(List<PlayerRequestDTO> players) {
00024         this.players = players;
00025     }
00026 }
00027

```

9.64 Football_Manager/src/main/java/com/example/football_manager/dto/PlayerRequestDTO.java File Reference

```

import com.example.football_manager.model.PlayerPosition;
import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

```

Classes

- class [com.example.football_manager.dto.PlayerRequestDTO](#)

Packages

- package [com.example.football_manager.dto](#)

9.65 PlayerRequestDTO.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.dto;
00002
00003 import com.example.football_manager.model.PlayerPosition;
00004 import io.swagger.v3.oas.annotations.media.Schema;
00005 import jakarta.validation.constraints.Min;
00006 import jakarta.validation.constraints.NotBlank;

```



```

00007 import jakarta.validation.constraints.NotNull;
00008 import jakarta.validation.constraints.Size;
00009
00010 public class PlayerRequestDTO {
00011     @NotBlank(message = "Player name is required")
00012     @Size(max = 30, message = "Player name must be at most 30 characters")
00013     @Schema(description = "Player name", example = "Mikel Oyarzabal")
00014     private String name;
00015
00016     @NotNull(message = "Player number is required")
00017     @Min(value = 1, message = "Player number must be greater than 0")
00018     @Schema(description = "Squad number", example = "10")
00019     private Integer number;
00020
00021     @NotNull(message = "Player position is required")
00022     @Schema(description = "Player position", example = "FORWARD")
00023     private PlayerPosition position;
00024
00025     public PlayerRequestDTO() {
00026     }
00027
00028     public PlayerRequestDTO(String name, Integer number, PlayerPosition position) {
00029         this.name = name;
00030         this.number = number;
00031         this.position = position;
00032     }
00033
00034     public String getName() {
00035         return name;
00036     }
00037
00038     public void setName(String name) {
00039         this.name = name;
00040     }
00041
00042     public Integer getNumber() {
00043         return number;
00044     }
00045
00046     public void setNumber(Integer number) {
00047         this.number = number;
00048     }
00049
00050     public PlayerPosition getPosition() {
00051         return position;
00052     }
00053
00054     public void setPosition(PlayerPosition position) {
00055         this.position = position;
00056     }
00057 }
00058 }

```

9.66 Football_Manager/src/main/java/com/example/football_manager/dto/ProfileUpdatedTO.java File Reference

```

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.AssertTrue;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
import jakarta.validation.constraints.Size;
import io.swagger.v3.oas.annotations.media.Schema;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

```

Classes

- class [com.example.football_manager.dto.ProfileUpdateDTO](#)

Packages

- package [com.example.football_manager.dto](#)

9.67 ProfileUpdateDTO.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.dto;
00002
00003 import jakarta.validation.constraints.Email;
00004 import jakarta.validation.constraints.AssertTrue;
00005 import jakarta.validation.constraints.NotBlank;
00006 import jakarta.validation.constraints.Pattern;
00007 import jakarta.validation.constraints.Size;
00008 import io.swagger.v3.oas.annotations.media.Schema;
00009 import lombok.AllArgsConstructor;
00010 import lombok.Data;
00011 import lombok.NoArgsConstructor;
00012
00013 @Data
00014 @NoArgsConstructor
00015 @AllArgsConstructor
00016 public class ProfileUpdateDTO {
00017
00018     @NotBlank(message = "Username is required")
00019     @Size(max = 30, message = "Username must be at most 30 characters")
00020     @Schema(description = "New username", example = "john_doe")
00021     private String username;
00022
00023     @NotBlank(message = "Email is required")
00024     @Email(message = "Email must be valid")
00025     @Schema(description = "New email address", example = "john_doe@example.com")
00026     private String email;
00027
00028     @Schema(description = "Current password (required when changing password)", example =
"OldP@ssw0rd")
00029     private String currentPassword;
00030
00031     @Pattern(regexp = "^[^$.{}$]", message = "New password must be at least 8 characters")
00032     @Schema(description = "New password (min 8 characters)", example = "NewP@ssw0rd123")
00033     private String newPassword;
00034
00035     @Schema(description = "Confirmation of new password", example = "NewP@ssw0rd123")
00036     private String confirmNewPassword;
00037
00038     @AssertTrue(message = "New passwords do not match")
00039     public boolean isPasswordConfirmationValid() {
00040         if ((newPassword == null || newPassword.isBlank()) && (confirmNewPassword == null ||
confirmNewPassword.isBlank())) {
00041             return true;
00042         }
00043
00044         if (newPassword == null || confirmNewPassword == null) {
00045             return false;
00046         }
00047
00048         return newPassword.equals(confirmNewPassword);
00049     }
00050 }
00051
00052
00053
00054

```

9.68 Football_Manager/src/main/java/com/example/football_ manager/dto/RegisterDTO.java File Reference ↩

```

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import io.swagger.v3.oas.annotations.media.Schema;

```

Classes

- class [com.example.football_manager.dto.RegisterDTO](#)

Packages

- package [com.example.football_manager.dto](#)

9.69 RegisterDTO.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.dto;
00002
00003 import jakarta.validation.constraints.Email;
00004 import jakarta.validation.constraints.NotBlank;
00005 import jakarta.validation.constraints.Size;
00006 import io.swagger.v3.oas.annotations.media.Schema;
00007
00008 public class RegisterDTO {
00009
00010     @NotBlank(message = "Username is required")
00011     @Size(max = 30, message = "Username must be at most 30 characters")
00012     @Schema(description = "Username", example = "user123")
00013     private String username;
00014
00015     @NotBlank(message = "Email is required")
00016     @Email(message = "Invalid email format")
00017     @Schema(description = "Email address", example = "user123@example.com")
00018     private String email;
00019
00020     @NotBlank(message = "Password is required")
00021     @Size(min = 8, message = "Password must be at least 8 characters")
00022     @Schema(description = "Password (min 8 characters)", example = "P@ssw0rd123")
00023     private String password;
00024
00025     @Schema(description = "Whether the user should be registered as admin", example = "false")
00026     private Boolean isAdmin;
00027
00028     public String getUsername() { return username; }
00029     public void setUsername(String username) { this.username = username; }
00030
00031     public String getEmail() { return email; }
00032     public void setEmail(String email) { this.email = email; }
00033
00034     public String getPassword() { return password; }
00035     public void setPassword(String password) { this.password = password; }
00036
00037     public Boolean getIsAdmin() { return isAdmin; }
00038     public void setIsAdmin(Boolean isAdmin) { this.isAdmin = isAdmin; }
00039 }

```

9.70 Football_Manager/src/main/java/com/example/football_manager/dto/StandingDTO.java File Reference

Classes

- class [com.example.football_manager.dto.StandingDTO](#)
View-only DTO used to render a single row of the standings table.

Packages

- package [com.example.football_manager.dto](#)

9.71 StandingDTO.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.dto;
00002
00008 public class StandingDTO {
00009
00010     private String teamName;
00011     private String logoUrl;
00012     private int matchesPlayed;
00013     private int won;
00014     private int drawn;
00015     private int lost;
00016     private int goalsFor;
00017     private int goalsAgainst;
00018     private int goalDifference;
00019     private int points;
00020
00021     public StandingDTO() {
00022     }

```

```
00023
00024     public StandingDTO(String teamName, String logoUrl, int matchesPlayed, int won, int drawn,
00025                         int lost, int goalsFor, int goalsAgainst, int goalDifference, int points) {
00026         this.teamName = teamName;
00027         this.logoUrl = logoUrl;
00028         this.matchesPlayed = matchesPlayed;
00029         this.won = won;
00030         this.drawn = drawn;
00031         this.lost = lost;
00032         this.goalsFor = goalsFor;
00033         this.goalsAgainst = goalsAgainst;
00034         this.goalDifference = goalDifference;
00035         this.points = points;
00036     }
00037
00038     public String getTeamName() {
00039         return teamName;
00040     }
00041
00042     public void setTeamName(String teamName) {
00043         this.teamName = teamName;
00044     }
00045
00046     public String getLogoUrl() {
00047         return logoUrl;
00048     }
00049
00050     public void setLogoUrl(String logoUrl) {
00051         this.logoUrl = logoUrl;
00052     }
00053
00054     public int getMatchesPlayed() {
00055         return matchesPlayed;
00056     }
00057
00058     public void setMatchesPlayed(int matchesPlayed) {
00059         this.matchesPlayed = matchesPlayed;
00060     }
00061
00062     public int getWon() {
00063         return won;
00064     }
00065
00066     public void setWon(int won) {
00067         this.won = won;
00068     }
00069
00070     public int getDrawn() {
00071         return drawn;
00072     }
00073
00074     public void setDrawn(int drawn) {
00075         this.drawn = drawn;
00076     }
00077
00078     public int getLost() {
00079         return lost;
00080     }
00081
00082     public void setLost(int lost) {
00083         this.lost = lost;
00084     }
00085
00086     public int getGoalsFor() {
00087         return goalsFor;
00088     }
00089
00090     public void setGoalsFor(int goalsFor) {
00091         this.goalsFor = goalsFor;
00092     }
00093
00094     public int getGoalsAgainst() {
00095         return goalsAgainst;
00096     }
00097
00098     public void setGoalsAgainst(int goalsAgainst) {
00099         this.goalsAgainst = goalsAgainst;
00100     }
00101
00102     public int getGoalDifference() {
00103         return goalDifference;
00104     }
00105
00106     public void setGoalDifference(int goalDifference) {
00107         this.goalDifference = goalDifference;
00108     }
00109
```

```
00110     public int getPoints() {
00111         return points;
00112     }
00113
00114     public void setPoints(int points) {
00115         this.points = points;
00116     }
00117 }
```

9.72 Football_Manager/src/main/java/com/example/football_manager/dto/TeamRequestDTO.java File Reference

```
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import io.swagger.v3.oas.annotations.media.Schema;
```

Classes

- class [com.example.football_manager.dto.TeamRequestDTO](#)

Packages

- package [com.example.football_manager.dto](#)

9.73 TeamRequestDTO.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.dto;
00002
00003 import jakarta.validation.constraints.NotBlank;
00004 import jakarta.validation.constraints.NotNull;
00005 import jakarta.validation.constraints.Size;
00006 import io.swagger.v3.oas.annotations.media.Schema;
00007
00008 public class TeamRequestDTO {
00009
00010     @NotBlank(message = "Team name is required")
00011     @Size(max = 30, message = "Team name must be at most 30 characters")
00012     @Schema(description = "Team name", example = "Real Sociedad")
00013     private String name;
00014
00015     @NotBlank(message = "Logo URL is required")
00016     @Schema(description = "Public logo URL", example =
00017         "https://cdn.example.com/logos/real-sociedad.png")
00018     private String logoUrl;
00019
00020     @NotNull(message = "Country ID is required")
00021     @Schema(description = "Identifier of the country the team belongs to", example = "34")
00022     private Long countryId;
00023
00024     public TeamRequestDTO() {
00025     }
00026
00027     public TeamRequestDTO(String name, String logoUrl, Long countryId) {
00028         this.name = name;
00029         this.logoUrl = logoUrl;
00030         this.countryId = countryId;
00031     }
00032
00033     public String getName() {
00034         return name;
00035     }
00036
00037     public void setName(String name) {
00038         this.name = name;
00039     }
00040
00041     public String getLogoUrl() {
00042         return logoUrl;
00043     }
00044
00045     public void setLogoUrl(String logoUrl) {
```

```

00045         this.logoUrl = logoUrl;
00046     }
00047
00048     public Long getCountryId() {
00049         return countryId;
00050     }
00051
00052     public void setCountryId(Long countryId) {
00053         this.countryId = countryId;
00054     }
00055 }

```

9.74 Football_Manager/src/main/java/com/example/football_manager/↔ FootballManagerApplication.java File Reference

```

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

```

Classes

- class [com.example.football_manager.FootballManagerApplication](#)

Packages

- package [com.example.football_manager](#)

9.75 FootballManagerApplication.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager;
00002
00003 import org.springframework.boot.SpringApplication;
00004 import org.springframework.boot.autoconfigure.SpringBootApplication;
00005
00006 @SpringBootApplication
00007 public class FootballManagerApplication {
00008
00009     public static void main(String[] args) {
00010         SpringApplication.run(FootballManagerApplication.class, args);
00011     }
00012
00013 }

```

9.76 Football_Manager/src/main/java/com/example/football_↔ manager/model/Competition.java File Reference

```

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

```

Classes

- class [com.example.football_manager.model.Competition](#)

Packages

- package [com.example.football_manager.model](#)

9.77 Competition.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.model;
00002
00003 import jakarta.persistence.*;
00004 import lombok.AllArgsConstructor;
00005 import lombok.Data;
00006 import lombok.NoArgsConstructor;
00007
00008 @Entity
00009 @Table(name = "competition")
00010 @Data
00011 @NoArgsConstructor
00012 @AllArgsConstructor
00013 public class Competition {
00014
00015     @Id
00016     @GeneratedValue(strategy = GenerationType.IDENTITY)
00017     private Long id;
00018
00019     @Column(nullable = false, length = 50)
00020     private String name;
00021 }
00022

```

9.78 Football_Manager/src/main/java/com/example/football_manager/model/Country.java File Reference

```

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

```

Classes

- class [com.example.football_manager.model.Country](#)

Packages

- package [com.example.football_manager.model](#)

9.79 Country.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.model;
00002
00003 import jakarta.persistence.*;
00004 import lombok.AllArgsConstructor;
00005 import lombok.Data;
00006 import lombok.NoArgsConstructor;
00007
00008 @Entity
00009 @Table(name = "country")
00010 @Data
00011 @NoArgsConstructor
00012 @AllArgsConstructor
00013 public class Country {
00014
00015     @Id
00016     @GeneratedValue(strategy = GenerationType.IDENTITY)
00017     private Long id;
00018
00019     @Column(nullable = false, length = 50)
00020     private String name;
00021 }
00022

```

9.80 Football_Manager/src/main/java/com/example/football_manager/model/Match.java File Reference

```
import jakarta.persistence.*;
import lombok.*;
import java.time.OffsetDateTime;
```

Classes

- class [com.example.football_manager.model.Match](#)

Packages

- package [com.example.football_manager.model](#)

9.81 Match.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.model;
00002
00003 import jakarta.persistence.*;
00004 import lombok.*;
00005 import java.time.OffsetDateTime;
00006
00007 @Entity
00008 @Table(name = "match")
00009 @Data
00010 @NoArgsConstructor
00011 @AllArgsConstructor
00012 public class Match {
00013
00014     @Id
00015     @GeneratedValue(strategy = GenerationType.IDENTITY)
00016     private Long id;
00017
00018     @ManyToOne
00019     @JoinColumn(name = "fk_left_team_id", nullable = false)
00020     private Team leftTeam;
00021
00022     @ManyToOne
00023     @JoinColumn(name = "fk_right_team_id", nullable = false)
00024     private Team rightTeam;
00025
00026     @ManyToOne
00027     @JoinColumn(name = "fk_league_id", nullable = false)
00028     private Competition competition;
00029
00030     @Column(nullable = false)
00031     private OffsetDateTime datetime;
00032
00033     @Column(nullable = false, length = 100)
00034     private String venue;
00035
00036     @Column(name = "left_score", nullable = false)
00037     private short leftScore;
00038
00039     @Column(name = "right_score", nullable = false)
00040     private short rightScore;
00041
00042     @Column(nullable = false)
00043     private boolean finished;
00044 }
```

9.82 Football_Manager/src/main/java/com/example/football_manager/model/MatchGoal.java File Reference

```
import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
```



```
import lombok.NoArgsConstructor;
```

Classes

- class [com.example.football_manager.model.MatchGoal](#)

Packages

- package [com.example.football_manager.model](#)

9.83 MatchGoal.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.model;
00002
00003 import jakarta.persistence.*;
00004 import lombok.AllArgsConstructor;
00005 import lombok.Data;
00006 import lombok.NoArgsConstructor;
00007
00008 @Entity
00009 @Table(name = "match_goal")
00010 @Data
00011 @NoArgsConstructor
00012 @AllArgsConstructor
00013 public class MatchGoal {
00014
00015     @Id
00016     @GeneratedValue(strategy = GenerationType.IDENTITY)
00017     private Long id;
00018
00019     @ManyToOne
00020     @JoinColumn(name = "fk_match_id", nullable = false)
00021     private Match match;
00022
00023     @ManyToOne
00024     @JoinColumn(name = "fk_team_id", nullable = false)
00025     private Team team;
00026
00027     @Column(name = "goal_minute", nullable = false)
00028     private short minute;
00029
00030     @Column(name = "stoppage_minute")
00031     private Short stoppageMinute;
00032 }
```

9.84 Football_Manager/src/main/java/com/example/football_manager/model/Player.java File Reference

```
import com.fasterxml.jackson.annotation.JsonBackReference;
import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.NoArgsConstructor;
import lombok.ToString;
```

Classes

- class [com.example.football_manager.model.Player](#)

Packages

- package [com.example.football_manager.model](#)

9.85 Player.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.model;
00002
00003 import com.fasterxml.jackson.annotation.JsonBackReference;
00004 import jakarta.persistence.*;
00005 import lombok.AllArgsConstructor;
00006 import lombok.Data;
00007 import lombok.EqualsAndHashCode;
00008 import lombok.NoArgsConstructor;
00009 import lombok.ToString;
00010
00011 @Entity
00012 @Table(
00013     name = "player",
00014     uniqueConstraints = @UniqueConstraint(
00015         name = "uk_player_team_number",
00016         columnNames = {"fk_team_id", "number"}
00017     )
00018 )
00019 @Data
00020 @NoArgsConstructor
00021 @AllArgsConstructor
00022 public class Player {
00023
00024     @Id
00025     @GeneratedValue(strategy = GenerationType.IDENTITY)
00026     private Long id;
00027
00028     @Column(nullable = false, length = 30)
00029     private String name;
00030
00031     @Column(nullable = false)
00032     private Integer number;
00033
00034     @Enumerated(EnumType.STRING)
00035     @Column(nullable = false, length = 30)
00036     private PlayerPosition position;
00037
00038     @ManyToOne(fetch = FetchType.LAZY)
00039     @JoinColumn(name = "fk_team_id", nullable = false)
00040     @JsonBackReference
00041     @ToString.Exclude
00042     @EqualsAndHashCode.Exclude
00043     private Team team;
00044 }
00045

```

9.86 Football_Manager/src/main/java/com/example/football_manager/model/PlayerPosition.java File Reference ↩

Classes

- enum [com.example.football_manager.model.PlayerPosition](#)

Packages

- package [com.example.football_manager.model](#)

9.87 PlayerPosition.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.model;
00002
00003 public enum PlayerPosition {
00004     GOALKEEPER,
00005     DEFENDER,
00006     MIDFIELDER,
00007     FORWARD
00008 }

```

9.88 Football_Manager/src/main/java/com/example/football_manager/model/Team.java File Reference

```
import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.NoArgsConstructor;
import lombok.ToString;
import java.util.ArrayList;
import java.util.List;
```

Classes

- class [com.example.football_manager.model.Team](#)

Packages

- package [com.example.football_manager.model](#)

9.89 Team.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.model;
00002
00003 import com.fasterxml.jackson.annotation.JsonIgnore;
00004 import jakarta.persistence.*;
00005 import lombok.AllArgsConstructor;
00006 import lombok.Data;
00007 import lombok.EqualsAndHashCode;
00008 import lombok.NoArgsConstructor;
00009 import lombok.ToString;
00010
00011 import java.util.ArrayList;
00012 import java.util.List;
00013
00014 @Entity
00015 @Table(name = "team")
00016 @Data
00017 @NoArgsConstructor
00018 @AllArgsConstructor
00019 public class Team {
00020
00021     @Id
00022     @GeneratedValue(strategy = GenerationType.IDENTITY)
00023     private Long id;
00024
00025     @Column(nullable = false, length = 30)
00026     private String name;
00027
00028     @Column(name = "logo_url", nullable = false, columnDefinition = "TEXT")
00029     private String logoUrl;
00030
00031     @ManyToOne
00032     @JoinColumn(name = "fk_country_id", nullable = false)
00033     private Country country;
00034
00035     @OneToMany(mappedBy = "team", cascade = CascadeType.ALL, orphanRemoval = true)
00036     @OrderBy("number ASC")
00037     @JsonIgnore
00038     @ToString.Exclude
00039     @EqualsAndHashCode.Exclude
00040     private List<Player> players = new ArrayList<>();
00041
00042     public Team(Long id, String name, String logoUrl, Country country) {
00043         this.id = id;
00044         this.name = name;
00045         this.logoUrl = logoUrl;
00046         this.country = country;
00047     }
00048 }
```

9.90 Football_Manager/src/main/java/com/example/football_manager/model/User.java File Reference

```
import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import java.util.Set;
```

Classes

- class [com.example.football_manager.model.User](#)

Packages

- package [com.example.football_manager.model](#)

9.91 User.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.model;
00002
00003 import jakarta.persistence.*;
00004 import lombok.AllArgsConstructor;
00005 import lombok.Data;
00006 import lombok.NoArgsConstructor;
00007
00008 import java.util.Set;
00009
00010 @Entity
00011 @Table(name = "\"user\"")
00012 @Data
00013 @NoArgsConstructor
00014 @AllArgsConstructor
00015 public class User {
00016
00017     @Id
00018     @GeneratedValue(strategy = GenerationType.IDENTITY)
00019     private Long id;
00020
00021     @Column(nullable = false, length = 30)
00022     private String username;
00023
00024     @Column(nullable = false, columnDefinition = "TEXT")
00025     private String email;
00026
00027     @Column(name = "hashed_password", nullable = false, columnDefinition = "TEXT")
00028     private String hashedPassword;
00029
00030     @Column(name = "is_admin", nullable = false)
00031     private Boolean isAdmin = false;
00032
00033     @ManyToMany
00034     @JoinTable(
00035         name = "favourite_team",
00036         joinColumns = @JoinColumn(name = "fk_user_id"),
00037         inverseJoinColumns = @JoinColumn(name = "fk_team_id")
00038     )
00039     private Set<Team> favouriteTeams;
00040 }
```

9.92 Football_Manager/src/main/java/com/example/football_manager/repository/CompetitionRepository.java File Reference

```
import com.example.football_manager.model.Competition;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
```

Classes

- interface [com.example.football_manager.repository.CompetitionRepository](#)

Packages

- package [com.example.football_manager.repository](#)

9.93 CompetitionRepository.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.repository;
00002
00003 import com.example.football_manager.model.Competition;
00004 import org.springframework.data.jpa.repository.JpaRepository;
00005 import org.springframework.stereotype.Repository;
00006
00007 @Repository
00008 public interface CompetitionRepository extends JpaRepository<Competition, Long> {}
```

9.94 Football_Manager/src/main/java/com/example/football_↵ manager/repository/CountryRepository.java File Reference

```
import com.example.football_manager.model.Country;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
```

Classes

- interface [com.example.football_manager.repository.CountryRepository](#)

Packages

- package [com.example.football_manager.repository](#)

9.95 CountryRepository.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.repository;
00002
00003 import com.example.football_manager.model.Country;
00004 import org.springframework.data.jpa.repository.JpaRepository;
00005 import org.springframework.stereotype.Repository;
00006
00007 @Repository
00008 public interface CountryRepository extends JpaRepository<Country, Long> {
00009 }
00010
```

9.96 Football_Manager/src/main/java/com/example/football_↵ manager/repository/MatchGoalRepository.java File Reference

```
import com.example.football_manager.model.MatchGoal;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;
```

Classes

- interface [com.example.football_manager.repository.MatchGoalRepository](#)

Packages

- package [com.example.football_manager.repository](#)

9.97 MatchGoalRepository.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.repository;
00002
00003 import com.example.football_manager.model.MatchGoal;
00004 import org.springframework.data.jpa.repository.JpaRepository;
00005 import org.springframework.stereotype.Repository;
00006
00007 import java.util.List;
00008
00009 @Repository
00010 public interface MatchGoalRepository extends JpaRepository<MatchGoal, Long> {
00011
00012     void deleteByMatchId(Long matchId);
00013
00014     List<MatchGoal> findByMatchIdOrderByMinuteAscStoppageMinuteAscIdAsc(Long matchId);
00015 }

```

9.98 Football_Manager/src/main/java/com/example/football_manager/repository/MatchRepository.java File Reference

```

import com.example.football_manager.model.Match;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;
import java.time.OffsetDateTime;
import java.util.Collection;
import java.util.List;

```

Classes

- interface [com.example.football_manager.repository.MatchRepository](#)

Packages

- package [com.example.football_manager.repository](#)

9.99 MatchRepository.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.repository;
00002
00003 import com.example.football_manager.model.Match;
00004 import org.springframework.data.jpa.repository.JpaRepository;
00005 import org.springframework.data.jpa.repository.Query;
00006 import org.springframework.data.repository.query.Param;
00007 import org.springframework.stereotype.Repository;
00008
00009 import java.time.OffsetDateTime;
00010 import java.util.Collection;
00011 import java.util.List;
00012
00013 @Repository
00014 public interface MatchRepository extends JpaRepository<Match, Long> {
00015

```

```
00016     @Query("""
00017         SELECT m FROM Match m
00018         WHERE (:teamId IS NULL OR m.leftTeam.id = :teamId OR m.rightTeam.id = :teamId)
00019         AND (:finished IS NULL OR m.finished = :finished)
00020         AND (:competitionId IS NULL OR m.competition.id = :competitionId)
00021         """)
00022     List<Match> findByFilters(
00023         @Param("teamId") Long teamId,
00024         @Param("finished") Boolean finished,
00025         @Param("competitionId") Long competitionId
00026     );
00027
00028     List<Match> findByFinishedTrueOrderByDatetimeDesc();
00029
00030     List<Match> findByFinishedTrueAndLeftTeamIdOrFinishedTrueAndRightTeamIdOrderByDatetimeDesc(
00031         Long leftTeamId,
00032         Long rightTeamId
00033     );
00034
00035     List<Match>
00036         findByFinishedFalseAndDatetimeAfterAndLeftTeamIdOrFinishedFalseAndDatetimeAfterAndRightTeamIdOrderByDatetimeAsc(
00037         OffsetDateTime now1,
00038         Long leftTeamId,
00039         OffsetDateTime now2,
00040         Long rightTeamId
00041     );
00042
00043     List<Match>
00044         findByFinishedFalseAndDatetimeAfterAndLeftTeamIdInOrFinishedFalseAndDatetimeAfterAndRightTeamIdInOrderByDatetimeAsc(
00045         OffsetDateTime now1,
00046         Collection<Long> leftTeamIds,
00047         OffsetDateTime now2,
00048         Collection<Long> rightTeamIds
00049     );
00050 }
```

9.100 Football_Manager/src/main/java/com/example/football_manager/repository/PlayerRepository.java File Reference

```
import com.example.football_manager.model.Player;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;
```

Classes

- interface [com.example.football_manager.repository.PlayerRepository](#)

Packages

- package [com.example.football_manager.repository](#)

9.101 PlayerRepository.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.repository;
00002
00003 import com.example.football_manager.model.Player;
00004 import org.springframework.data.jpa.repository.JpaRepository;
00005 import org.springframework.stereotype.Repository;
00006
00007 import java.util.List;
00008
00009 @Repository
00010 public interface PlayerRepository extends JpaRepository<Player, Long> {
00011     boolean existsByTeamIdAndNumber(Long teamId, Integer number);
00012
00013     boolean existsByTeamIdAndNumberAndIdNot(Long teamId, Integer number, Long id);
00014
00015     List<Player> findByTeamIdOrderByNumberAsc(Long teamId);
00016
00017     java.util.Optional<Player> findByIdAndTeamId(Long id, Long teamId);
00018 }
```

00019

9.102 Football_Manager/src/main/java/com/example/football_manager/repository/TeamRepository.java File Reference

```
import com.example.football_manager.model.Team;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;
```

Classes

- interface [com.example.football_manager.repository.TeamRepository](#)

Packages

- package [com.example.football_manager.repository](#)

9.103 TeamRepository.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.repository;
00002
00003 import com.example.football_manager.model.Team;
00004 import org.springframework.data.jpa.repository.JpaRepository;
00005 import org.springframework.stereotype.Repository;
00006
00007 import java.util.List;
00008
00009 @Repository
00010 public interface TeamRepository extends JpaRepository<Team, Long> {
00011     List<Team> findByNameContainingIgnoreCase(String name);
00012 }
00013
```

9.104 Football_Manager/src/main/java/com/example/football_manager/repository/UserRepository.java File Reference

```
import com.example.football_manager.model.User;
import org.springframework.data.jpa.repository.EntityGraph;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.Optional;
```

Classes

- interface [com.example.football_manager.repository.UserRepository](#)

Packages

- package [com.example.football_manager.repository](#)

9.105 UserRepository.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.repository;
00002
```



```
00003 import com.example.football_manager.model.User;
00004 import org.springframework.data.jpa.repository.EntityGraph;
00005 import org.springframework.data.jpa.repository.JpaRepository;
00006 import org.springframework.stereotype.Repository;
00007
00008 import java.util.Optional;
00009
00010 @Repository
00011 public interface UserRepository extends JpaRepository<User, Long> {
00012     Optional<User> findByUsername(String username);
00013     Optional<User> findByEmail(String email);
00014     @EntityGraph(attributePaths = "favouriteTeams")
00015     Optional<User> findWithFavouriteTeamsById(Long id);
00016     boolean existsByUsername(String username);
00017     boolean existsByEmail(String email);
00018     boolean existsByUsernameAndIdNot(String username, Long id);
00019     boolean existsByEmailAndIdNot(String email, Long id);
00020 }
```

9.106 Football_Manager/src/main/java/com/example/football_manager/service/CompetitionService.java File Reference

```
import com.example.football_manager.dto.CompetitionRequestDTO;
import com.example.football_manager.model.Competition;
import com.example.football_manager.repository.CompetitionRepository;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;
```

Classes

- class [com.example.football_manager.service.CompetitionService](#)

Packages

- package [com.example.football_manager.service](#)

9.107 CompetitionService.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.service;
00002
00003 import com.example.football_manager.dto.CompetitionRequestDTO;
00004 import com.example.football_manager.model.Competition;
00005 import com.example.football_manager.repository.CompetitionRepository;
00006 import org.springframework.stereotype.Service;
00007
00008 import java.util.List;
00009 import java.util.Optional;
00010
00011 @Service
00012 public class CompetitionService {
00013
00014     private final CompetitionRepository competitionRepository;
00015
00016     public CompetitionService(CompetitionRepository competitionRepository) {
00017         this.competitionRepository = competitionRepository;
00018     }
00019
00020     public Competition createCompetition(CompetitionRequestDTO dto) {
00021         Competition competition = new Competition();
00022         competition.setName(dto.getName().trim());
00023         return competitionRepository.save(competition);
00024     }
00025
00026     public Competition updateCompetition(Long id, CompetitionRequestDTO dto) {
00027         Competition competition = competitionRepository.findById(id)
00028             .orElseThrow(() -> new RuntimeException("Competition not found with id: " + id));
00029
00030         competition.setName(dto.getName().trim());
00031         return competitionRepository.save(competition);
00032     }
}
```

```

00033
00034     public Optional<Competition> getCompetitionById(Long id) {
00035         return competitionRepository.findById(id);
00036     }
00037
00038     public List<Competition> getAllCompetitions() {
00039         return competitionRepository.findAll();
00040     }
00041
00042     public void deleteCompetition(Long id) {
00043         Competition competition = competitionRepository.findById(id)
00044             .orElseThrow(() -> new RuntimeException("Competition not found with id: " + id));
00045
00046         competitionRepository.delete(competition);
00047     }
00048 }

```

9.108 Football_Manager/src/main/java/com/example/football_manager/service/CountryService.java File Reference ↩

```

import com.example.football_manager.dto.CountryRequestDTO;
import com.example.football_manager.model.Country;
import com.example.football_manager.repository.CountryRepository;
import org.springframework.stereotype.Service;
import java.util.List;

```

Classes

- class [com.example.football_manager.service.CountryService](#)

Packages

- package [com.example.football_manager.service](#)

9.109 CountryService.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.service;
00002
00003 import com.example.football_manager.dto.CountryRequestDTO;
00004 import com.example.football_manager.model.Country;
00005 import com.example.football_manager.repository.CountryRepository;
00006 import org.springframework.stereotype.Service;
00007
00008 import java.util.List;
00009
00010 @Service
00011 public class CountryService {
00012
00013     private final CountryRepository countryRepository;
00014
00015     public CountryService(CountryRepository countryRepository) {
00016         this.countryRepository = countryRepository;
00017     }
00018
00019     public Country createCountry(CountryRequestDTO dto) {
00020         Country country = new Country();
00021         country.setName(dto.getName().trim());
00022         return countryRepository.save(country);
00023     }
00024
00025     public List<Country> getAllCountries() {
00026         return countryRepository.findAll();
00027     }
00028
00029     public void deleteCountry(Long id) {
00030         Country country = countryRepository.findById(id)
00031             .orElseThrow(() -> new RuntimeException("Country not found with id: " + id));
00032
00033         countryRepository.delete(country);
00034     }
00035 }

```

9.110 Football_Manager/src/main/java/com/example/football_manager/service/MatchService.java File Reference

```
import com.example.football_manager.dto.MatchRequestDTO;
import com.example.football_manager.dto.MatchResultDTO;
import com.example.football_manager.dto.MatchResultRequestDTO;
import com.example.football_manager.model.Competition;
import com.example.football_manager.model.Match;
import com.example.football_manager.model.MatchGoal;
import com.example.football_manager.model.Team;
import com.example.football_manager.repository.CompetitionRepository;
import com.example.football_manager.repository.MatchGoalRepository;
import com.example.football_manager.repository.MatchRepository;
import com.example.football_manager.repository.TeamRepository;
import org.springframework.cache.annotation.CacheEvict;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.time.OffsetDateTime;
import java.time.ZoneId;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
import java.util.Optional;
import java.util.Set;
```

Classes

- class [com.example.football_manager.service.MatchService](#)

Packages

- package [com.example.football_manager.service](#)

9.111 MatchService.java

[Go to the documentation of this file.](#)

```
00001 package com.example.football_manager.service;
00002
00003 import com.example.football_manager.dto.MatchRequestDTO;
00004 import com.example.football_manager.dto.MatchResultDTO;
00005 import com.example.football_manager.dto.MatchResultRequestDTO;
00006 import com.example.football_manager.model.Competition;
00007 import com.example.football_manager.model.Match;
00008 import com.example.football_manager.model.MatchGoal;
00009 import com.example.football_manager.model.Team;
00010 import com.example.football_manager.repository.CompetitionRepository;
00011 import com.example.football_manager.repository.MatchGoalRepository;
00012 import com.example.football_manager.repository.MatchRepository;
00013 import com.example.football_manager.repository.TeamRepository;
00014
00015 import org.springframework.cache.annotation.CacheEvict;
00016 import org.springframework.cache.annotation.Cacheable;
00017 import org.springframework.stereotype.Service;
00018 import org.springframework.transaction.annotation.Transactional;
00019
00020 import java.time.OffsetDateTime;
00021 import java.time.ZoneId;
00022 import java.util.ArrayList;
00023 import java.util.List;
00024 import java.util.Objects;
00025 import java.util.Optional;
00026 import java.util.Set;
00027
00028 @Service
00029 public class MatchService {
```

```

00030
00031     private final MatchRepository matchRepository;
00032     private final MatchGoalRepository matchGoalRepository;
00033     private final TeamRepository teamRepository;
00034     private final CompetitionRepository competitionRepository;
00035
00036
00037     public MatchService(
00038         MatchRepository matchRepository,
00039         MatchGoalRepository matchGoalRepository,
00040         TeamRepository teamRepository,
00041         CompetitionRepository competitionRepository
00042     ) {
00043         this.matchRepository = matchRepository;
00044         this.matchGoalRepository = matchGoalRepository;
00045         this.teamRepository = teamRepository;
00046         this.competitionRepository = competitionRepository;
00047     }
00048
00049     @Transactional
00050     @CacheEvict(cacheNames = {"matches", "upcomingMatches", "standings", "matchResults"}, allEntries =
00051 true)
00052     public String createMatch(MatchRequestDTO request) {
00053         validateMatchRequest(request, true);
00054
00055         Team homeTeam = teamRepository.findById(request.getHomeTeamId())
00056             .orElseThrow(() -> new IllegalArgumentException("Home team not found with id: " +
00057 request.getHomeTeamId()));
00058
00059         Team awayTeam = teamRepository.findById(request.getAwayTeamId())
00060             .orElseThrow(() -> new IllegalArgumentException("Away team not found with id: " +
00061 request.getAwayTeamId()));
00062
00063         Competition competition = competitionRepository.findById(request.getCompetitionId())
00064             .orElseThrow(() -> new IllegalArgumentException("Competition not found with id: " +
00065 request.getCompetitionId()));
00066
00067         Match match = new Match();
00068         match.setLeftTeam(homeTeam);
00069         match.setRightTeam(awayTeam);
00070         match.setCompetition(competition);
00071         match.setDatetime(request.getKickoffTime().atZone(ZoneId.systemDefault()).toOffsetDateTime());
00072         match.setVenue(request.getVenue().trim());
00073         match.setLeftScore(toShortScore(request.getHomeScore()));
00074         match.setRightScore(toShortScore(request.getAwayScore()));
00075         match.setFinished(request.getStatus() == MatchRequestDTO.MatchStatus.FINISHED);
00076
00077         matchRepository.save(match);
00078
00079         return "Match scheduled successfully.";
00080     }
00081
00082     private void validateMatchRequest(MatchRequestDTO request, boolean competitionRequired) {
00083         if (request == null) {
00084             throw new IllegalArgumentException("Validation Error: Match data is required.");
00085         }
00086
00087         if (request.getHomeTeamId() == null || request.getAwayTeamId() == null) {
00088             throw new IllegalArgumentException("Validation Error: Both Home and Away team IDs are
00089 required.");
00090         }
00091
00092         if (competitionRequired && request.getCompetitionId() == null) {
00093             throw new IllegalArgumentException("Validation Error: Competition ID is required.");
00094         }
00095
00096         if (request.getKickoffTime() == null || request.getVenue() == null) {
00097             throw new IllegalArgumentException("Validation Error: Kickoff time and Venue are
00098 required.");
00099         }
00100
00101         if (request.getHomeTeamId().equals(request.getAwayTeamId())) {
00102             throw new IllegalArgumentException("Validation Error: Home and Away teams must be
00103 different.");
00104         }
00105
00106         if (request.getStatus() == null) {
00107             request.setStatus(MatchRequestDTO.MatchStatus.SCHEDULED);
00108         }
00109     }
00110
00111     @Transactional
00112     @CacheEvict(cacheNames = {"matches", "upcomingMatches", "standings", "matchResults"}, allEntries =
00113 true)
00114     public String updateMatch(Long id, MatchRequestDTO request) {
00115         validateMatchUpdate(request);
00116     }

```

```

00109         Match match = matchRepository.findById(id)
00110             .orElseThrow(() -> new IllegalArgumentException("Match not found"));
00111
00112         Team homeTeam = teamRepository.findById(request.getHomeTeamId())
00113             .orElseThrow(() -> new IllegalArgumentException("Home team not found"));
00114         Team awayTeam = teamRepository.findById(request.getAwayTeamId())
00115             .orElseThrow(() -> new IllegalArgumentException("Away team not found"));
00116
00117         match.setLeftTeam(homeTeam);
00118         match.setRightTeam(awayTeam);
00119
00120         // Actualizamos la fecha usando la zona horaria del sistema de tu compañero
00121         if (request.getKickoffTime() != null) {
00122             match.setDatetime(request.getKickoffTime().atZone(ZoneId.systemDefault()).toOffsetDateTime());
00123         }
00124
00125         // Actualizamos la sede (venue)
00126         if (request.getVenue() != null && !request.getVenue().isBlank()) {
00127             match.setVenue(request.getVenue().trim());
00128         }
00129
00130         match.setFinished(request.getStatus() == MatchRequestDTO.MatchStatus.FINISHED);
00131
00132         if (request.getHomeScore() != null) {
00133             match.setLeftScore(request.getHomeScore().shortValue());
00134         }
00135         if (request.getAwayScore() != null) {
00136             match.setRightScore(request.getAwayScore().shortValue());
00137         }
00138
00139         matchRepository.save(match);
00140
00141         return "Match with ID " + id + " has been updated.";
00142     }
00143
00144     @Transactional
00145     @CacheEvict(cacheNames = {"matches", "upcomingMatches", "standings", "matchResults"}, allEntries =
true)
00146     public String deleteMatch(Long id) {
00147         Match match = matchRepository.findById(id)
00148             .orElseThrow(() -> new IllegalArgumentException("Match not found with id: " + id));
00149
00150         matchGoalRepository.deleteByMatchId(id);
00151         matchRepository.delete(match);
00152
00153         return "Match with ID " + id + " has been deleted.";
00154     }
00155
00156     @Transactional
00157     @CacheEvict(cacheNames = {"matches", "upcomingMatches", "standings", "matchResults"}, allEntries =
true)
00158     public String registerResult(Long id, MatchResultRequestDTO request) {
00159         if (request == null || request.getGoals() == null) {
00160             throw new IllegalArgumentException("Validation Error: Goals are required.");
00161         }
00162
00163         Match match = matchRepository.findById(id)
00164             .orElseThrow(() -> new IllegalArgumentException("Match not found with id: " + id));
00165
00166         List<MatchGoal> goals = new ArrayList<>();
00167         int leftScore = 0;
00168         int rightScore = 0;
00169
00170         for (MatchResultRequestDTO.GoalDTO goalDTO : request.getGoals()) {
00171             Team scoringTeam = resolveScoringTeam(match, goalDTO);
00172
00173             if (Objects.equals(scoringTeam.getId(), match.getLeftTeam().getId())) {
00174                 leftScore++;
00175             } else {
00176                 rightScore++;
00177             }
00178
00179             MatchGoal goal = new MatchGoal();
00180             goal.setMatch(match);
00181             goal.setTeam(scoringTeam);
00182             goal.setMinute(goalDTO.getMinute().shortValue());
00183             goal.setStoppageMinute(toShort(goalDTO.getStoppageMinute()));
00184             goals.add(goal);
00185         }
00186
00187         matchGoalRepository.deleteByMatchId(id);
00188         matchGoalRepository.saveAll(goals);
00189
00190         match.setLeftScore((short) leftScore);
00191         match.setRightScore((short) rightScore);
00192         match.setFinished(true);

```

```

00193         matchRepository.save(match);
00194
00195         return "Result registered for match " + id + ": " + leftScore + " - " + rightScore;
00196     }
00197
00198     @Cacheable(cacheNames = "matchResults")
00199     public List<MatchResultDTO> getFinishedMatchResults() {
00200         List<Match> matches = matchRepository.findByFinishedTrueOrderByDatetimeDesc();
00201
00202         return matches.stream()
00203             .map(match -> new MatchResultDTO(
00204                 match.getId(),
00205                 match.getLeftTeam().getName(),
00206                 match.getRightTeam().getName(),
00207                 match.getLeftScore(),
00208                 match.getRightScore(),
00209                 match.getDatetime()
00210             ))
00211             .toList();
00212     }
00213
00214     public Optional<MatchRequestDTO> getMatchForEdit(Long id) {
00215         if (matchRepository == null) {
00216             return Optional.empty();
00217         }
00218
00219         return matchRepository.findById(id).map(match -> {
00220             MatchRequestDTO dto = new MatchRequestDTO();
00221             dto.setHomeTeamId(match.getLeftTeam().getId());
00222             dto.setAwayTeamId(match.getRightTeam().getId());
00223             dto.setKickoffTime(match.getDatetime().toLocalDateTime());
00224             dto.setVenue(match.getVenue()); // Carga el venue real, no "TBD"
00225             dto.setStatus(match.isFinished()
00226                 ? MatchRequestDTO.MatchStatus.FINISHED
00227                 : MatchRequestDTO.MatchStatus.SCHEDULED);
00228             dto.setHomeScore((int) match.getLeftScore());
00229             dto.setAwayScore((int) match.getRightScore());
00230             return dto;
00231         });
00232     }
00233
00234     private void validateMatchUpdate(MatchRequestDTO request) {
00235         if (request.getHomeTeamId() == null || request.getAwayTeamId() == null) {
00236             throw new IllegalArgumentException("Validation Error: Both Home and Away team IDs are
00237 required.");
00238         }
00239
00240         if (request.getKickoffTime() == null || request.getVenue() == null ||
00241 request.getVenue().isBlank()) {
00242             throw new IllegalArgumentException("Validation Error: Kickoff time and Venue are
00243 required.");
00244         }
00245
00246         if (request.getStatus() == null) {
00247             throw new IllegalArgumentException("Validation Error: Match status is required.");
00248         }
00249
00250         if (request.getHomeTeamId().equals(request.getAwayTeamId())) {
00251             throw new IllegalArgumentException("Validation Error: Home and Away teams must be
00252 different.");
00253         }
00254
00255         if (request.getHomeScore() != null && request.getAwayScore() == null) {
00256             throw new IllegalArgumentException("Validation Error: Away score is required when home
00257 score is provided.");
00258         }
00259
00260         if (request.getAwayScore() != null && request.getHomeScore() == null) {
00261             throw new IllegalArgumentException("Validation Error: Home score is required when away
00262 score is provided.");
00263         }
00264
00265         if (request.getStatus() == MatchRequestDTO.MatchStatus.FINISHED
00266             && (request.getHomeScore() == null || request.getAwayScore() == null)) {
00267             throw new IllegalArgumentException("Validation Error: Both scores are required when status
00268 is FINISHED.");
00269         }
00270
00271         if (request.getStatus() != MatchRequestDTO.MatchStatus.FINISHED
00272             && (request.getHomeScore() != null || request.getAwayScore() != null)) {
00273             throw new IllegalArgumentException("Validation Error: Scores can only be submitted when
00274 status is FINISHED.");
00275         }
00276     }
00277
00278     public List<Match> getAllMatches() {
00279         return matchRepository.findAll();
00280     }

```

```

00272     }
00273
00274     @Cacheable(cacheNames = "upcomingMatches")
00275     public List<Match> getUpcomingMatches() {
00276         return matchRepository.findAll().stream()
00277             .filter(match -> !match.isFinished())
00278             .sorted((m1, m2) -> m1.getDatetime().compareTo(m2.getDatetime()))
00279             .toList();
00280     }
00281
00282     @Cacheable(cacheNames = "matches", key = "T(java.util.Objects).hash(#teamId, #status,
#competitionId)")
00283     public List<Match> getMatches(Long teamId, MatchRequestDTO.MatchStatus status, Long competitionId)
{
00284         Boolean finished = toFinishedFilter(status);
00285
00286         if (teamId == null && finished == null && competitionId == null) {
00287             return getAllMatches();
00288         }
00289
00290         return matchRepository.findByFilters(teamId, finished, competitionId);
00291     }
00292
00293     public Match getMatchById(Long id) {
00294         return matchRepository.findById(id)
00295             .orElseThrow(() -> new IllegalArgumentException("Match not found with id: " + id));
00296     }
00297
00298     public List<MatchGoal> getGoalsByMatchId(Long matchId) {
00299         return matchGoalRepository.findByMatchIdOrderByMinuteAscStoppageMinuteAscIdAsc(matchId);
00300     }
00301
00302     public List<Match> getRecentMatchesByTeamId(Long teamId) {
00303         return matchRepository
00304             .findByFinishedTrueAndLeftTeamIdOrFinishedTrueAndRightTeamIdOrderByDatetimeDesc(
00305                 teamId,
00306                 teamId
00307             );
00308     }
00309
00310     public List<Match> getFutureMatchesByTeamId(Long teamId) {
00311         OffsetDateTime now = OffsetDateTime.now();
00312
00313         return matchRepository
00314             .findByFinishedFalseAndDatetimeAfterAndLeftTeamIdOrFinishedFalseAndDatetimeAfterAndRightTeamIdOrderByDatetimeAsc(
00315                 now,
00316                 teamId,
00317                 now,
00318                 teamId
00319             );
00320     }
00321
00322     public List<Match> getFutureMatchesByFavouriteTeamIds(Set<Long> favouriteTeamIds) {
00323         if (favouriteTeamIds == null || favouriteTeamIds.isEmpty()) {
00324             return List.of();
00325         }
00326
00327         OffsetDateTime now = OffsetDateTime.now();
00328
00329         return matchRepository
00330             .findByFinishedFalseAndDatetimeAfterAndLeftTeamIdInOrFinishedFalseAndDatetimeAfterAndRightTeamIdInOrderByDatetimeAsc(
00331                 now,
00332                 favouriteTeamIds,
00333                 now,
00334                 favouriteTeamIds
00335             );
00336     }
00337
00338     private Team resolveScoringTeam(Match match, MatchResultRequestDTO.GoalDTO goalDTO) {
00339         if (goalDTO == null || goalDTO.getTeamId() == null || goalDTO.getMinute() == null) {
00340             throw new IllegalArgumentException("Validation Error: Every goal must include team ID and
minute.");
00341         }
00342
00343         if (goalDTO.getMinute() < 1 || goalDTO.getMinute() > 120) {
00344             throw new IllegalArgumentException("Validation Error: Goal minute must be between 1 and
120.");
00345         }
00346
00347         if (goalDTO.getStoppageMinute() != null
&& (goalDTO.getStoppageMinute() < 0 || goalDTO.getStoppageMinute() > 30)) {
00348             throw new IllegalArgumentException("Validation Error: Stoppage minute must be between 0
and 30.");
00349         }
00350     }
00351

```

```

00352         if (Objects.equals(goalDTO.getTeamId(), match.getLeftTeam().getId())) {
00353             return match.getLeftTeam();
00354         }
00355
00356         if (Objects.equals(goalDTO.getTeamId(), match.getRightTeam().getId())) {
00357             return match.getRightTeam();
00358         }
00359
00360         throw new IllegalArgumentException("Validation Error: Scoring team must belong to the
match.");
00361     }
00362
00363     private Short toShort(Integer value) {
00364         return value == null ? null : value.shortValue();
00365     }
00366
00367     private short toShortScore(Integer score) {
00368         if (score == null) {
00369             return 0;
00370         }
00371
00372         if (score < 0 || score > Short.MAX_VALUE) {
00373             throw new IllegalArgumentException("Validation Error: Score must be a positive number.");
00374         }
00375
00376         return score.shortValue();
00377     }
00378
00379     private Boolean toFinishedFilter(MatchRequestDTO.MatchStatus status) {
00380         if (status == null) {
00381             return null;
00382         }
00383
00384         return switch (status) {
00385             case FINISHED -> true;
00386             case SCHEDULED -> false;
00387             default -> throw new IllegalArgumentException("Validation Error: Match status filter must
be FINISHED or SCHEDULED.");
00388         };
00389     }
00390 }

```

9.112 Football_Manager/src/main/java/com/example/football_↵ manager/service/PlayerService.java File Reference

```

import com.example.football_manager.dto.PlayerRequestDTO;
import com.example.football_manager.model.Player;
import com.example.football_manager.model.Team;
import com.example.football_manager.repository.PlayerRepository;
import com.example.football_manager.repository.TeamRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

```

Classes

- class [com.example.football_manager.service.PlayerService](#)

Packages

- package [com.example.football_manager.service](#)

9.113 PlayerService.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.service;
00002
00003 import com.example.football_manager.dto.PlayerRequestDTO;

```



```

00004 import com.example.football_manager.model.Player;
00005 import com.example.football_manager.model.Team;
00006 import com.example.football_manager.repository.PlayerRepository;
00007 import com.example.football_manager.repository.TeamRepository;
00008 import org.springframework.stereotype.Service;
00009 import org.springframework.transaction.annotation.Transactional;
00010
00011 import java.util.HashSet;
00012 import java.util.List;
00013 import java.util.Set;
00014
00015 @Service
00016 public class PlayerService {
00017
00018     private final PlayerRepository playerRepository;
00019     private final TeamRepository teamRepository;
00020
00021     public PlayerService(PlayerRepository playerRepository, TeamRepository teamRepository) {
00022         this.playerRepository = playerRepository;
00023         this.teamRepository = teamRepository;
00024     }
00025
00026     public Player createPlayer(Long teamId, PlayerRequestDTO dto) {
00027         validatePlayerRequest(teamId, dto);
00028
00029         Team team = teamRepository.findById(teamId)
00030             .orElseThrow(() -> new IllegalArgumentException("Team not found with id: " + teamId));
00031
00032         if (playerRepository.existsByTeamIdAndNumber(teamId, dto.getNumber())) {
00033             throw new IllegalArgumentException(
00034                 "Validation Error: Team already has a player with number " + dto.getNumber() + "."
00035             );
00036         }
00037
00038         Player player = new Player();
00039         player.setName(dto.getName().trim());
00040         player.setNumber(dto.getNumber());
00041         player.setPosition(dto.getPosition());
00042         player.setTeam(team);
00043
00044         Player savedPlayer = playerRepository.save(player);
00045         team.getPlayers().add(savedPlayer);
00046
00047         return savedPlayer;
00048     }
00049
00050     @Transactional
00051     public List<Player> createPlayers(Long teamId, List<PlayerRequestDTO> playerRequests) {
00052         if (teamId == null) {
00053             throw new IllegalArgumentException("Validation Error: Team ID is required.");
00054         }
00055
00056         if (playerRequests == null || playerRequests.isEmpty()) {
00057             throw new IllegalArgumentException("Validation Error: Player list is required.");
00058         }
00059
00060         Team team = teamRepository.findById(teamId)
00061             .orElseThrow(() -> new IllegalArgumentException("Team not found with id: " + teamId));
00062
00063         Set<Integer> numbersInRequest = new HashSet<>();
00064         for (PlayerRequestDTO playerRequest : playerRequests) {
00065             validatePlayerRequest(teamId, playerRequest);
00066
00067             if (!numbersInRequest.add(playerRequest.getNumber())) {
00068                 throw new IllegalArgumentException(
00069                     "Validation Error: Request contains duplicated player number " +
00070                     playerRequest.getNumber() + "."
00071                 );
00072             }
00073
00074             if (playerRepository.existsByTeamIdAndNumber(teamId, playerRequest.getNumber())) {
00075                 throw new IllegalArgumentException(
00076                     "Validation Error: Team already has a player with number " +
00077                     playerRequest.getNumber() + "."
00078                 );
00079             }
00080
00081             List<Player> players = playerRequests.stream().map(playerRequest -> {
00082                 Player player = new Player();
00083                 player.setName(playerRequest.getName().trim());
00084                 player.setNumber(playerRequest.getNumber());
00085                 player.setPosition(playerRequest.getPosition());
00086                 player.setTeam(team);
00087                 return player;
00088             }).toList();

```

```

00089         List<Player> savedPlayers = playerRepository.saveAll(players);
00090         team.getPlayers().addAll(savedPlayers);
00091         return savedPlayers;
00092     }
00093
00094     public List<Player> getPlayersByTeamId(Long teamId) {
00095         if (teamId == null) {
00096             throw new IllegalArgumentException("Validation Error: Team ID is required.");
00097         }
00098
00099         if (!teamRepository.existsById(teamId)) {
00100             throw new IllegalArgumentException("Team not found with id: " + teamId);
00101         }
00102
00103         return playerRepository.findByTeamIdOrderByNumberAsc(teamId);
00104     }
00105
00106     public Player getPlayerByTeamId(Long teamId, Long playerId) {
00107         if (teamId == null) {
00108             throw new IllegalArgumentException("Validation Error: Team ID is required.");
00109         }
00110
00111         if (playerId == null) {
00112             throw new IllegalArgumentException("Validation Error: Player ID is required.");
00113         }
00114
00115         if (!teamRepository.existsById(teamId)) {
00116             throw new IllegalArgumentException("Team not found with id: " + teamId);
00117         }
00118
00119         return playerRepository.findByIdAndTeamId(playerId, teamId)
00120             .orElseThrow(() -> new IllegalArgumentException(
00121                 "Player not found with id: " + playerId + " for team id: " + teamId
00122             ));
00123     }
00124
00125     public Player updatePlayer(Long teamId, Long playerId, PlayerRequestDTO dto) {
00126         validatePlayerRequest(teamId, dto);
00127
00128         if (playerId == null) {
00129             throw new IllegalArgumentException("Validation Error: Player ID is required.");
00130         }
00131
00132         Team team = teamRepository.findById(teamId)
00133             .orElseThrow(() -> new IllegalArgumentException("Team not found with id: " + teamId));
00134
00135         Player player = playerRepository.findByIdAndTeamId(playerId, teamId)
00136             .orElseThrow(() -> new IllegalArgumentException(
00137                 "Player not found with id: " + playerId + " for team id: " + teamId
00138             ));
00139
00140         if (playerRepository.existsByTeamIdAndNumberAndIdNot(teamId, dto.getNumber(), playerId)) {
00141             throw new IllegalArgumentException(
00142                 "Validation Error: Team already has a player with number " + dto.getNumber() + "."
00143             );
00144         }
00145
00146         player.setName(dto.getName().trim());
00147         player.setNumber(dto.getNumber());
00148         player.setPosition(dto.getPosition());
00149         player.setTeam(team);
00150
00151         return playerRepository.save(player);
00152     }
00153
00154     public void deletePlayer(Long teamId, Long playerId) {
00155         if (teamId == null) {
00156             throw new IllegalArgumentException("Validation Error: Team ID is required.");
00157         }
00158
00159         if (playerId == null) {
00160             throw new IllegalArgumentException("Validation Error: Player ID is required.");
00161         }
00162
00163         Team team = teamRepository.findById(teamId)
00164             .orElseThrow(() -> new IllegalArgumentException("Team not found with id: " + teamId));
00165
00166         Player player = playerRepository.findByIdAndTeamId(playerId, teamId)
00167             .orElseThrow(() -> new IllegalArgumentException(
00168                 "Player not found with id: " + playerId + " for team id: " + teamId
00169             ));
00170
00171         team.getPlayers().remove(player);
00172         playerRepository.delete(player);
00173     }
00174
00175     private void validatePlayerRequest(Long teamId, PlayerRequestDTO dto) {

```

```

00176         if (teamId == null) {
00177             throw new IllegalArgumentException("Validation Error: Team ID is required.");
00178         }
00179
00180         if (dto == null) {
00181             throw new IllegalArgumentException("Validation Error: Player data is required.");
00182         }
00183
00184         if (dto.getName() == null || dto.getName().trim().isEmpty()) {
00185             throw new IllegalArgumentException("Validation Error: Player name is required.");
00186         }
00187
00188         if (dto.getName().trim().length() > 30) {
00189             throw new IllegalArgumentException("Validation Error: Player name must be at most 30
00190 characters.");
00191         }
00192
00193         if (dto.getNumber() == null) {
00194             throw new IllegalArgumentException("Validation Error: Player number is required.");
00195         }
00196
00197         if (dto.getNumber() <= 0) {
00198             throw new IllegalArgumentException("Validation Error: Player number must be greater than
00199 0.");
00200         }
00201
00202         if (dto.getPosition() == null) {
00203             throw new IllegalArgumentException("Validation Error: Player position is required.");
00204         }

```

9.114 Football_Manager/src/main/java/com/example/football_manager/service/StandingsService.java File Reference

```

import com.example.football_manager.dto.StandingDTO;
import com.example.football_manager.model.Competition;
import com.example.football_manager.model.Match;
import com.example.football_manager.model.Team;
import com.example.football_manager.repository.CompetitionRepository;
import com.example.football_manager.repository.MatchRepository;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.stereotype.Service;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

```

Classes

- class [com.example.football_manager.service.StandingsService](#)

Packages

- package [com.example.football_manager.service](#)

9.115 StandingsService.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.service;
00002
00003 import com.example.football_manager.dto.StandingDTO;
00004 import com.example.football_manager.model.Competition;
00005 import com.example.football_manager.model.Match;
00006 import com.example.football_manager.model.Team;
00007 import com.example.football_manager.repository.CompetitionRepository;
00008 import com.example.football_manager.repository.MatchRepository;

```

```

00009 import org.springframework.cache.annotation.Cacheable;
00010 import org.springframework.stereotype.Service;
00011
00012 import java.util.ArrayList;
00013 import java.util.Comparator;
00014 import java.util.LinkedHashMap;
00015 import java.util.List;
00016 import java.util.Map;
00017
00018 @Service
00019 public class StandingsService {
00020
00021     private final CompetitionRepository competitionRepository;
00022     private final MatchRepository matchRepository;
00023
00024     public StandingsService(CompetitionRepository competitionRepository,
00025                             MatchRepository matchRepository) {
00026         this.competitionRepository = competitionRepository;
00027         this.matchRepository = matchRepository;
00028     }
00029
00030     @Cacheable(cacheNames = "competitions")
00031     public List<Competition> getCompetitions() {
00032         return competitionRepository.findAll();
00033     }
00034
00035     @Cacheable(cacheNames = "standings", key = "#competitionId")
00036     public List<StandingDTO> getStandingsForCompetition(Long competitionId) {
00037         List<Match> matches = matchRepository.findByFilters(null, null, competitionId);
00038
00039         Map<Long, Row> rowsByTeamId = new LinkedHashMap<>();
00040         for (Match match : matches) {
00041             Row left = rowsByTeamId.computeIfAbsent(
00042                 match.getLeftTeam().getId(), id -> new Row(match.getLeftTeam()));
00043             Row right = rowsByTeamId.computeIfAbsent(
00044                 match.getRightTeam().getId(), id -> new Row(match.getRightTeam()));
00045
00046             if (!match.isFinished()) {
00047                 continue;
00048             }
00049
00050             int leftGoals = match.getLeftScore();
00051             int rightGoals = match.getRightScore();
00052
00053             left.matchesPlayed++;
00054             right.matchesPlayed++;
00055             left.goalsFor += leftGoals;
00056             left.goalsAgainst += rightGoals;
00057             right.goalsFor += rightGoals;
00058             right.goalsAgainst += leftGoals;
00059
00060             if (leftGoals > rightGoals) {
00061                 left.won++;
00062                 right.lost++;
00063             } else if (leftGoals < rightGoals) {
00064                 right.won++;
00065                 left.lost++;
00066             } else {
00067                 left.drawn++;
00068                 right.drawn++;
00069             }
00070         }
00071
00072         List<StandingDTO> standings = new ArrayList<>(rowsByTeamId.size());
00073         for (Row row : rowsByTeamId.values()) {
00074             int goalDifference = row.goalsFor - row.goalsAgainst;
00075             int points = row.won * 3 + row.drawn;
00076             standings.add(new StandingDTO(
00077                 row.team.getName(),
00078                 row.team.getLogoUrl(),
00079                 row.matchesPlayed,
00080                 row.won,
00081                 row.drawn,
00082                 row.lost,
00083                 row.goalsFor,
00084                 row.goalsAgainst,
00085                 goalDifference,
00086                 points
00087             ));
00088         }
00089
00090         standings.sort(Comparator
00091             .comparingInt(StandingDTO::getPoints).reversed()
00092             .thenComparing(Comparator.comparingInt(StandingDTO::getGoalDifference).reversed())
00093             .thenComparing(Comparator.comparingInt(StandingDTO::getGoalsFor).reversed())
00094             .thenComparing(StandingDTO::getTeamName));
00095

```

```

00096         return standings;
00097     }
00098
00099     private static class Row {
00100         final Team team;
00101         int matchesPlayed;
00102         int won;
00103         int drawn;
00104         int lost;
00105         int goalsFor;
00106         int goalsAgainst;
00107
00108         Row(Team team) {
00109             this.team = team;
00110         }
00111     }
00112 }

```

9.116 Football_Manager/src/main/java/com/example/football_manager/service/TeamService.java File Reference

```

import com.example.football_manager.dto.TeamRequestDTO;
import com.example.football_manager.model.Country;
import com.example.football_manager.model.Team;
import com.example.football_manager.repository.CountryRepository;
import com.example.football_manager.repository.TeamRepository;
import java.util.List;
import java.util.Optional;
import org.springframework.cache.annotation.Cacheable;
import org.springframework.cache.annotation.CacheEvict;
import org.springframework.stereotype.Service;

```

Classes

- class [com.example.football_manager.service.TeamService](#)

Packages

- package [com.example.football_manager.service](#)

9.117 TeamService.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.service;
00002
00003 import com.example.football_manager.dto.TeamRequestDTO;
00004 import com.example.football_manager.model.Country;
00005 import com.example.football_manager.model.Team;
00006 import com.example.football_manager.repository.CountryRepository;
00007 import com.example.football_manager.repository.TeamRepository;
00008
00009 import java.util.List;
00010 import java.util.Optional;
00011
00012 import org.springframework.cache.annotation.Cacheable;
00013 import org.springframework.cache.annotation.CacheEvict;
00014 import org.springframework.stereotype.Service;
00015
00016 @Service
00017 public class TeamService {
00018
00019     private final TeamRepository teamRepository;
00020     private final CountryRepository countryRepository;
00021
00022     public TeamService(TeamRepository teamRepository, CountryRepository countryRepository) {
00023         this.teamRepository = teamRepository;
00024         this.countryRepository = countryRepository;
00025     }
00026
00027     @CacheEvict(cacheNames = "teams", allEntries = true)

```

```

00028     public Team createTeam(TeamRequestDTO dto) {
00029         Country country = countryRepository.findById(dto.getCountryId())
00030             .orElseThrow(() -> new RuntimeException("Country not found with id: " +
00031                 dto.getCountryId()));
00032         Team team = new Team();
00033         team.setName(dto.getName().trim());
00034         team.setLogoUrl(dto.getLogoUrl().trim());
00035         team.setCountry(country);
00036
00037         return teamRepository.save(team);
00038     }
00039
00040     @CacheEvict(cacheNames = "teams", allEntries = true)
00041     public Team updateTeam(Long id, TeamRequestDTO dto) {
00042         Team existingTeam = teamRepository.findById(id)
00043             .orElseThrow(() -> new RuntimeException("Team not found with id: " + id));
00044
00045         Country country = countryRepository.findById(dto.getCountryId())
00046             .orElseThrow(() -> new RuntimeException("Country not found with id: " +
00047                 dto.getCountryId()));
00048
00049         existingTeam.setName(dto.getName().trim());
00050         existingTeam.setLogoUrl(dto.getLogoUrl().trim());
00051         existingTeam.setCountry(country);
00052
00053         return teamRepository.save(existingTeam);
00054     }
00055
00056     @CacheEvict(cacheNames = "teams", allEntries = true)
00057     public void deleteTeam(Long id) {
00058         Team existingTeam = teamRepository.findById(id)
00059             .orElseThrow(() -> new RuntimeException("Team not found with id: " + id));
00060
00061         teamRepository.delete(existingTeam);
00062     }
00063
00064     public Optional<Team> getTeamById(Long id) {
00065         return teamRepository.findById(id);
00066     }
00067
00068     public List<Team> getTeamsContainingName(String name) {
00069         if (name == null || name.trim().isEmpty()) {
00070             return List.of();
00071         }
00072         return teamRepository.findByNameContainingIgnoreCase(name.trim());
00073     }
00074
00075     @Cacheable(cacheNames = "teams")
00076     public List<Team> getAllTeams() {
00077         return teamRepository.findAll();
00078     }

```

9.118 Football_Manager/src/main/java/com/example/football_↵ manager/service/UserService.java File Reference

```

import com.example.football_manager.model.Team;
import com.example.football_manager.model.User;
import com.example.football_manager.repository.TeamRepository;
import com.example.football_manager.repository.UserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import java.util.Collections;
import java.util.HashSet;
import java.util.Objects;
import java.util.Set;
import java.util.stream.Collectors;

```

Classes

- class [com.example.football_manager.service.UserService](#)

Packages

- package [com.example.football_manager.service](#)

9.119 UserService.java

[Go to the documentation of this file.](#)

```

00001 package com.example.football_manager.service;
00002
00003 import com.example.football_manager.model.Team;
00004 import com.example.football_manager.model.User;
00005 import com.example.football_manager.repository.TeamRepository;
00006 import com.example.football_manager.repository.UserRepository;
00007 import org.springframework.beans.factory.annotation.Autowired;
00008 import org.springframework.security.crypto.password.PasswordEncoder;
00009 import org.springframework.stereotype.Service;
00010
00011 import java.util.Collections;
00012 import java.util.HashSet;
00013 import java.util.Objects;
00014 import java.util.Set;
00015 import java.util.stream.Collectors;
00016
00017 @Service
00018 public class UserService {
00019
00020     @Autowired
00021     private UserRepository userRepository;
00022
00023     @Autowired
00024     private TeamRepository teamRepository;
00025
00026     @Autowired
00027     private PasswordEncoder passwordEncoder;
00028
00029     public User registerUser(String username, String email, String rawPassword, Boolean isAdmin) {
00030         if (userRepository.existsByUsername(username)) {
00031             throw new IllegalArgumentException("Username already in use");
00032         }
00033
00034         if (userRepository.existsByEmail(email)) {
00035             throw new IllegalArgumentException("Email already in use");
00036         }
00037
00038         User user = new User();
00039         user.setUsername(username);
00040         user.setEmail(email);
00041         user.setHashedPassword(passwordEncoder.encode(rawPassword));
00042         user.setIsAdmin(Boolean.TRUE.equals(isAdmin));
00043
00044         return userRepository.save(user);
00045     }
00046
00047     public User login(String username, String rawPassword) {
00048         User user = userRepository.findByUsername(username)
00049             .orElseThrow(() -> new IllegalArgumentException("Invalid credentials"));
00050
00051         if (!passwordEncoder.matches(rawPassword, user.getHashedPassword())) {
00052             throw new IllegalArgumentException("Invalid credentials");
00053         }
00054
00055         return user;
00056     }
00057
00058     public User getUserById(Long id) {
00059         return userRepository.findById(id)
00060             .orElseThrow(() -> new IllegalArgumentException("User not found"));
00061     }
00062
00063     public User getUserByUsername(String username) {
00064         return userRepository.findByUsername(username)
00065             .orElseThrow(() -> new IllegalArgumentException("User not found"));
00066     }
00067
00068     public User addFavouriteTeam(Long userId, Long teamId) {
00069         User user = getUserById(userId);
00070         Team team = teamRepository.findById(teamId)
00071             .orElseThrow(() -> new IllegalArgumentException("Team not found"));
00072
00073         Set<Team> favouriteTeams = user.getFavouriteTeams();
00074         if (favouriteTeams == null) {
00075             favouriteTeams = new HashSet<>();
00076             user.setFavouriteTeams(favouriteTeams);

```

```

00077     }
00078
00079     if (!favouriteTeams.add(team)) {
00080         throw new IllegalArgumentException("Team already in favourites");
00081     }
00082
00083     return userRepository.save(user);
00084 }
00085
00086 public User removeFavouriteTeam(Long userId, Long teamId) {
00087     User user = getUserById(userId);
00088
00089     if (!teamRepository.existsById(teamId)) {
00090         throw new IllegalArgumentException("Team not found");
00091     }
00092
00093     Set<Team> favouriteTeams = user.getFavouriteTeams();
00094     if (favouriteTeams == null || favouriteTeams.isEmpty()) {
00095         throw new IllegalArgumentException("Team not in favourites");
00096     }
00097
00098     boolean removed = favouriteTeams.removeIf(team -> teamId.equals(team.getId()));
00099     if (!removed) {
00100         throw new IllegalArgumentException("Team not in favourites");
00101     }
00102
00103     return userRepository.save(user);
00104 }
00105
00106 public Set<Long> getFavouriteTeamIdsByUserId(Long userId) {
00107     if (userId == null) {
00108         return Collections.emptySet();
00109     }
00110
00111     User user = getUserById(userId);
00112     Set<Team> favouriteTeams = user.getFavouriteTeams();
00113     if (favouriteTeams == null || favouriteTeams.isEmpty()) {
00114         return Collections.emptySet();
00115     }
00116
00117     return favouriteTeams.stream()
00118         .map(Team::getId)
00119         .collect(Collectors.toSet());
00120 }
00121
00122 public User getUserProfile(Long userId) {
00123     return userRepository.findWithFavouriteTeamsById(userId)
00124         .orElseThrow(() -> new IllegalArgumentException("User not found"));
00125 }
00126
00127 public User updateProfile(Long userId,
00128     String username,
00129     String email,
00130     String currentPassword,
00131     String newPassword,
00132     String confirmNewPassword) {
00133     User user = getUserById(userId);
00134
00135     String normalizedUsername = normalizeField(username, "Username is required");
00136     String normalizedEmail = normalizeField(email, "Email is required");
00137
00138     if (userRepository.existsByUsernameAndIdNot(normalizedUsername, userId)) {
00139         throw new IllegalArgumentException("Username already in use");
00140     }
00141
00142     if (userRepository.existsByEmailAndIdNot(normalizedEmail, userId)) {
00143         throw new IllegalArgumentException("Email already in use");
00144     }
00145
00146     if (!Objects.equals(user.getUsername(), normalizedUsername)) {
00147         user.setUsername(normalizedUsername);
00148     }
00149
00150     if (!Objects.equals(user.getEmail(), normalizedEmail)) {
00151         user.setEmail(normalizedEmail);
00152     }
00153
00154     if (isPasswordChangeRequested(currentPassword, newPassword, confirmNewPassword)) {
00155         if (isBlank(currentPassword)) {
00156             throw new IllegalArgumentException("Current password is required");
00157         }
00158         if (isBlank(newPassword) || isBlank(confirmNewPassword)) {
00159             throw new IllegalArgumentException("New password and confirmation are required");
00160         }
00161         if (!newPassword.equals(confirmNewPassword)) {
00162             throw new IllegalArgumentException("New passwords do not match");
00163         }
00164     }

```



```
00164         if (newPassword.length() < 8) {
00165             throw new IllegalArgumentException("New password must be at least 8 characters");
00166         }
00167         if (!passwordEncoder.matches(currentPassword, user.getHashedPassword())) {
00168             throw new IllegalArgumentException("Current password is incorrect");
00169         }
00170         user.setHashedPassword(passwordEncoder.encode(newPassword));
00171     }
00172     }
00173     return userRepository.save(user);
00174 }
00175 }
00176 private boolean isPasswordChangeRequested(String currentPassword, String newPassword, String
00177 confirmNewPassword) {
00178     return !isBlank(currentPassword) || !isBlank(newPassword) || !isBlank(confirmNewPassword);
00179 }
00180 private boolean isBlank(String value) {
00181     return value == null || value.isBlank();
00182 }
00183 }
00184 private String normalizeField(String value, String message) {
00185     if (value == null || value.trim().isEmpty()) {
00186         throw new IllegalArgumentException(message);
00187     }
00188     return value.trim();
00189 }
00190 }
00191 }
```

9.120 README.md File Reference

Index

addFavouriteTeam
 com.example.football_manager.controller.FavouriteTeamController, 37
 com.example.football_manager.service.UserService, 74

addInterceptors
 com.example.football_manager.config.LocaleConfig, 39

addIsAdmin
 com.example.football_manager.controller.GlobalModelAttribute, 37

AdminCountryViewController
 com.example.football_manager.controller.AdminCountryViewController, 23

adminPanel
 com.example.football_manager.controller.AdminViewController, 27

AdminPlayerViewController
 com.example.football_manager.controller.AdminPlayerViewController, 24

AdminUserViewController
 com.example.football_manager.controller.AdminUserViewController, 26

AuthResponseDTO
 com.example.football_manager.dto.AuthResponseDTO, 28

cacheManager
 com.example.football_manager.config.CacheConfig, 30

CANCELLED
 com.example.football_manager.dto.MatchRequestDTO.MatchStatus, 49

com.example.football_manager, 19

com.example.football_manager.config, 19

com.example.football_manager.config.AdminSecurityFilter, 26

 doFilter, 26

com.example.football_manager.config.CacheConfig, 30

 cacheManager, 30

com.example.football_manager.config.LocaleConfig, 39

 addInterceptors, 39

 localeChangeInterceptor, 39

 localeResolver, 39

com.example.football_manager.config.OpenApiConfig, 52

com.example.football_manager.config.SecurityConfig, 61

 filterChain, 61

 passwordEncoder, 61

com.example.football_manager.controller, 19

com.example.football_manager.controller.AdminCountryViewController, 23

 AdminCountryViewController, 23

 countriesPage, 23

 createCountry, 23

 deleteCountry, 23

com.example.football_manager.controller.AdminPlayerViewController, 24

 AdminPlayerViewController, 24

 createPlayer, 24

 deletePlayer, 24

 manageTeamPlayers, 25

 showEditPlayerForm, 25

com.example.football_manager.controller.AdminUserViewController, 26

 AdminUserViewController, 26

 createUser, 27

 usersPage, 27

com.example.football_manager.controller.AdminViewController, 27

 adminPanel, 27

 manageTeams, 27

com.example.football_manager.controller.AuthController, 28

 login, 28

 register, 28

com.example.football_manager.controller.AuthViewController, 29

 loginPage, 29

 registerPage, 29

com.example.football_manager.controller.CompetitionController, 30

 CompetitionController, 31

 createCompetition, 31

 deleteCompetition, 31

 getAllCompetitions, 31

 getCompetitionById, 31

 updateCompetition, 31

com.example.football_manager.controller.CountryController, 34

 CountryController, 34

 createCountry, 34

 deleteCountry, 34

 getAllCountries, 34

com.example.football_manager.controller.FavouriteTeamController, 36
 addFavouriteTeam, 37
 FavouriteTeamController, 36
 removeFavouriteTeam, 37
 com.example.football_manager.controller.GlobalModelAttributes, 37
 addIsAdmin, 37
 com.example.football_manager.controller.MatchController, 40
 createMatchFromForm, 41
 createMatchFromJson, 41
 deleteMatch, 41
 getMatches, 41
 getMatchResults, 41
 getUpcomingMatches, 41
 registerResult, 41
 updateMatch, 41
 com.example.football_manager.controller.MatchViewController, 50
 MatchViewController, 50
 saveMatchResult, 51
 showEditForm, 51
 showMatchesPage, 51
 showResultsPage, 51
 showScheduleForm, 51
 showScoreForm, 51
 showUpcomingMatchesPage, 51
 submitScoreForm, 52
 updateMatch, 52
 com.example.football_manager.controller.PlayerController, 53
 createPlayer, 54
 createPlayers, 54
 deletePlayer, 54
 getPlayersByTeamId, 54
 PlayerController, 53
 updatePlayer, 54
 com.example.football_manager.controller.ProfileViewController, 59
 profilePage, 59
 ProfileViewController, 59
 updateProfile, 59
 com.example.football_manager.controller.StandingsController, 64
 getStandings, 65
 StandingsController, 65
 com.example.football_manager.controller.StandingsViewController, 66
 standingsPage, 66
 StandingsViewController, 66
 com.example.football_manager.controller.TeamController, 67
 createTeam, 67
 deleteTeam, 67
 getAllTeams, 67
 getTeamById, 67
 getTeamsContainingName, 68
 TeamController, 67
 updateTeam, 68
 com.example.football_manager.controller.TeamViewController, 71
 createTeam, 71
 deleteTeam, 71
 showAddTeamForm, 72
 showEditTeamForm, 72
 teamDetailsPage, 72
 teamsPage, 72
 updateTeam, 72
 com.example.football_manager.dto, 20
 MatchResultDTO, 20
 com.example.football_manager.dto.AuthResponseDTO, 28
 AuthResponseDTO, 28
 getEmail, 29
 getId, 29
 getIsAdmin, 29
 getMessage, 29
 getUsername, 29
 com.example.football_manager.dto.CompetitionRequestDTO, 32
 CompetitionRequestDTO, 32
 getName, 32
 setName, 32
 com.example.football_manager.dto.CountryRequestDTO, 35
 CountryRequestDTO, 35
 getName, 35
 setName, 35
 com.example.football_manager.dto.LoginDTO, 39
 getPassword, 40
 getUsername, 40
 setPassword, 40
 setUsername, 40
 com.example.football_manager.dto.MatchRequestDTO, 44
 getAwayScore, 44
 getAwayTeamId, 44
 getCompetitionId, 45
 getHomeScore, 45
 getHomeTeamId, 45
 getKickoffTime, 45
 getStatus, 45
 getVenue, 45
 setAwayScore, 45
 setAwayTeamId, 45
 setCompetitionId, 45
 setHomeScore, 45
 setHomeTeamId, 46
 setKickoffTime, 46
 setStatus, 46
 setVenue, 46
 com.example.football_manager.dto.MatchRequestDTO.MatchStatus, 49
 CANCELLED, 49
 FINISHED, 49

- IN_PROGRESS, 50
- SCHEDULED, 50
- com.example.football_manager.dto.MatchResultRequestDTO, 46
 - getGoals, 46
 - setGoals, 46
- com.example.football_manager.dto.MatchResultRequestDTO.GoalDTO, 38
 - getMinute, 38
 - getStoppageMinute, 38
 - getTeamId, 38
 - setMinute, 38
 - setStoppageMinute, 38
 - setTeamId, 38
- com.example.football_manager.dto.PlayerBulkRequestDTO, 52
 - getPlayers, 53
 - PlayerBulkRequestDTO, 53
 - setPlayers, 53
- com.example.football_manager.dto.PlayerRequestDTO, 56
 - getName, 57
 - getNumber, 57
 - getPosition, 57
 - PlayerRequestDTO, 56
 - setName, 57
 - setNumber, 57
 - setPosition, 57
- com.example.football_manager.dto.ProfileUpdateDTO, 59
 - isPasswordConfirmationValid, 59
- com.example.football_manager.dto.RegisterDTO, 60
 - getEmail, 60
 - getIsAdmin, 60
 - getPassword, 60
 - getUsername, 60
 - setEmail, 60
 - setIsAdmin, 60
 - setPassword, 61
 - setUsername, 61
- com.example.football_manager.dto.StandingDTO, 61
 - getDrawn, 62
 - getGoalDifference, 62
 - getGoalsAgainst, 63
 - getGoalsFor, 63
 - getLogoUrl, 63
 - getLost, 63
 - getMatchesPlayed, 63
 - getPoints, 63
 - getTeamName, 63
 - getWon, 63
 - setDrawn, 63
 - setGoalDifference, 63
 - setGoalsAgainst, 63
 - setGoalsFor, 64
 - setLogoUrl, 64
 - setLost, 64
 - setMatchesPlayed, 64
 - setPoints, 64
 - setTeamName, 64
 - setWon, 64
 - StandingDTO, 62
- com.example.football_manager.dto.TeamRequestDTO, 69
 - getCountryId, 69
 - getLogoUrl, 69
 - getName, 69
 - setCountryId, 69
 - setLogoUrl, 69
 - setName, 70
 - TeamRequestDTO, 69
- com.example.football_manager.FootballManagerApplication, 37
 - main, 37
- com.example.football_manager.model, 20
 - com.example.football_manager.model.Competition, 30
 - com.example.football_manager.model.Country, 33
 - com.example.football_manager.model.Match, 40
 - com.example.football_manager.model.MatchGoal, 42
 - com.example.football_manager.model.Player, 52
 - com.example.football_manager.model.PlayerPosition, 54
 - DEFENDER, 55
 - FORWARD, 55
 - GOALKEEPER, 55
 - MIDFIELDER, 55
 - com.example.football_manager.model.Team, 66
 - Team, 67
 - com.example.football_manager.model.User, 73
- com.example.football_manager.repository, 21
 - com.example.football_manager.repository.CompetitionRepository, 31
 - com.example.football_manager.repository.CountryRepository, 34
 - com.example.football_manager.repository.MatchGoalRepository, 42
 - deleteByMatchId, 42
 - findByMatchIdOrderByMinuteAscStoppageMinuteAscIdAsc, 42
 - com.example.football_manager.repository.MatchRepository, 42
 - findByFilters, 43
 - findByFinishedFalseAndDatetimeAfterAndLeftTeamIdInOrFinishedFalseAndDatetimeAfterAndRightTeamIdInOrFinishedFalseAndDatetimeAfterAndLeftTeamIdOrFinishedFalseAndDatetimeAfterAndRightTeamIdOrderByDatetimeDesc, 43
 - findByFinishedTrueAndLeftTeamIdOrFinishedTrueAndRightTeamIdOrderByDatetimeDesc, 43
 - findByFinishedTrueOrderByDatetimeDesc, 44
- com.example.football_manager.repository.PlayerRepository, 55
 - existsByTeamIdAndNumber, 55
 - existsByTeamIdAndNumberAndIdNot, 55
 - findByIdAndTeamId, 56

- findByTeamIdOrderByNumberAsc, 56
- com.example.football_manager.repository.TeamRepository, 68
 - findByTeamIdOrderByNumberAsc, 56
 - findByNameContainingIgnoreCase, 68
- com.example.football_manager.repository.UserRepository, 73
 - existsByEmail, 73
 - existsByEmailAndIdNot, 73
 - existsByUsername, 73
 - existsByUsernameAndIdNot, 73
 - findByEmail, 74
 - findByUsername, 74
 - findWithFavouriteTeamsById, 74
- com.example.football_manager.service, 21
- com.example.football_manager.service.CompetitionService, 32
 - CompetitionService, 33
 - createCompetition, 33
 - deleteCompetition, 33
 - getAllCompetitions, 33
 - getCompetitionById, 33
 - updateCompetition, 33
- com.example.football_manager.service.CountryService, 35
 - CountryService, 36
 - createCountry, 36
 - deleteCountry, 36
 - getAllCountries, 36
- com.example.football_manager.service.MatchService, 47
 - createMatch, 47
 - deleteMatch, 47
 - getAllMatches, 48
 - getFinishedMatchResults, 48
 - getFutureMatchesByFavouriteTeamIds, 48
 - getFutureMatchesByTeamId, 48
 - getGoalsByMatchId, 48
 - getMatchById, 48
 - getMatches, 48
 - getMatchForEdit, 48
 - getRecentMatchesByTeamId, 49
 - getUpcomingMatches, 49
 - MatchService, 47
 - registerResult, 49
 - updateMatch, 49
- com.example.football_manager.service.PlayerService, 57
 - createPlayer, 58
 - createPlayers, 58
 - deletePlayer, 58
 - getPlayerByTeamId, 58
 - getPlayersByTeamId, 58
 - PlayerService, 58
 - updatePlayer, 58
- com.example.football_manager.service.StandingsService, 65
 - getCompetitions, 65
 - getStandingsForCompetition, 65
- StandingsService, 65
 - createTeam, 70
 - deleteTeam, 70
 - getAllTeams, 70
 - getTeamById, 71
 - getTeamsContainingName, 71
 - TeamService, 70
 - updateTeam, 71
- com.example.football_manager.service.UserService, 74
 - addFavouriteTeam, 74
 - getFavouriteTeamIdsByUserId, 74
 - getUserById, 75
 - getUserByUsername, 75
 - getUserProfile, 75
 - login, 75
 - registerUser, 75
 - removeFavouriteTeam, 75
 - updateProfile, 75
- CompetitionController
 - com.example.football_manager.controller.CompetitionController, 31
- CompetitionRequestDTO
 - com.example.football_manager.dto.CompetitionRequestDTO, 32
- CompetitionService
 - com.example.football_manager.service.CompetitionService, 33
- countriesPage
 - com.example.football_manager.controller.AdminCountryViewControll, 23
- CountryController
 - com.example.football_manager.controller.CountryController, 34
- CountryRequestDTO
 - com.example.football_manager.dto.CountryRequestDTO, 35
- CountryService
 - com.example.football_manager.service.CountryService, 36
- createCompetition
 - com.example.football_manager.controller.CompetitionController, 31
 - com.example.football_manager.service.CompetitionService, 33
- createCountry
 - com.example.football_manager.controller.AdminCountryViewControll, 23
 - com.example.football_manager.controller.CountryController, 34
 - com.example.football_manager.service.CountryService, 36
- createMatch
 - com.example.football_manager.service.MatchService, 47
- createMatchFromForm

- com.example.football_manager.controller.MatchController, com.example.football_manager.controller.TeamController, 41 67
- createMatchFromJson com.example.football_manager.controller.TeamViewController, 71
- com.example.football_manager.controller.MatchController, 41 com.example.football_manager.service.TeamService, 70
- createPlayer com.example.football_manager.controller.AdminPlayerController, 24 doFilter 77
- com.example.football_manager.controller.PlayerController, com.example.football_manager.config.AdminSecurityFilter, 54 26
- com.example.football_manager.service.PlayerService, doxygen_mainpage, 9 58
- createPlayers existsByEmail com.example.football_manager.controller.PlayerController, com.example.football_manager.repository.UserRepository, 54 73
- com.example.football_manager.service.PlayerService, existsByEmailAndIdNot com.example.football_manager.repository.UserRepository, 58 73
- createTeam existsByTeamIdAndNumber com.example.football_manager.controller.TeamController, 67 com.example.football_manager.repository.PlayerRepository, 55
- com.example.football_manager.controller.TeamViewController, 71 existsByTeamIdAndNumberAndIdNot com.example.football_manager.repository.PlayerRepository, 70 55
- com.example.football_manager.service.TeamService, existsByUsername com.example.football_manager.controller.AdminUserViewController, com.example.football_manager.repository.UserRepository, 27 73
- DEFENDER existsByUsernameAndIdNot com.example.football_manager.repository.UserRepository, 55 73
- com.example.football_manager.model.PlayerPosition, 55
- deleteByMatchId FavouriteTeamController com.example.football_manager.repository.MatchGoalRepository, com.example.football_manager.controller.FavouriteTeamController, 42 36
- deleteCompetition filterChain com.example.football_manager.controller.CompetitionController, com.example.football_manager.config.SecurityConfig, 31 61
- com.example.football_manager.service.CompetitionService, findByEmail com.example.football_manager.repository.UserRepository, 33 74
- deleteCountry com.example.football_manager.controller.AdminCountryController, 23 com.example.football_manager.repository.MatchRepository, 43
- com.example.football_manager.controller.CountryController, 34 findByFinishedFalseAndDatetimeAfterAndLeftTeamIdInOrFinishedFalseAnd com.example.football_manager.service.CountryService, com.example.football_manager.repository.MatchRepository, 36 43
- deleteMatch findByFinishedFalseAndDatetimeAfterAndLeftTeamIdOrFinishedFalseAnd com.example.football_manager.controller.MatchController, com.example.football_manager.repository.MatchRepository, 41 43
- com.example.football_manager.service.MatchService, findByFinishedTrueAndLeftTeamIdOrFinishedTrueAndRightTeamIdOrderBy com.example.football_manager.repository.MatchRepository, 47 43
- deletePlayer com.example.football_manager.controller.AdminPlayerController, 24 com.example.football_manager.repository.MatchRepository, 44
- com.example.football_manager.controller.PlayerController, 54 findByIdAndTeamId com.example.football_manager.service.PlayerService, com.example.football_manager.repository.PlayerRepository, 58 56
- deleteTeam findByMatchIdOrderByMinuteAscStoppageMinuteAscIdAsc

com.example.football_manager.repository.MatchGoalRepository, 104, 105
 42
 com.example.football_manager.repository.TeamRepository, 105, 106
 68
 com.example.football_manager.repository.PlayerRepository, 107, 108
 56
 com.example.football_manager.repository.UserRepository, 109, 110
 74
 com.example.football_manager.repository.UserRepository, 110, 111
 74
 com.example.football_manager.repository.UserRepository, 111
 74
 com.example.football_manager.repository.UserRepository, 112
 74
 com.example.football_manager.dto.MatchRequestDTO.MatchStatus, 112, 113
 49
 com.example.football_manager.dto.MatchStatus, 114
 77
 com.example.football_manager.dto.MatchStatus, 114, 115
 78
 com.example.football_manager.dto.MatchStatus, 115, 116
 79
 com.example.football_manager.dto.MatchStatus, 116
 80
 com.example.football_manager.dto.MatchStatus, 117, 118
 81
 com.example.football_manager.dto.MatchStatus, 118, 119
 81, 82
 com.example.football_manager.dto.MatchStatus, 119
 82, 83
 com.example.football_manager.dto.MatchStatus, 121
 85
 com.example.football_manager.dto.MatchStatus, 122
 85, 86
 com.example.football_manager.dto.MatchStatus, 122, 123
 86, 87
 com.example.football_manager.dto.MatchStatus, 123
 88
 com.example.football_manager.dto.MatchStatus, 124
 89
 com.example.football_manager.dto.MatchStatus, 124, 125
 90, 91
 com.example.football_manager.dto.MatchStatus, 125, 126
 92
 com.example.football_manager.dto.MatchStatus, 126
 93, 94
 com.example.football_manager.dto.MatchStatus, 127
 94, 95
 com.example.football_manager.dto.MatchStatus, 128
 97
 com.example.football_manager.dto.MatchStatus, 128, 129
 99, 100
 com.example.football_manager.dto.MatchStatus, 129
 101, 102
 com.example.football_manager.dto.MatchStatus, 129, 130
 103, 104
 com.example.football_manager.dto.MatchStatus, 130

Football_Manager/src/main/java/com/example/football_manager/controller/PlayerRepository.java,
131
com.example.football_manager.dto.TeamRequestDTO,
Football_Manager/src/main/java/com/example/football_manager/repository/TeamRepository.java,
132
getDrawn
Football_Manager/src/main/java/com/example/football_manager/repository/TeamRepository.java,
132
getStandingByTeamId
Football_Manager/src/main/java/com/example/football_manager/service/CompetitionService.java,
133
com.example.football_manager.dto.AuthResponseDTO,
Football_Manager/src/main/java/com/example/football_manager/service/CountryService.java,
134
com.example.football_manager.dto.RegisterDTO,
Football_Manager/src/main/java/com/example/football_manager/service/MatchService.java,
135
getFavouriteTeamIdsByUserId
Football_Manager/src/main/java/com/example/football_manager/service/MatchService.java,
140
com.example.football_manager.service.UserService,
Football_Manager/src/main/java/com/example/football_manager/service/MatchService.java,
143
getFutureMatchesByFavouriteTeamIds
Football_Manager/src/main/java/com/example/football_manager/service/MatchService.java,
145
getFutureMatchesByTeamId
Football_Manager/src/main/java/com/example/football_manager/service/MatchService.java,
146, 147
getFutureMatchesByTeamId
FORWARD
com.example.football_manager.model.PlayerPosition,
55
com.example.football_manager.service.MatchService,
48
getGoalDifference
com.example.football_manager.dto.StandingDTO,
getAllCompetitions
com.example.football_manager.controller.CompetitionController,
31
62
getGoals
com.example.football_manager.service.CompetitionService,
33
46
com.example.football_manager.dto.MatchResultRequestDTO,
getGoalsAgainst
com.example.football_manager.dto.StandingDTO,
getGoalsByMatchId
com.example.football_manager.service.MatchService,
36
48
getGoalsFor
com.example.football_manager.dto.StandingDTO,
getAllMatches
com.example.football_manager.service.MatchService,
48
63
getHomeScore
com.example.football_manager.dto.MatchRequestDTO,
getHomeTeamId
com.example.football_manager.dto.MatchRequestDTO,
44
45
getId
com.example.football_manager.dto.AuthResponseDTO,
getAwayTeamId
com.example.football_manager.dto.MatchRequestDTO,
44
29
getIsAdmin
com.example.football_manager.dto.AuthResponseDTO,
getCompetitionById
com.example.football_manager.controller.CompetitionController,
31
29
com.example.football_manager.dto.RegisterDTO,
com.example.football_manager.service.CompetitionService,
33
60
getKickoffTime
com.example.football_manager.dto.MatchRequestDTO,
45
45
getLogoUrl
com.example.football_manager.dto.StandingDTO,
65
63

com.example.football_manager.dto.TeamRequestDTO, 69
 getPosition, 57
 com.example.football_manager.dto.PlayerRequestDTO, 57
 getLost, 63
 com.example.football_manager.dto.StandingDTO, 63
 getMatchById, 49
 com.example.football_manager.service.MatchService, 49
 getStandings, 65
 com.example.football_manager.controller.StandingsController, 65
 getMatches, 41
 com.example.football_manager.controller.MatchController, 41
 getStandingsForCompetition, 65
 com.example.football_manager.service.StandingsService, 65
 com.example.football_manager.service.MatchService, 48
 getStatus, 45
 com.example.football_manager.dto.MatchRequestDTO, 45
 getMatchesPlayed, 63
 com.example.football_manager.dto.StandingDTO, 63
 getStoppageMinute, 38
 com.example.football_manager.dto.MatchResultRequestDTO.GoalDTO, 38
 getMatchForEdit, 48
 com.example.football_manager.service.MatchService, 48
 getTeamById, 67
 com.example.football_manager.controller.TeamController, 67
 com.example.football_manager.service.TeamService, 71
 getMatchResults, 41
 com.example.football_manager.controller.MatchController, 41
 getMessage, 29
 com.example.football_manager.dto.AuthResponseDTO, 29
 getTeamId, 38
 com.example.football_manager.dto.MatchResultRequestDTO.GoalDTO, 38
 getMinute, 38
 com.example.football_manager.dto.MatchResultRequestDTO.GoalDTO, 38
 com.example.football_manager.dto.StandingDTO, 63
 getName, 32
 com.example.football_manager.dto.CompetitionRequestDTO, 32
 getTeamsContainingName, 68
 com.example.football_manager.controller.TeamController, 68
 com.example.football_manager.dto.CountryRequestDTO, 35
 com.example.football_manager.service.TeamService, 71
 com.example.football_manager.dto.PlayerRequestDTO, 57
 getUpcomingMatches, 41
 com.example.football_manager.dto.TeamRequestDTO, 69
 com.example.football_manager.controller.MatchController, 41
 com.example.football_manager.service.MatchService, 49
 getNumber, 57
 com.example.football_manager.dto.PlayerRequestDTO, 57
 getUserById, 75
 com.example.football_manager.service.UserService, 75
 getPassword, 60
 com.example.football_manager.dto.LoginDTO, 40
 com.example.football_manager.dto.RegisterDTO, 60
 getUserByUsername, 75
 com.example.football_manager.service.UserService, 75
 getPlayerByTeamId, 58
 com.example.football_manager.service.PlayerService, 58
 getUsername, 29
 com.example.football_manager.dto.AuthResponseDTO, 29
 getPlayers, 53
 com.example.football_manager.dto.PlayerBulkRequestDTO, 53
 com.example.football_manager.dto.LoginDTO, 40
 com.example.football_manager.dto.RegisterDTO, 60
 getPlayerByTeamId, 54
 com.example.football_manager.controller.PlayerController, 54
 getUserProfile, 75
 com.example.football_manager.service.UserService, 75
 com.example.football_manager.service.PlayerService, 58
 getVenue, 45
 com.example.football_manager.dto.MatchRequestDTO, 45
 getPoints, 63
 com.example.football_manager.dto.StandingDTO, 63
 getWon

- com.example.football_manager.dto.StandingDTO, 63
- GOALKEEPER
 - com.example.football_manager.model.PlayerPosition, 55
- IN_PROGRESS
 - com.example.football_manager.dto.MatchRequestDTO.MatchStatus, 50
- index
 - com.example.football_manager.controller.AuthViewController, 29
- isPasswordConfirmationValid
 - com.example.football_manager.dto.ProfileUpdateDTO, 59
- localeChangeInterceptor
 - com.example.football_manager.config.LocaleConfig, 39
- localeResolver
 - com.example.football_manager.config.LocaleConfig, 39
- login
 - com.example.football_manager.controller.AuthController, 28
 - com.example.football_manager.service.UserService, 75
- loginPage
 - com.example.football_manager.controller.AuthViewController, 29
- main
 - com.example.football_manager.FootballManagerApplication, 37
- managePlayersLanding
 - com.example.football_manager.controller.AdminPlayerViewController, 25
- manageTeamPlayers
 - com.example.football_manager.controller.AdminPlayerViewController, 25
- manageTeams
 - com.example.football_manager.controller.AdminViewController, 27
- MatchResultDTO
 - com.example.football_manager.dto, 20
- MatchService
 - com.example.football_manager.service.MatchService, 47
- MatchViewController
 - com.example.football_manager.controller.MatchViewController, 50
- MIDFIELDER
 - com.example.football_manager.model.PlayerPosition, 55
- passwordEncoder
 - com.example.football_manager.config.SecurityConfig, 61
- PlayerBulkRequestDTO
 - com.example.football_manager.dto.PlayerBulkRequestDTO, 53
- PlayerController
 - com.example.football_manager.controller.PlayerController, 53
- PlayerRequestDTO
 - com.example.football_manager.dto.PlayerRequestDTO, 53
- PlayerService
 - com.example.football_manager.service.PlayerService, 58
- profilePage
 - com.example.football_manager.controller.ProfileViewController, 59
- ProfileViewController
 - com.example.football_manager.controller.ProfileViewController, 59
- README.md, 149
- register
 - com.example.football_manager.controller.AuthController, 28
- registerPage
 - com.example.football_manager.controller.AuthViewController, 29
- registerResult
 - com.example.football_manager.controller.MatchController, 41
- com.example.football_manager.service.MatchService, 49
- registerUser
 - com.example.football_manager.service.UserService, 75
- removeFavouriteTeam
 - com.example.football_manager.controller.FavouriteTeamController, 37
- com.example.football_manager.service.UserService, 75
- saveMatchResult
 - com.example.football_manager.controller.MatchViewController, 51
- SCHEDULED
 - com.example.football_manager.dto.MatchRequestDTO.MatchStatus, 50
- setAwayScore
 - com.example.football_manager.dto.MatchRequestDTO, 45
- setAwayTeamId
 - com.example.football_manager.dto.MatchRequestDTO, 45
- setCompetitionId
 - com.example.football_manager.dto.MatchRequestDTO, 45
- setCountryId
 - com.example.football_manager.dto.TeamRequestDTO, 69
- setDrawn

com.example.football_manager.dto.StandingDTO, 63
 setEmail
 com.example.football_manager.dto.RegisterDTO, 60
 setGoalDifference
 com.example.football_manager.dto.StandingDTO, 63
 setGoals
 com.example.football_manager.dto.MatchResultRequestDTO, 46
 setGoalsAgainst
 com.example.football_manager.dto.StandingDTO, 63
 setGoalsFor
 com.example.football_manager.dto.StandingDTO, 64
 setHomeScore
 com.example.football_manager.dto.MatchRequestDTO, 45
 setHomeTeamId
 com.example.football_manager.dto.MatchRequestDTO, 46
 setIsAdmin
 com.example.football_manager.dto.RegisterDTO, 60
 setKickoffTime
 com.example.football_manager.dto.MatchRequestDTO, 46
 setLogoUrl
 com.example.football_manager.dto.StandingDTO, 64
 com.example.football_manager.dto.TeamRequestDTO, 69
 setLost
 com.example.football_manager.dto.StandingDTO, 64
 setMatchesPlayed
 com.example.football_manager.dto.StandingDTO, 64
 setMinute
 com.example.football_manager.dto.MatchResultRequestDTO.GoalIDTO, 38
 setName
 com.example.football_manager.dto.CompetitionRequestDTO, 32
 com.example.football_manager.dto.CountryRequestDTO, 35
 com.example.football_manager.dto.PlayerRequestDTO, 57
 com.example.football_manager.dto.TeamRequestDTO, 70
 setNumber
 com.example.football_manager.dto.PlayerRequestDTO, 57
 setPassword
 com.example.football_manager.dto.LoginDTO, 40
 com.example.football_manager.dto.RegisterDTO, 61
 setPlayers
 com.example.football_manager.dto.PlayerBulkRequestDTO, 53
 setPoints
 com.example.football_manager.dto.StandingDTO, 64
 setPosition
 com.example.football_manager.dto.PlayerRequestDTO, 57
 setStatus
 com.example.football_manager.dto.MatchRequestDTO, 46
 setStoppageMinute
 com.example.football_manager.dto.MatchResultRequestDTO.GoalIDTO, 38
 setTeamId
 com.example.football_manager.dto.MatchResultRequestDTO.GoalIDTO, 38
 setTeamName
 com.example.football_manager.dto.StandingDTO, 64
 setUsername
 com.example.football_manager.dto.LoginDTO, 40
 com.example.football_manager.dto.RegisterDTO, 61
 setVenue
 com.example.football_manager.dto.MatchRequestDTO, 46
 setWon
 com.example.football_manager.dto.StandingDTO, 64
 showAddTeamForm
 com.example.football_manager.controller.TeamViewController, 72
 showEditForm
 com.example.football_manager.controller.MatchViewController, 51
 showEditPlayerForm
 com.example.football_manager.controller.AdminPlayerViewController, 65
 showEditTeamForm
 com.example.football_manager.controller.TeamViewController, 72
 showMatchesPage
 com.example.football_manager.controller.MatchViewController, 51
 showResultsPage
 com.example.football_manager.controller.MatchViewController, 51
 showScheduleForm
 com.example.football_manager.controller.MatchViewController, 51
 showScoreForm
 com.example.football_manager.controller.MatchViewController, 51
 showUpcomingMatchesPage

[com.example.football_manager.controller.MatchViewController](#),
[51](#)
[StandingDTO](#)
[com.example.football_manager.dto.StandingDTO](#),
[62](#)
[StandingsController](#)
[com.example.football_manager.controller.StandingsController](#),
[65](#)
[standingsPage](#)
[com.example.football_manager.controller.StandingsViewController](#),
[66](#)
[StandingsService](#)
[com.example.football_manager.service.StandingsService](#),
[65](#)
[StandingsViewController](#)
[com.example.football_manager.controller.StandingsViewController](#),
[66](#)
[submitScoreForm](#)
[com.example.football_manager.controller.MatchViewController](#),
[52](#)

[Team](#)
[com.example.football_manager.model.Team](#), [67](#)
[TeamController](#)
[com.example.football_manager.controller.TeamController](#),
[67](#)
[teamDetailsPage](#)
[com.example.football_manager.controller.TeamViewController](#),
[72](#)
[TeamRequestDTO](#)
[com.example.football_manager.dto.TeamRequestDTO](#),
[69](#)
[TeamService](#)
[com.example.football_manager.service.TeamService](#),
[70](#)
[teamsPage](#)
[com.example.football_manager.controller.TeamViewController](#),
[72](#)

[updateCompetition](#)
[com.example.football_manager.controller.CompetitionController](#),
[31](#)
[com.example.football_manager.service.CompetitionService](#),
[33](#)
[updateMatch](#)
[com.example.football_manager.controller.MatchController](#),
[41](#)
[com.example.football_manager.controller.MatchViewController](#),
[52](#)
[com.example.football_manager.service.MatchService](#),
[49](#)
[updatePlayer](#)
[com.example.football_manager.controller.AdminPlayerViewController](#),
[25](#)
[com.example.football_manager.controller.PlayerController](#),
[54](#)
[com.example.football_manager.service.PlayerService](#),
[58](#)
[updateProfile](#)

[com.example.football_manager.controller.ProfileViewController](#),
[59](#)
[com.example.football_manager.service.UserService](#),
[75](#)
[updateTeam](#)
[com.example.football_manager.controller.TeamController](#),
[68](#)
[com.example.football_manager.controller.TeamViewController](#),
[72](#)
[com.example.football_manager.service.TeamService](#),
[71](#)
[usersPage](#)
[com.example.football_manager.controller.AdminUserViewController](#),
[27](#)